

DESARROLLO WEB EN ENTORNO SERVIDOR

DESARROLLO DE APLICACIONES WEB

ILERNA

© Ilerna Online S.L., 2023

Maquetado e impreso por Ilerna Online S.L.

© Imágenes: Shutterstock

Impreso en España - Printed in Spain

Reservado todos los derechos. No se permite la reproducción total o parcial de esta obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de los titulares del copyright. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.

Ilerna Online S.L. ha puesto todos los recursos necesarios para reconocer los derechos de terceros en esta obra y se excusa con antelación por posibles errores u omisiones y queda a disposición de corregirlos en posteriores ediciones.

3.ª edición: marzo 2023

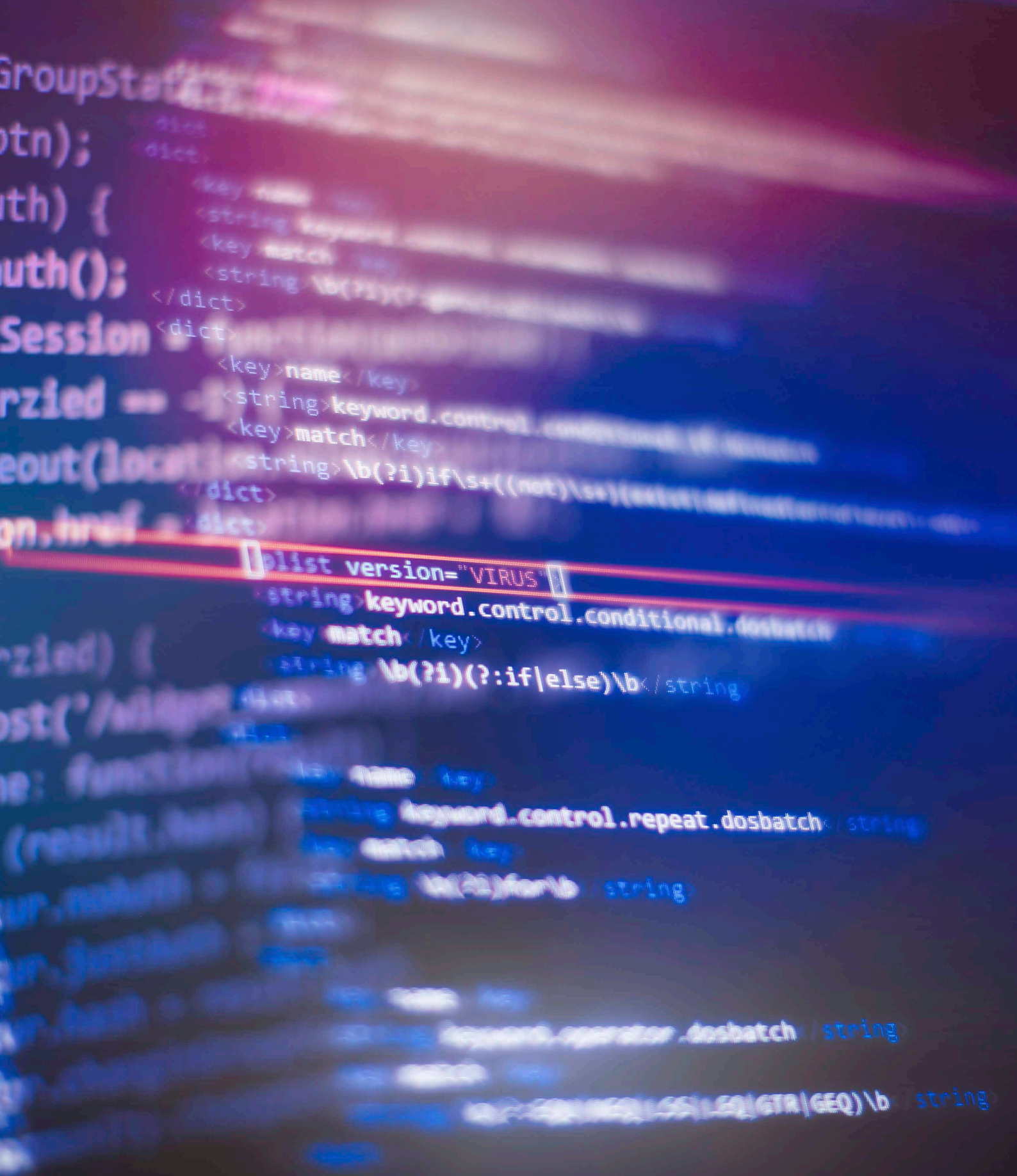
ÍNDICE

Desarrollo web en entorno servidor

1. Selección de arquitecturas y herramientas de programación	6
1.1. Modelos de programación en entornos cliente/servidor	7
1.2. Generación dinámica de páginas web	7
1.3. Lenguajes de programación en entorno web	10
1.4. Integración con los servidores web	11
1.5. Lenguajes de programación en entorno servidor	13
1.6. Integración con los lenguajes de marcas.....	16
1.7. Herramientas de programación	18
2. Inserción de páginas web.....	20
2.1. Obtención de los lenguajes de marcas para mostrar al cliente..	21
2.2. Tecnologías asociadas: PHP, ASP, JSP, miniaplicaciones de servidores (<i>servlets</i>), entre otras	22
2.3. Etiquetas para inserción de código.....	24
2.4. Tipos de datos. Conversiones entre tipos de datos. Sintaxis del lenguaje. Sentencias	26
2.5. Variables	28
3. Programación basada en lenguajes de marcas con código empotrado	32
3.1. Tomas de decisión.....	33
3.2. Bucles	35
3.3. Tipos de datos compuestos.....	37
3.4. Funciones	38
3.5. Recuperación y utilización de información proveniente del cliente web	40
3.6. Procesamiento de la información introducida en un formulario.....	42
3.7. Comentarios	45
4. Desarrollo de aplicaciones web utilizando código empotrado	46
4.1. Mantenimiento del estado	47
4.2. <i>Cookies</i> (galletas).....	49
4.3. Seguridad: usuarios, perfiles, roles. <i>Cookies</i>	51
4.4. Autenticación de usuarios.....	52
4.5. Adaptación a webs existentes	54
4.6. Pruebas y depuración	54

5. Generación dinámica de páginas web	56
5.1. Mecanismos de separación de la lógica de negocio.....	57
5.2. Controles de servidor.....	58
5.3. Mecanismos de generación dinámica de la interfaz web	60
5.4. Configuración	61
5.5. Pruebas y depuración	63
6. Utilización de técnicas de acceso a datos.....	64
6.1. Tecnologías que permiten acceder a datos desde una aplicación web	65
6.2. Establecimiento de conexiones.....	67
6.3. Ejecución de sentencias SQL. Recuperación y edición de información	68
6.4. Ejecución de sentencias SQL. Utilización de conjuntos de resultados	70
6.5. Creación de aplicaciones web con acceso de escritura a bases de datos	71
6.6. Transacciones	74
6.7. Pruebas y documentación	75
7. Programación de servicios web	76
7.1. Características y ámbitos de aplicación.....	77
7.2. Ventajas	78
7.3. Mecanismos y protocolos implicados.....	79
7.4. Generación de un servicio web	82
7.5. Descripción de un servicio	83
7.6. Interfaz de un servicio web. Pruebas.....	84
8. Generación dinámica de páginas web interactivas	86
8.1. Ejecución del código cliente y servidor	87
8.2. Librerías y tecnologías relacionadas.....	88
8.3. Generación dinámica de páginas interactivas	89
8.4. Obtención remota de información. Formularios.....	91
8.5. Modificación dinámica de la estructura de la página web	93
8.6. Programación de aplicaciones web	93
9. Desarrollo de aplicaciones web híbridas	96
9.1. Reutilización del código y de la información	97
9.2. Utilización de información proveniente de depósitos.....	98
9.3. Creación de depósitos	100

9.4.	Incorporación de funcionalidades específicas.....	100
9.5.	Programación de servicios y de aplicaciones web.....	102
9.6.	Pruebas, depuración y documentación	103
Bibliografía / webgrafía		104
Solucionario		105



1

SELECCIÓN DE ARQUITECTURAS Y HERRAMIENTAS DE PROGRAMACIÓN

1.1. MODELOS DE PROGRAMACIÓN EN ENTORNOS CLIENTE/SERVIDOR

En el desarrollo web, se deben distinguir distintos **modelos de programación** dependiendo de dónde se esté realizando la ejecución de código:

- **Programación del lado del servidor:** tecnología que permite interactuar a un cliente con un servidor, realizando peticiones y respuestas respectivamente, para la gestión y consulta de páginas web de manera dinámica.
- **Programación del lado del cliente:** tecnología que consiste en mostrar el contenido recibido por el servidor.

A lo largo del módulo se estudiará todo lo relacionado con la programación en el entorno servidor.

En función del entorno en el que se programe, existen distintos **lenguajes especializados**:

- Si se va a programar del lado del **cliente**, se utilizará JavaScript, XML o *applets*.
- Si se va a programar del lado del **servidor**, los lenguajes utilizados son ASP, PHP, Perl, Python, JSP o *servlets*, entre otros. Estos últimos se estudiarán en profundidad a lo largo del curso.

1.2. GENERACIÓN DINÁMICA DE PÁGINAS WEB

A continuación, veremos las **principales diferencias** entre las páginas web dinámicas y las páginas web estáticas:

- Las **páginas webs dinámicas** son aquellas que, durante su ejecución, cambian la información que muestran al usuario. Es decir, cada vez que un usuario visita la página, el contenido se actualiza mediante programación y bases de datos. Puesto que el contenido se renueva, es necesario que el administrador del sitio web pueda realizar los cambios necesarios en un panel de administración, más conocido como CMS (*content management system*).
- Por otra parte, las **páginas webs estáticas** se desarrollan en HTML y CSS, sin ningún lenguaje de programación. En este tipo de páginas el contenido es fijo, es decir, no se actualiza. Aunque es posible modificar la información, esto requiere de un proceso más laborioso. Para que los cambios se vean reflejados es necesario descargar del servidor el código de la página, modificarlo y volverlo a subir.



Además del HTML y el CSS, existen otras tecnologías que se pueden usar en este tipo de páginas, tales como Ajax o jQuery. Estas herramientas ayudan a que la página resulte más interactiva y más vistosa para el usuario.

CONCEPTO

Es importante aclarar que una **página web dinámica** no es aquella que tiene animaciones; esto solamente significa que utiliza otro tipo de tecnología, como puede ser la tecnología Flash.

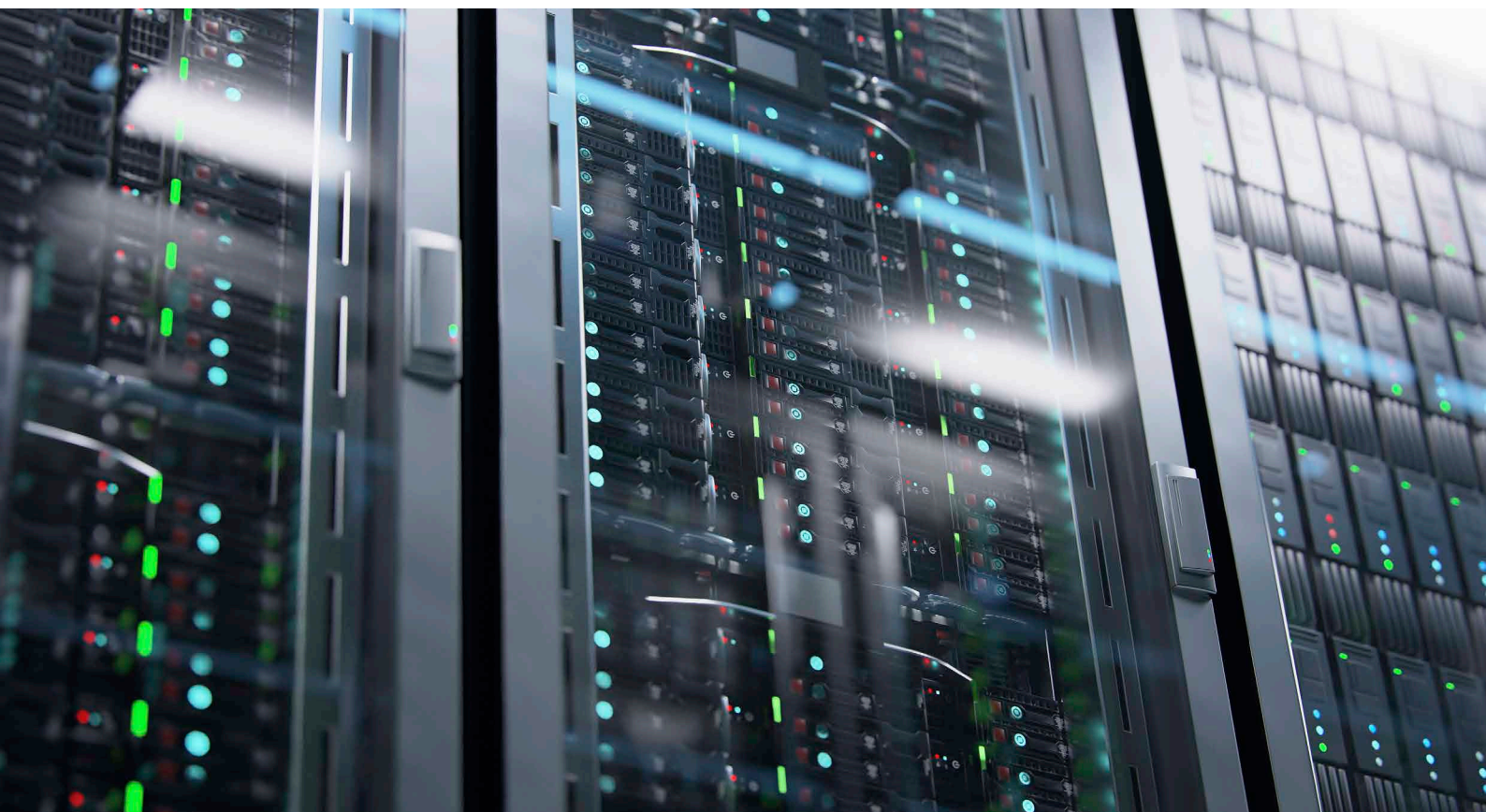
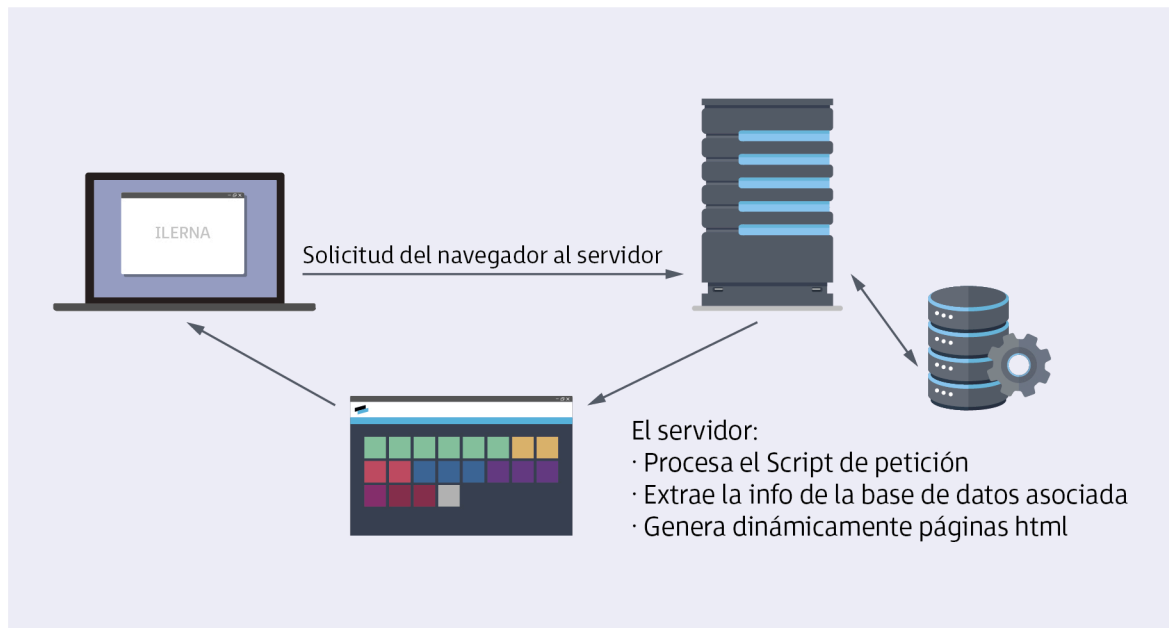
Antes de comenzar el diseño de una página, es necesario reflexionar sobre qué tipo de web es conveniente crear. Para ello, repasaremos las **principales características** de las páginas estáticas y las páginas dinámicas:

- **Características de una página web estática:**

- Se realiza con HTML.
- Es más económica.
- Su desarrollo es más rápido.
- No contiene datos de páginas web.
- La actualización de contenidos es costosa.
- Sin modificaciones.
- El usuario no interacciona con los contenidos ofrecidos en la página web.

- **Características de una página web dinámica:**

- Utiliza algún lenguaje de programación.
- El desarrollo es más complejo, por lo que es más lento y laborioso. Además, el mantenimiento del servidor web también resulta más costoso.
- Permite mostrar datos de fuentes, tales como una base de datos.
- El administrador puede modificar el usuario fácilmente.
- Ofrece la posibilidad de que el usuario modifique la vista.



1.3. LENGUAJES DE PROGRAMACIÓN EN ENTORNO WEB

Existen múltiples **lenguajes** que se utilizan en el desarrollo de una página web. En este apartado se van a estudiar algunos de ellos:



CSS

Es el lenguaje utilizado para las denominadas **hojas de estilo**, es decir, archivos que definen las propiedades de todos los elementos de la web. Esto posibilita la creación de un diseño que hará más atractiva la web.

La función principal de las hojas de estilo es **separar la estructura de la web del diseño**. La versión actual es CSS3.



HTML

HTML es el **lenguaje de marcas** que se encarga de definir el contenido de la página web y cómo este se distribuye en el navegador web.

La versión actual es HTML5, que incluye algunas mejoras en relación con las anteriores versiones, tales como la incorporación de elementos multimedia o la creación nuevas etiquetas.



JavaScript

JavaScript es un **lenguaje de programación interpretado** que se encarga de aportar funcionalidad a HTML. Gracias a ello, es posible que el usuario pueda interactuar con la página web.



Applets

Los *applets* son **pequeños programas creados con Java** a los que se puede acceder a través de un navegador web, aunque son independientes del mismo. Por este mismo motivo, no es posible interactuar con otros elementos de la web. En este caso, ofrecen más funcionalidades que una web creada con JavaScript.



Flash

Flash es una plataforma que utiliza el lenguaje de programación **orientado a objetos ActionScript** para crear animaciones y contenido multimedia en las páginas web.



XML

XML es un lenguaje utilizado para el **intercambio de datos de forma legible** entre distintas aplicaciones, ya formen parte de la misma plataforma o de una plataforma distinta. La utilización de este lenguaje permite compartir información de forma libre, fiable y fácil.



JSON

Al igual que XML, es un lenguaje utilizado para el **intercambio de datos de forma legible** entre aplicaciones multiplataforma, pero su uso es menor en el entorno web.

JSON realiza este intercambio mediante un **análisis sintáctico**, más conocido como **parser**. *Parser* es más sencillo que las etiquetas XML, lo que supone una gran ventaja.

1.4. INTEGRACIÓN CON LOS SERVIDORES WEB

Antes de conocer cómo funcionan las peticiones a un servidor web, es necesario conocer cómo se accede a este servidor.



¡RECUERDA!

Para **acceder a un servidor web** es necesario que el usuario especifique el recurso al que quiere acceder y, para esto, debe indicar la dirección URI correspondiente.

La **dirección URI** es el identificador único de los recursos de la web, teniendo en cuenta que estos recursos pueden variar a lo largo del tiempo.

La **estructura** de una dirección URI es la siguiente:

- **Esquema:** especifica el protocolo de acceso al recurso. Suele ser el protocolo de internet HTTP o HTTPS si es de modo seguro.
- **Autoridad:** identifica la información necesaria para acceder al servidor, es decir, el dominio en el que se encuentra.
- **Ruta:** es el camino que se debe seguir una vez que se ha accedido al servidor para poder llegar al recurso. Se puede ver como la ruta que existe en el ordenador para llegar a un fichero especificado.

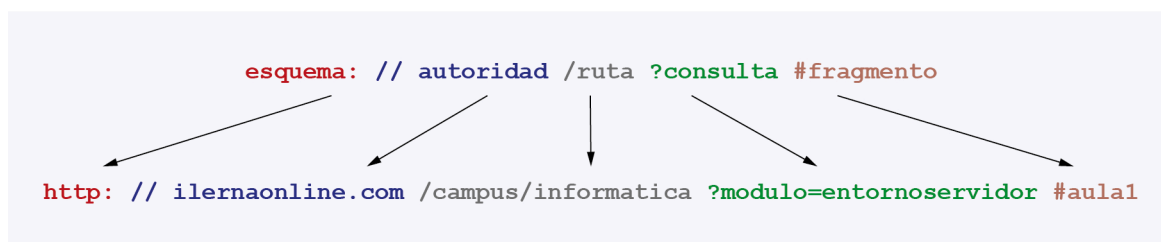


ponte a prueba

¿Qué es el HTML5?

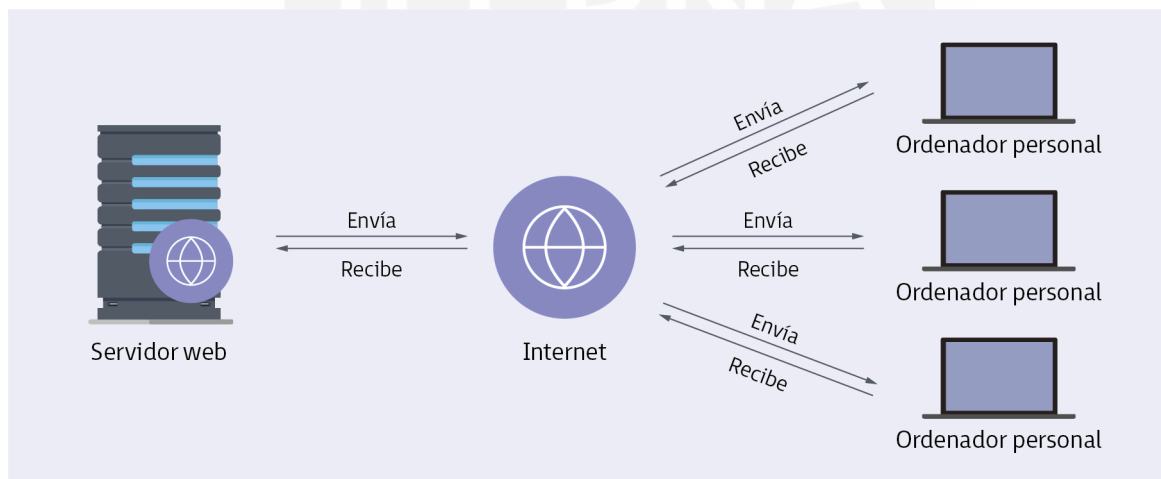
- Un lenguaje que se utiliza principalmente para crear hojas de estilo.
- Un lenguaje de programación interpretado que permite interactuar con una web.
- Una versión avanzada del HTML que sirva para intercambiar datos entre aplicaciones.
- Un lenguaje de marcas para definir el contenido de una página web.

- **Consulta:** es la información referida a una especificación del recurso. Tanto la autoridad como la ruta se refieren a información jerárquica; la consulta, en cambio, la conforman conjuntos de clave=valor que identifican el recurso en el servidor. Este campo de la URI se precede del símbolo **?**.
- **Fragmento:** identifica una parte del recurso. Se considera una subdirección dentro de la dirección URI. Este campo está precedido por el símbolo **#**. Hay que tener en cuenta que este campo es el que diferencia una dirección URL de una dirección URI.



Petición de un recurso a un servidor

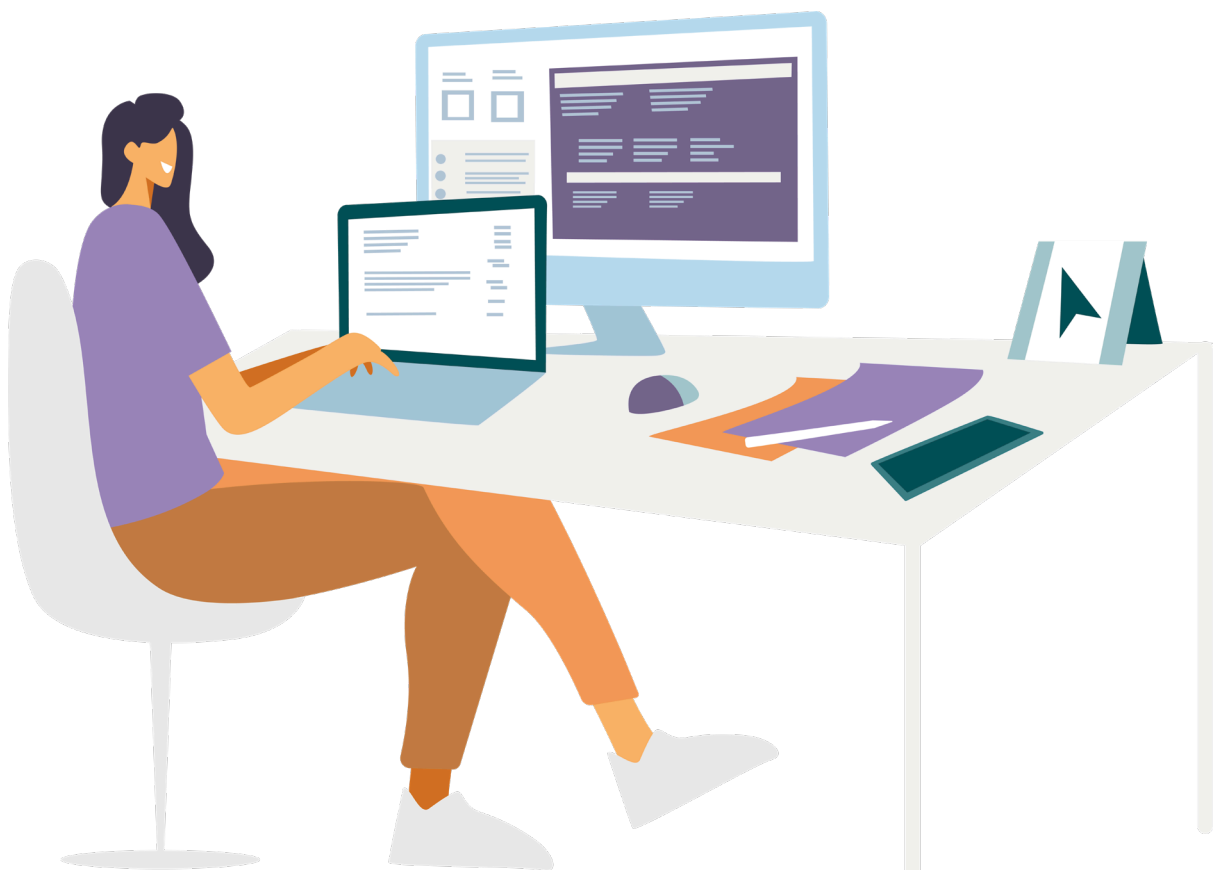
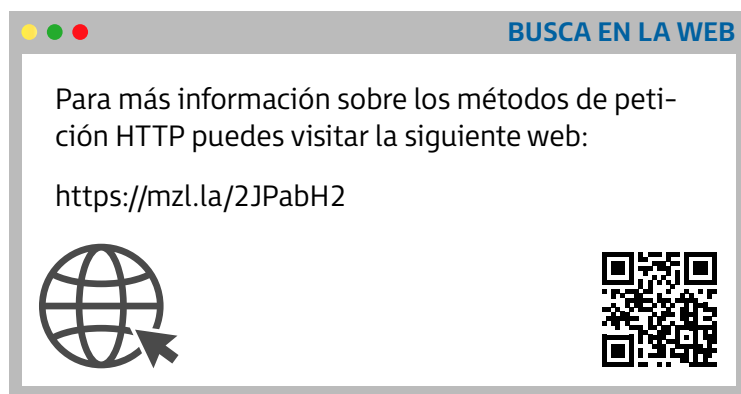
A continuación, se explicará cómo funciona la **comunicación entre el cliente y el servidor** cuando se realiza una petición para acceder a un recurso:



1. Para que un usuario realice una petición al servidor web, debe indicar la URL a la que quiere acceder en el navegador web. El propio navegador descodifica esta URL para conocer cada uno de los campos de esta dirección, y abre una conexión TCP/IP con el servidor correspondiente.
2. Después, es posible el envío de mensajes del protocolo HTTP. Como se verá posteriormente, HTTP tiene tres tipos de mensajes.
3. Por último, el servidor web devolverá la respuesta correspondiente al cliente y se cerrará la conexión TCP.

El protocolo HTTP tiene **nueve tipos de mensajes** para comunicarse con el servidor:

- **GET**: recoge datos del servidor.
- **HEAD**: recoge información sobre el recurso.
- **POST**: envía datos al servidor.
- **PUT**: reemplaza todas las representaciones actuales del recurso de destino con la carga útil de la petición.
- **DELETE**: borra un recurso en específico.
- **CONNECT**: establece un túnel hacia el servidor identificado por el recurso.
- **OPTIONS**: describe las opciones de comunicación para el recurso de destino.
- **TRACE**: realiza una prueba de bucle de retorno de mensaje a lo largo de la ruta al recurso de destino.
- **PATCH**: aplica modificaciones parciales a un recurso.





1.5. LENGUAJES DE PROGRAMACIÓN EN ENTORNO SERVIDOR

Tal y como se ha visto con anterioridad, dentro del servidor existen distintas tecnologías. Citamos sus **principales características**:

PHP

PHP es un lenguaje de programación que permite la **inserción de código en el servidor web**. Es posible realizarlo mediante archivos con extensión **.php**, mediante etiquetas **<?php** para abrir la etiqueta y **?>** para cerrarla. Permite ser incrustado en un HTML, por este motivo es una tecnología muy utilizada en el desarrollo web.

La forma más común de usar PHP es **incrustado en el código HTML**. Es importante destacar que PHP no es un lenguaje compilado, sino que es ejecutado en el servidor y permite generar un HTML que posteriormente es enviado al cliente.

```
1      <!DOCTYPE html>
2
3      <html lang="es">
4
5      <head>
6      <title>
7      |      Programación en
entorno servidor
8      </title>
9      </head>
10
11     <body>
12     <h1>
13     |      <?php
14     |      echo "Prueba de PHP";
15     |      ?>
16     </h1>
17
18     </body>
19
20     </html>
```

Perl

Perl es un lenguaje de programación especialmente centrado en el desarrollo de programas que permitan la comunicación entre cliente y servidor, comúnmente llamados **CGI** (*common gateway interface*).

Esta comunicación retorna como resultado **objetos MIME**. Las aplicaciones CGI fueron unas de las primeras en la creación de páginas web dinámicas.

Perl es un lenguaje de programación **interpretado**, por lo que cada petición del cliente pasa por la interpretación y ejecución de los *scripts* correspondientes de código. Estos archivos se identifican por su extensión **.pl**.

```
#!/usr/bin/perl

print "Content-type: text/html\n\n";
print "<html>";
print "<head>";
print "<title> Programacion entorno
servidor </title>"
print "</head>";
print "<body>";
print "<h1> Prueba perl </h1>";
print "</body>";
print "</html>";
```



Python

Python es un lenguaje de programación **interpretado**. Es poderoso y fácil de entender. Permite hacer uso de la programación **orientada a objetos**.

Es un lenguaje considerado de alto nivel, que utiliza **frameworks** que se integran para permitir el desarrollo de aplicaciones web. El paradigma de programación es muy diferente a otros, como PHP, y, además, es más abstracto. La extensión de sus archivos es **.py**.



ASP

Tecnología para entornos web desarrollada y comercializada por Microsoft para la creación de páginas web dinámicas. Hace uso del popular **.NET Framework** cuyo objetivo es facilitar la rápida creación de sitios web.

Sus archivos tienen extensión **.asp**. ASP es una **tecnología dinámica que se ejecuta del lado del servidor**, por lo que, al igual que en PHP, es necesario ejecutar los *scripts* para la generación de HTML devuelto al cliente como respuesta a sus peticiones. El lenguaje utilizado en esta tecnología es **C#**.



JSP

Tecnología orientada a desarrollar páginas web con programación Java. Su principal objetivo es **crear aplicaciones web que se ejecuten en diferentes servidores**. El desarrollo de este lenguaje implica la instalación de un motor JSP.

Este motor está basado en los **servlets de Java**. Se puede identificar una página creada en JSP porque su extensión será **.jsp**. Estos archivos contienen las diferentes sentencias Java que se deben ejecutar en el servidor. JSP es un lenguaje **compilado**.



Servlets

Se trata de un **objeto Java dentro del lenguaje de programación Java**. Su finalidad es la creación de páginas web. Un *servlet* es un pequeño programa que admite peticiones bajo la utilización de protocolo HTTP.

Son la capa intermedia entre los navegadores y la base de datos. Se encargan de procesar las peticiones realizadas por los clientes para devolver como resultado un HTML. Estos programas suelen estar incluidos en servidores web como Apache o **Apache Tomcat**.

1.6. INTEGRACIÓN CON LOS LENGUAJES DE MARCAS

En ocasiones, el contenido que aparece en una misma página cambia dependiendo del navegador en el que se ejecuta. Esto obliga a realizar un mayor esfuerzo, ya que lo ideal es desarrollar una **web que se adapte correctamente a todos los navegadores**.

La aparición de nuevas plataformas de conexión lo convierte en un proceso casi imposible de satisfacer. Para dar una solución a este problema, apareció un tipo de arquitectura denominada CGI.

ponte a prueba

El objetivo de CGI es hacer más compleja la interacción de los usuarios con las webs.

- a) Verdadero
- b) Falso

PARA + INFO

El **objetivo de CGI** es hacer más fácil la interacción de los usuarios con las webs para ofrecer contenido dinámico en las mismas.

Como se ha comentado anteriormente, CGI fue la primera arquitectura que permitió la creación de webs dinámicas. Además, permite ofrecer un esquema de peticiones entre cliente y servidor.

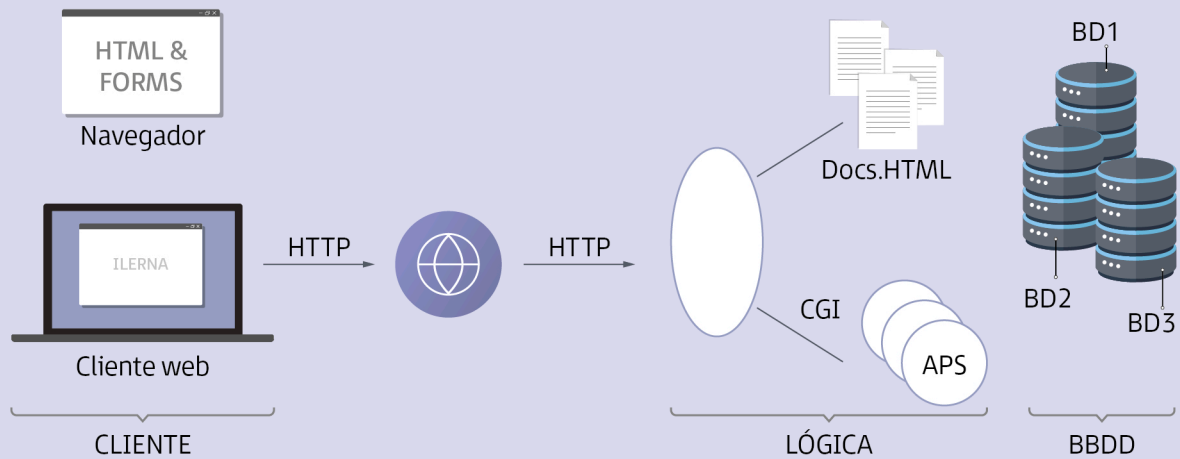
El cliente web realiza una petición usando el protocolo HTTP. El módulo CGI, a partir de esa petición que contiene los parámetros, se encarga de realizar la comunicación con la base de datos. La base de datos devolverá la información al módulo CGI, que, con esos datos recibidos, generará el resultado (normalmente, un documento HTML).

Esta característica de **dinamismo** se puede ver en los valores recibidos por parte del módulo, el cual, dependiendo de los valores que el cliente ha introducido, determinará una operación u otra.

Por otro lado, este tipo de arquitectura presenta algunas **vulnerabilidades**. De hecho, es difícil controlar el acceso a los servicios, ya que se trata de una comunicación sencilla de implementar y que el contenido viaja vía HTTP. Además, también es complejo determinar qué elementos son accesibles, por lo que la seguridad es un aspecto que no queda garantizado.

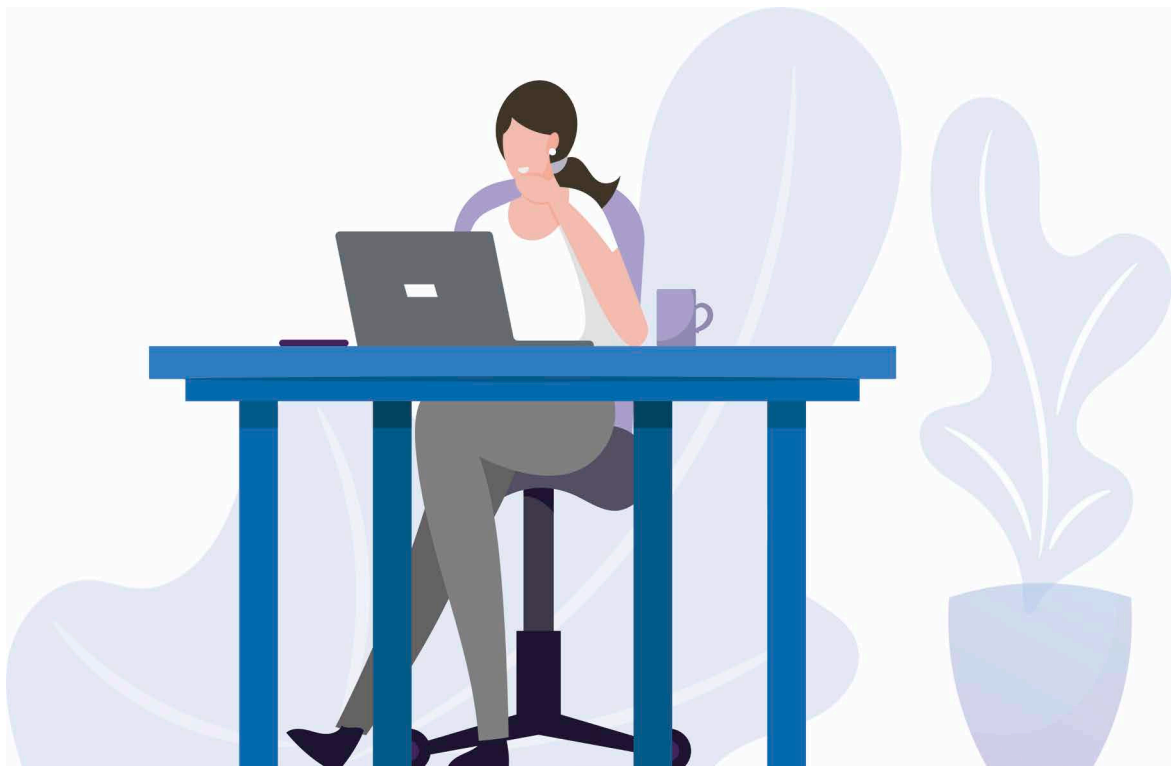
La llegada masiva de nuevas tecnologías y dispositivos ha incrementado el número de peticiones y, por ese motivo, resultan difíciles de gestionar. Eso hace que CGI presente problemas en cuanto a la escalabilidad. CGI es un protocolo que no almacena el estado, por eso, una vez que se ha realizado una petición, no será almacenada, ni tampoco los resultados obtenidos.

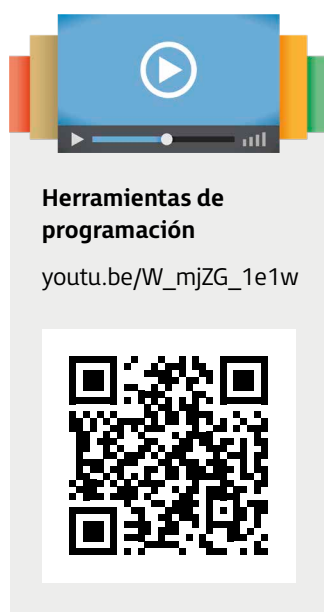
La **construcción de una invocación** de CGI se hace manteniendo los formatos de construcción de una dirección URL.



Existen diferentes **tipos** de CGI:

- **Contador de accesos:** cuenta el número de veces que se ha solicitado una página determinada y guarda este valor en un fichero. Cada vez que se invoca, se incrementa.
- **Buscador:** busca páginas que contengan coincidencias con un valor especificado y devuelve la localización encontrada.
- **Correo:** recoge los datos del usuario.
- **Colaboración:** añade la posibilidad de generar nuevos enlaces o contenido en una determinada web y devuelve la localización del contenido.
- **Estadísticas:** presenta un registro de *log* en un archivo que guarda las distintas estadísticas recogidas.
- **Administración:** permite interactuar con el servidor y gestionar su comportamiento.





1.7. HERRAMIENTAS DE PROGRAMACIÓN

A la hora de escribir código, en cualquier lenguaje, es imprescindible elegir cuáles serán las herramientas de construcción de estos ficheros de código.

A continuación, se exponen las **características** de las herramientas de programación:

- Poseen una gran cantidad de temas que permiten personalizar y diferenciar cada uno de los componentes de un lenguaje.
- Permiten diferenciar métodos, variables, constantes, comentarios, etcétera.
- Ayudan a visualizar el código de una manera más limpia y ordenada. Para ello, cuentan con algunas herramientas que hacen más fácil la programación.
- Permiten la integración de *plugins* de lenguaje, funciones de autocompletado y sugerencias, tabulaciones automáticas de código, resaltar paréntesis y llaves concordantes, etcétera.
- Reconocen la sintaxis de la mayor parte de los lenguajes de programación.

Existen diferentes **tipos de herramientas** de programación:

- **Editor de texto plano:** programa que permite crear y modificar archivos de texto. Mediante caracteres especiales, puede ofrecer algo de formato al archivo; este tipo de caracteres se denominan caracteres de control. Un ejemplo de estos programas es Sublime Text.
- **IDE:** programa que proporciona, en un mismo entorno, todas las herramientas necesarias para el desarrollo completo de una web. La elección de uno de ellos, muchas veces, viene determinada por ser un *software* libre o uno privado. En otras ocasiones, viene determinada por la integración y las facilidades que se ofrecen con el lenguaje de programación empleado. Algunos de los IDE más utilizados son Eclipse, Netbeans y Microsoft Visual Studio.



¡RECUERDA!

Es **importante** tener en cuenta que todo IDE tiene incluido un fichero de texto plano en el que se inserta el código.





2 INSERCIÓN DE PÁGINAS WEB

2.1. OBTENCIÓN DE LOS LENGUAJES DE MARCAS PARA MOSTRAR AL CLIENTE

Cuando un cliente lanza una petición a un servidor web cuyo contenido es una página con código del servidor, implica que este sea interpretado antes de ser mostrado al cliente. Si no se realiza este proceso, la información es ilegible para el cliente. Por ese motivo, es necesario que, a medida que se interpreta el código, se vaya generando el código resultante en HTML.

Este es el **proceso**, explicado de forma más detallada:

1. El usuario ejecuta un evento y el navegador realiza una petición al servidor correspondiente a través de la URL bajo el protocolo HTTP.
2. El servidor recibe la información y la procesa: se van ejecutando todos los archivos o *scripts* necesarios y esto genera el correspondiente código HTML.
3. Una vez que se ha obtenido el resultado, se envía el código HTML completo al navegador del lado del cliente, quien se encargará de mostrárselo al usuario.

El proceso de obtención del código HTML es relativamente sencillo y depende de la complejidad del lenguaje de programación empleado, que puede ser más o menos fácil de comprender.

Básicamente, ambos archivos de código presentan una serie de etiquetas que describen el tipo de elemento que es interpretado. Este código será gestionado por un controlador que enlazará los datos obtenidos del servidor para su marcado en la web.

Las **etiquetas** son fáciles de transformar, pues permiten deducir el tipo de elementos que se representarán. Por el contrario, la parte de interpretación de eventos del servidor puede resultar más complicada.

Para ello, se utilizan lenguajes como **JavaScript**, que permite la creación de la estructura asociada a un evento. Esto dotará de dinamismo a una página web y, a su vez, producirá una serie de cambios en la web ofrecida denominados **roundtrip**.

CONCEPTO

Roundtrip es un proceso en el que una web es modificada por la ejecución desencadenada por un evento, que puede producirse por acciones como pulsar un botón, seleccionar diferentes elementos de un desplegable, etcétera.

El problema de este proceso es que cada petición genera una nueva web, lo que supone una carga adicional para el servidor cuando el número de peticiones es alto. Para hacer frente a este problema, se hace uso de lenguajes como **AJAX**.

2.2. TECNOLOGÍAS ASOCIADAS: PHP, ASP, JSP, MINIAPLICACIONES DE SERVIDORES (*SERVLETS*), ENTRE OTRAS

En el apartado anterior se ha detallado el proceso de obtención de HTML en el lado del cliente. Estos procesos que se repiten de manera periódica es lo que se conoce como **ciclo de vida de una página**.

El **ciclo de vida** de una página dinámica consta de los siguientes pasos:

- **Inicialización:** ejecución de los *scripts* en cliente y servidor.
- **Instanciación** de los controles de la página.
- **Mantenimiento** de estado de la página.
- **Ejecución de los eventos** asociados a una página.
- **Envío de la página** al cliente.



Dependiendo del lenguaje de programación, este ciclo de vida puede suponer diferentes etapas.

Por ejemplo, PHP lo divide en **dos etapas**:

1. El proceso de inicio y cierre del motor PHP

Esta fase se encarga de arrancar el motor PHP donde se inician todos los módulos declarados en el fichero **PHP.ini**.

Esto asignará los recursos correspondientes que se utilizarán durante todo el ciclo de vida, mientras se ejecuta el motor PHP. Una vez que este se ha cerrado, se liberan.

2. El ciclo de peticiones

En esta segunda fase se recogen las peticiones realizadas sobre un determinado recurso. En este punto se comunica al navegador, que realiza la petición al servidor, ejecuta los *scripts* y el intérprete se encarga de recoger la respuesta de la base de datos y enviarla vía URL de nuevo hasta el cliente.

Lenguajes como ASP o ASP.NET (versión que actualiza a ASP) se fundamentan en estos mismos principios, diferenciándose únicamente en la ejecución de los métodos propios de ese lenguaje para la gestión de peticiones.

Uno de los lenguajes que realizan este proceso de manera diferente es JSP (Java Server Page). Las peticiones realizadas a través de las páginas web basadas en esta tecnología no son interpretadas, sino compiladas. Estas peticiones son gestionadas por pequeños programas denominados **servlets**, que se encargan de procesar una petición y generar la página web que se mostrará posteriormente.

El ciclo de vida de esta tecnología se puede dividir en **tres fases**:

1. **Traducción:** cada página JSP solicitada se traduce en código del *servlet*, básicamente Java. Se hace uso de las directivas propias de este lenguaje para realizar la compilación de dicha página.
2. **Compilado:** es la fase propia que aplica la compilación del *servlet* generado.
3. **Ejecución:** el resultado de la compilación se ejecuta.

Como se ha podido comprobar, cada tecnología emplea una serie de mecanismos que permiten obtener como resultado una página web.



ponte a prueba

Verdadero o falso: El JSP es un lenguaje interpretado que para realizar peticiones al servidor y devolver la respuesta al navegador a través de una URL.

- a) Verdadero
- b) Falso

2.3. ETIQUETAS PARA INSERCIÓN DE CÓDIGO

Cualquiera de los lenguajes de programación de los que se vaya a hacer uso a la hora de programar se basa en la aplicación de unas determinadas etiquetas que permiten identificar los elementos que se van a representar. De esta forma, el servidor puede entender cuál es la estructura que se debe generar.

Normalmente, se emplean los símbolos `<%` y `%>`, que permiten delimitar las secciones de código que se deben interpretar. Es el equivalente a indicar respectivamente el comienzo y el fin de la interpretación del código.

Las **etiquetas en PHP** son representadas en diferentes formatos a la hora de insertar fragmentos de código de servidor dentro de una página web. Esto significa que es posible realizar funciones propias de un servidor dentro de una web, de esta forma, se puede aliviar de sobrecarga al servidor. Por el contrario, esta práctica obliga a realizar configuraciones adicionales para asegurar que la ejecución se llevará a cabo de forma correcta.

Esto no es algo obligatorio, es posible utilizar las etiquetas que aparecen en la documentación de PHP para indicar en el HTML que se trata de una representación en dicho lenguaje. Estas etiquetas son `<?php` y `?>` (para apertura y cierre respectivamente).

```
<!doctype html>
<html lang="en">
<head>
  <meta http-equiv="X-UA-Compatible" content="IE-edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>
    <?php
      if(isset($nombreVariable)) {
        echo $nombreVariable . ' Este es el nombre de la variable ';
      }
    ?>
  </title>
</head>
<body>
  Cuerpo del HTML
</body>
</html>
```

Las **etiquetas JSP** se distinguen por su aparición, estas pueden ser:

- **Directivas:** este tipo de etiquetas se utilizan para indicar al JSP los pasos a seguir para llevar a cabo el procesamiento.

- **Scripting:** porciones de código, por lo general en lenguaje Java, que se ejecutarán una vez que la página sea procesada.
- **Comentarios:** sirven para indicar líneas de texto que no serán interpretadas. Es posible visualizarlas en la compilación obtenida como resultado.
- **Acciones:** determinan las interacciones que se van a realizar, como puede ser la redirección a otra página, la especificación de ejecución de un *servlet* o la interacción con otros componentes externos.

Las etiquetas en ASP.NET siguen las mismas pautas que lenguajes como PHP. ASP.NET emplea los símbolos `<%` y `%>` para la inserción de código. Como se puede comprobar, este tipo de práctica es muy habitual a la hora de insertar código en las aplicaciones web.

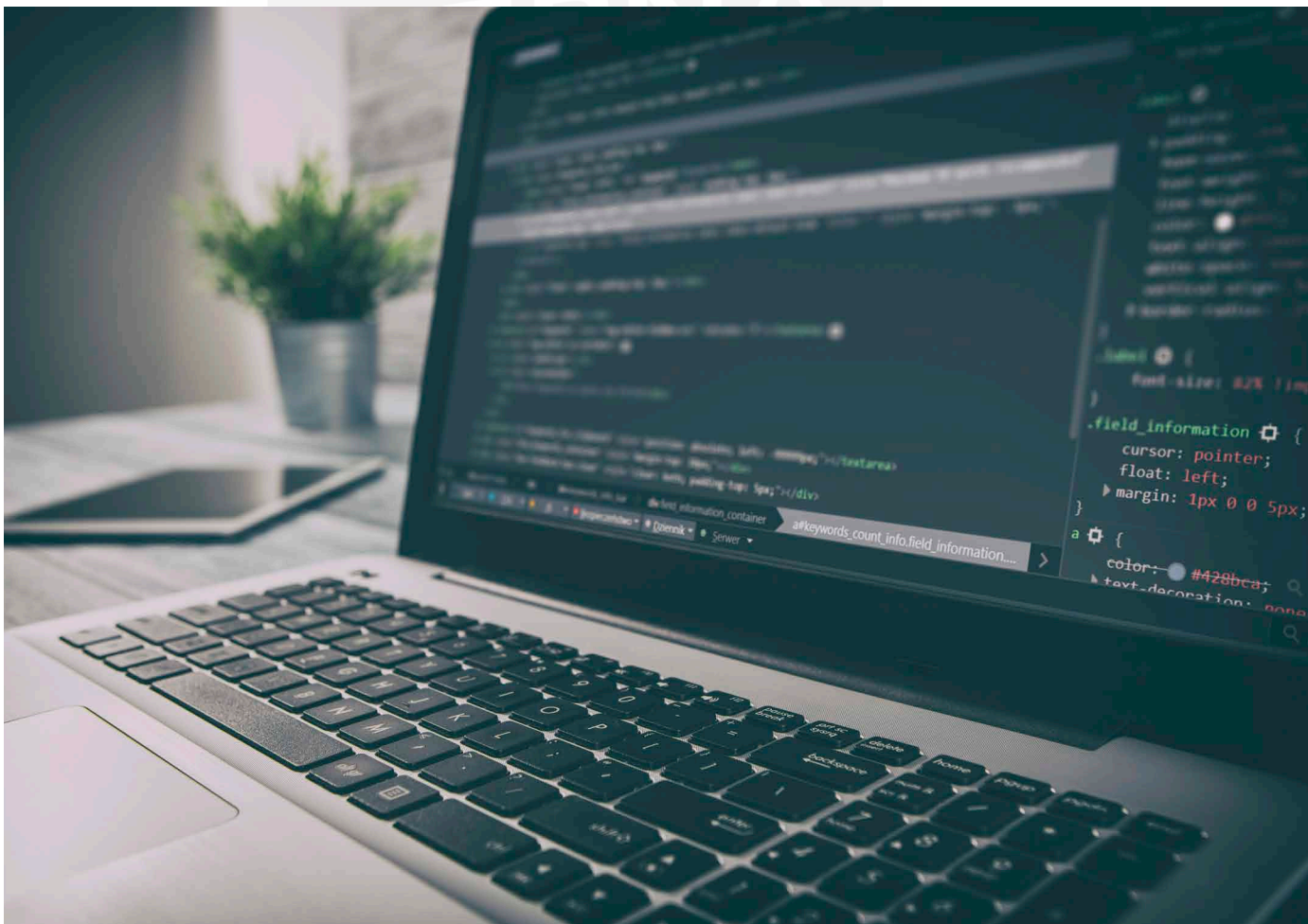
Actualmente es posible, además, utilizar etiquetas de carácter general que son interpretadas en HTML para señalar la inserción de código, como es `<script>`. Estas etiquetas permiten, a través de un parámetro, especificar el lenguaje del código que se empleará dentro de ellas. De esta forma es posible facilitar aún más el desarrollo de dichas partes dentro de una página web.



ponte a prueba

¿Cuáles son las etiquetas que abren y cierran un fragmento de código PHP dentro de un documento HTML?

- `<?php y php?>`
- `<php? y ¿php>`
- `<?php y ?>`
- No se puede declarar código PHP dentro de documentos HTML.



2.4. TIPOS DE DATOS. CONVERSIONES ENTRE TIPOS DE DATOS. SINTAXIS DEL LENGUAJE. SENTENCIAS

CONCEPTO

Un **tipo de dato** es un atributo de los datos que indica la forma en la que un compilador o intérprete comprenderá un dato.

De esta forma, cada tipo de dato permite una serie de operaciones que se pueden ejecutar sobre él.

CONCEPTO

Los **tipos de datos primitivos** son los tipos de datos originales propios de un lenguaje de programación.

Estos tipos de datos ofrecidos por el lenguaje de programación permiten a los programadores construir estructuras de datos y tipos abstractos de datos más complejos posteriormente.

Los **principales tipos de datos** usados en la programación web son:

- **Char** (carácter).
- **Int** (entero).
- **Float** (real - coma flotante).

Además, existen otros tipos de datos que, aunque dependiendo del lenguaje se pueden o no considerar primitivos, se encontrarán habitualmente:

- **Booleano** (valor lógico: verdadero o falso).
- **String** (cadena de caracteres).
- **Puntero** (dirección de memoria - *int*).

El proceso de conversión de un dato consiste en cambiar el tipo de dato asociado a una variable de uno a otro.

Existen algunos lenguajes, **como PHP, que transforman automáticamente de un tipo a otro siempre que sea posible**. La función **settype()** de PHP permite hacerlo de forma permanente.

```
$nombreVariable = 'valor inicial';  
$nombreVariable = 34 //La variable $ nombreVariable cambió de tipo
```

Otros lenguajes como **C#** permiten conversiones de tipo implícitas y explícitas. Java emplea el mismo mecanismo para realizar las conversiones de sus variables.

En cuanto a la sintaxis empleada para la programación, cabe destacar que depende de cada lenguaje. Esta sintaxis permite que el programador conozca el conjunto de reglas que posibilitan al compilador comprender el código que se está desarrollando. Para que este reconozca, además, la aplicación como válida, es necesario seguir el tipo de estructura que viene marcado por dicho lenguaje.

Si estas reglas no son cumplidas no será posible compilar correctamente una aplicación.

En el caso de las aplicaciones web, la idea es la misma: para aquellas páginas que emplean lenguajes como Java (que deben ser compilados) es necesario seguir estas normas. Si el lenguaje es interpretado, es posible que se consiga mostrar el resultado, pero su visualización no aparecerá de forma correcta. Esto puede llevar a que se produzcan eventos indeseados.

Esta sintaxis es algo que, dependiendo del lenguaje, puede llevar diferentes periodos de tiempo a los programadores. Algunos lenguajes, como Java, tienen una estructura que, en un primer momento, resulta algo difícil, pero que, una vez comprendida, permite un gran campo de acciones.

Es importante **diferenciar una línea de código de una sentencia**, ya que su significado no es el mismo. Una sentencia es toda aquella parte de código escrita hasta la finalización de la misma a través de un símbolo establecido en la sintaxis de ese lenguaje, de forma general, este suele ser el punto y coma “;”, por lo que una sentencia puede estar formada por varias líneas de código.



2.5. VARIABLES

Una variable se puede definir como un **espacio reservado en el sistema de almacenaje de la memoria principal de un ordenador**. Dicho espacio está asociado a un identificador.

Las variables pueden ser **constantes o variables**:

- **Constantes:** una constante es un identificador para un valor simple, el tamaño de dicha variable no cambiará durante la ejecución del programa. Por defecto, una constante es *case sensitive* (distingue entre mayúsculas y minúsculas). Es una buena praxis declarar los identificadores de constantes en mayúsculas.

Sintaxis:

```
define('nombre constante', 'valor');
```

Ejemplo:

```
define('VALOR_MAXIMO', '60');
echo VALOR_MAXIMO;

//A partir de la versión PHP 5.3.0
const PROYECTO = 'Ilerna Online';
echo PROYECTO;

//A partir de la versión PHP 5.6.0
const PROYECTO2 = PROYECTO.'; Lleida';
echo PROYECTO2;

const PERROS = array('golden', 'boxer',
    'podenco');
echo PERROS[0];

//A partir de la versión PHP 7
define(= array(
    'Roger Federer',
    'Rafael Nadal',
    'Novak Đoković'
));
echo TENISTAS[1];
```

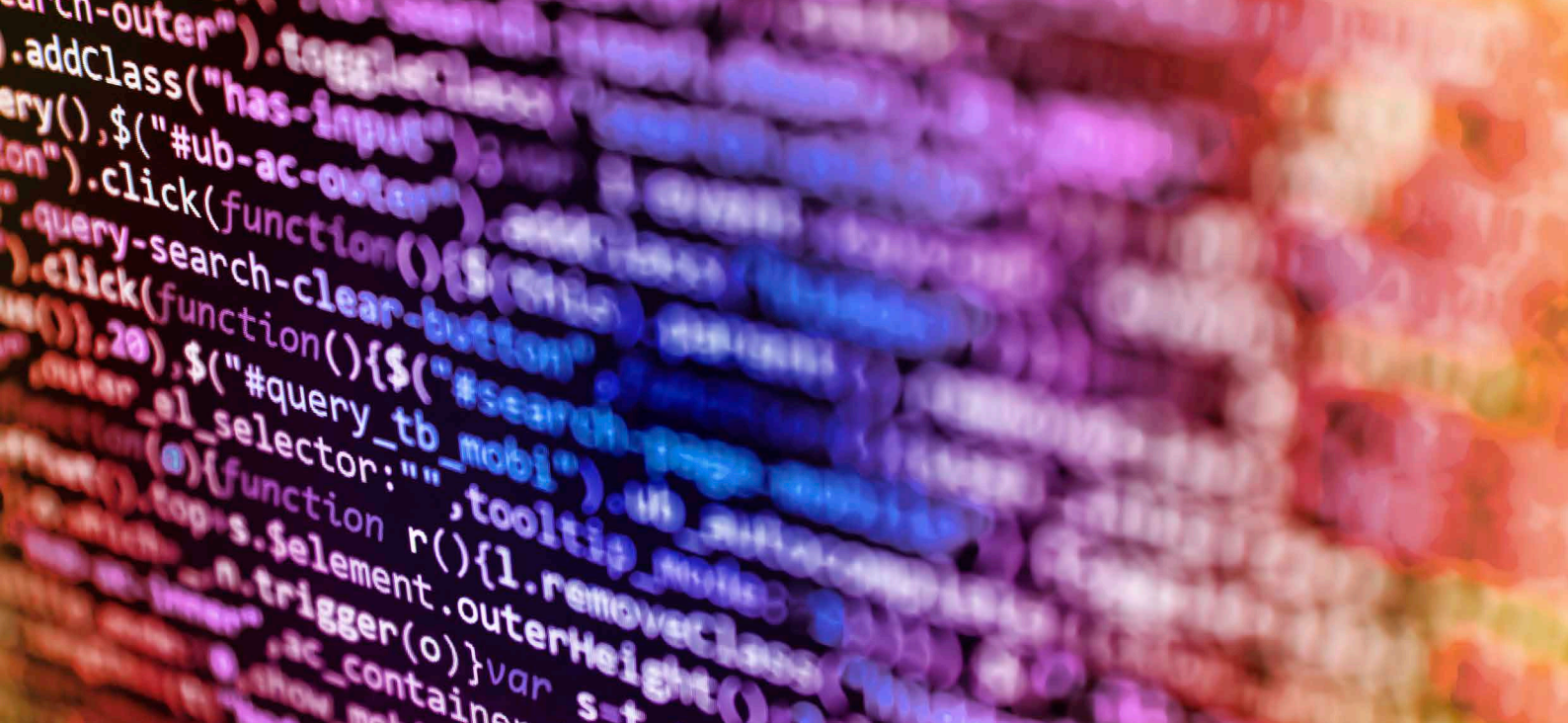
- **Variable:** no es posible predecir el valor que va a contener y este puede ser modificado durante la ejecución de un programa.

Sintaxis:

```
$nombreVariable = 'Valor';
```

Ejemplo:

```
$var = 33;
```



Es posible **clasificar las variables** dependiendo de su tiempo de vida o duración en la ejecución de un programa:

- **Globales:** serán visibles dentro de todo el programa. El tiempo de vida de este tipo de variables pasa a ser el tiempo que dure la ejecución de un programa.
- **Locales:** estas serán visibles dentro de una clase o una sentencia, por lo que su tiempo de vida depende del tiempo de ejecución de dicha clase o sentencia.
- **Estáticas locales:** serán visibles dentro de una clase o sentencia. Su tiempo de vida será lo que dure la ejecución de un programa.

Un aspecto muy importante de las variables es su ámbito. De esta forma, podemos hablar de ámbito local y ámbito global.

Las variables utilizadas dentro de una función (locales) son diferentes de aquellas que se utilizan fuera de ella.

Vamos a ver un ejemplo que ilustra los diferentes tipos de ámbitos de las variables y cómo afecta este al valor de dicha variable:

```
function mitad($a) {  
    $var = $a / 2;  
}  
$num = 6;  
mitad($num);  
echo "El valor de la variable \ $var es: $var";
```

En este primer ejemplo, el valor de la variable `$var` no se muestra por pantalla, ya que dicha variable tiene un ámbito local en la función `mitad`, siendo solo visible dentro de esta.

Como podemos ver en la siguiente imagen, el resultado de la página no muestra valor:

Notice: Undefined variable: var in **C:\xampp\htdocs\M07\pruebas\prueba.php** on line 8
El valor de la variable \$var es:

Para utilizar valores locales de funciones, disponemos de varias opciones. La forma más sencilla de utilizar el valor de una variable local es devolver el valor de dicha variable. Esta acción se realiza con la palabra clave *return*.

```
function mitad($a) {
    return($a / 2);
}
$var = 6;
$var = mitad($var);
echo "El valor de la variable \$var es: $var";
```

En este segundo ejemplo, vemos cómo la variable *\$var* sí que devuelve el resultado de la función:

El valor de la variable \$var es: 3

Otra forma de manejar los valores de variables es trabajar con variables globales.

Este tipo de variables disponen de varias sintaxis de código para trabajar con ellas. La primera forma es utilizar la palabra *global*.

```
function mitad() {
    global $var;
    $var = $var / 2;
}
$var = 6;
mitad($var);
echo "El valor de la variable \$var es: $var";
```

Como podemos ver en el código anterior, declaramos una variable global dentro del programa sin que esté contenida en ninguna función ni clase y la referenciamos dentro de funciones o clases con la palabra clave *global*. De este modo, podemos trabajar con esta variable y mostrar su valor en cualquier zona del programa.

Otra forma de sintaxis para trabajar con las variables globales es utilizando *\$GLOBALS* en lugar de la palabra reserva *global* del ejemplo anterior.

```
function mitad() {  
    $GLOBALS['var'] = $GLOBALS['var'] / 2;  
}  
$var = 6;  
mitad($var);  
echo "El valor de la variable \ $var es: $var";
```

Las variables, al igual que los tipos de datos, vienen determinadas por cada lenguaje, de forma que pueden diferir dependiendo del lenguaje de programación utilizado.



¡RECUERDA!

Para poder utilizar una variable, primero se debe declarar.



¡RECUERDA!

Hay que tener en cuenta que el uso de las variables globales no es una buena praxis de programación, el uso excesivo de este tipo de variables puede conllevar errores de código y una mayor dificultad para realizar su mantenimiento.

Algunos lenguajes trabajan como **lenguajes fuertemente tipados**, esto quiere decir que la declaración de un tipo de variable obliga a ese tipo a contener un valor del tipo especificado, como, por ejemplo, ASP.NET.

Sin embargo, el lenguaje PHP, al ser interpretado, permite una mayor flexibilidad en la declaración de variables, el tipo de dato será asignado en función del valor que contiene esa variable.

El uso de variables se repite de manera continuada durante la codificación de las diferentes funcionalidades de una aplicación web. Por ello, es una buena práctica hacer un uso eficiente de ellas. Cada una, como ya se ha comentado, reserva o hace uso de un espacio de memoria, por lo que ese espacio, una vez que las variables ya no son usadas, debe ser liberado. Lenguajes como Java poseen un mecanismo que se encarga automáticamente de este proceso. En el resto de lenguajes, es conveniente asegurarse de liberar el uso de los recursos.



3

PROGRAMACIÓN BASADA EN LENGUAJES DE MARCAS CON CÓDIGO EMPOTRADO



3.1. TOMAS DE DECISIÓN

El **desarrollo de una aplicación web** trae consigo la programación de muchos elementos que permiten al usuario elegir o ejecutar diferentes acciones que deben mostrar un resultado u otro dependiendo de dicha elección.

Cada una de estas ejecuciones debe ser controlada y definida dentro del código, lo que permitirá (en función de la selección del usuario) tomar un camino u otro, es decir, ejecutar un servicio u otro.

Las **estructuras condicionales** permiten controlar y gestionar todas y cada una de las ejecuciones que realizará la aplicación web. Estos condicionales permiten evaluar una determinada condición, normalmente de tipo booleana (verdadero/falso).

Habitualmente, es posible reconocer estas estructuras por las proposiciones **if** y **else** que forman parte de la mayor parte de lenguajes.

```
if ($variable1 >= $variable2) {  
    echo "variable1 es mayor igual que variable2";  
}else {  
    echo "variable2 es mayor que variable1";  
}
```

Este tipo de estructuras son usadas cuando el **valor de entrada recibido es único**, es decir, o se cumple o no se cumple. Pero es posible que, en determinados momentos, el valor recibido pueda ser escogido de entre varios posibles, por lo que es necesario diferenciar cuál ha sido escogido.

ponte a prueba

Las estructuras condicionales permiten controlar y gestionar todas y cada una de las ejecuciones que realizará la aplicación web.

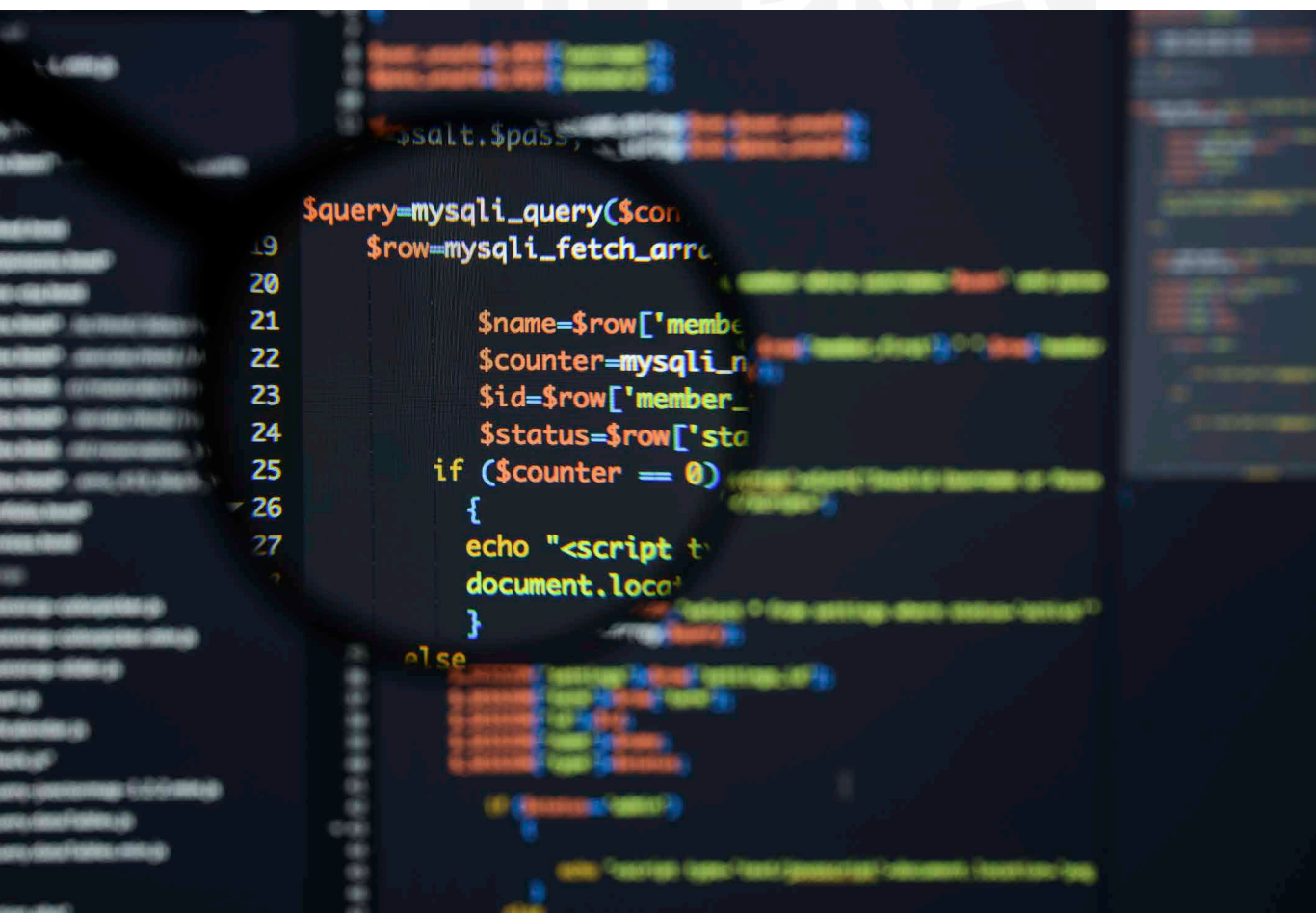
- a) Verdadero
- b) Falso

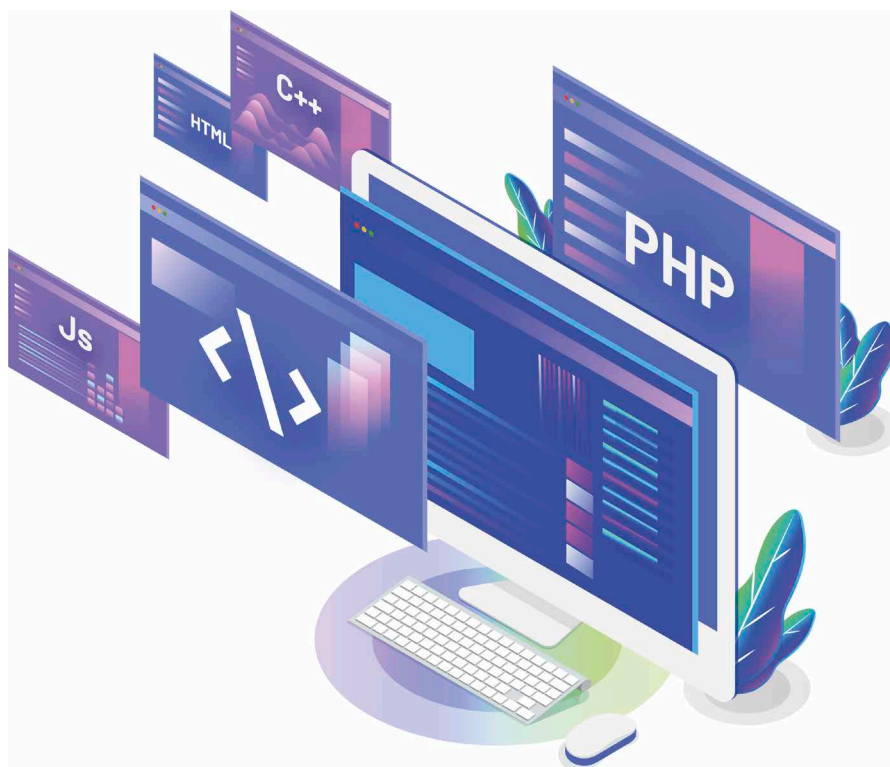
Para ello, se utilizan las **estructuras de selección múltiple**. Las cláusulas que se usan de forma generalizada son **switch** y **case**.

```
switch ($nombreVariable) {
    case "valor1":
        echo "$nombreVariable contiene valor1";
        break;
    case "valor2":
        echo "$nombreVariable contiene valor2";
        break;
    case "valor3":
        echo "$nombreVariable contiene valor3";
        break;
}
```

Este tipo de estructuras permiten dejar un código más limpio, ya que, a la hora de ser codificado, resulta más fácil de depurar. Esto ocurre, sobre todo, cuando es necesario encadenar un número elevado de posibilidades.

Java o JavaScript, al igual que PHP y ASP.NET, permiten hacer uso de estas estructuras durante la programación siguiendo prácticamente la misma sintaxis.





3.2. BUCLES

La realización de una operación cualquiera dentro de un lenguaje es empleada para conseguir un determinado resultado. En ocasiones, el número de operaciones que se debe realizar es demasiado grande, lo que implica un coste computacional muy elevado.

Este número de operaciones se puede ejecutar un número limitado de veces, o bien estar determinado por la evaluación de una condición.

Los lenguajes de programación facilitan estos procesos, clasificando cada uno de ellos en diferentes estructuras denominadas bucles.

- **Estructuras repetitivas condicionales**

Cada iteración en el bucle devolverá un único resultado, que volverá a ser evaluado. En caso de satisfacer la condición, se ejecutará una sentencia o el bucle finalizará, de lo contrario, el resultado volverá a ser evaluado. La sintaxis más empleada es la que utiliza los bucles ***while*** o ***do-while***.

```
$variable = 1;
while ($variable < 1000):
    echo $variable;
    $variable++;
endwhile;
```

• Estructuras repetitivas iterativas

Este tipo de estructuras son empleadas **cuando se conoce o se puede prever el número de iteraciones que se deben realizar**. La acción se ejecutará un número de veces ya predeterminado dentro de la sintaxis del bucle, que irá contando el número de ejecuciones como condición de evaluación. La sintaxis que mejor define estas estructuras es que emplea los bucles **for**.

```
for ($variable = 1; $variable < 1000; $variable++) {
    if ($variable = 500) {
        echo "Ya se han hecho la mitad de la comprobaciones...";
    }
    echo $variable;
}
```

• Estructuras repetitivas recorrido

Además de estructuras que permiten realizar o ejecutar una acción un número de veces, **es posible hacer uso de estas estructuras para la interpretación y recorrido de un conjunto de resultados obtenidos**.

Estas estructuras realizan un recorrido por los elementos de una matriz, *array*, colección u otros elementos independientemente del tamaño de elementos que contengan.

El uso de los bucles **for** o **foreach** específicos de algunos lenguajes suponen la mejor forma de realizar estos recorridos para poder interpretar o mostrar los resultados en una página web.

```
<?php
$array = array(1, 2, 3, 4);
foreach ($array as $valor) {
    echo "$valor";
}
?>
```

ponete a prueba

¿Cuál es el mejor uso de la instrucción *while*?

- a) Ejecutar un bucle iterativo
- b) Ejecutar un recorrido por los elementos de una matriz
- c) Ejecutar una selección múltiple
- d) Ejecutar un bucle condicional



3.3. TIPOS DE DATOS COMPUESTOS

CONCEPTO

Los **tipos de datos compuestos** son la composición de un conjunto de tipos de datos básicos.

Dentro de este tipo de datos más complejos, se pueden encontrar:

- **Resultados de SQL:** tuplas o registros de una base de datos.
- **Operaciones:** productos cartesianos, multiplicaciones, sumas, etcétera.
- **Conjuntos de datos:** enumeraciones, *arrays* o colecciones, entre otros.

Cada uno de los datos compuestos almacena dentro un conjunto de datos básicos. Por ejemplo, un *array* o lista declarado como entero se compondrá de un número determinado de valores de tipo entero.

```
$matriz = array(1,1,2,3,5,8,13,21,34,55);  
var_dump($matriz);
```

Este tipo de datos permite trabajar con estructuras dinámicas de datos, algo que resulta imprescindible en la programación web, puesto que una aplicación web o una página web dinámica debe permitir mostrar un número considerable de registros dentro de una interfaz. Un ejemplo sería el catálogo de artículos de venta dentro de una página web.

Hoy en día, la cantidad de información que se puede almacenar en una base de datos para un sitio web puede constar de cientos de miles de registros. Las páginas web, al igual que las bases de datos, se han tenido que adaptar a este tipo de necesidades haciendo uso de lo que se denomina **mapeo de datos**.

Cada uno de los registros almacenados en la base de datos será la representación de un objeto definido en una clase. Esto posibilita hacer uso de todas las propiedades relacionadas con un objeto, o que significa una mejor eficiencia en el almacenamiento de los datos y un enriquecimiento de información dentro de un sitio web.

El almacenamiento de objetos en base de datos se consigue serializando los tipos de datos compuestos.



3.4. FUNCIONES

CONCEPTO

Las **funciones** son elementos prácticamente imprescindibles en la totalidad de los lenguajes de programación.

Tienen como objetivo **encapsular en una parte de código una funcionalidad específica de una aplicación**. En muchas ocasiones, será necesario repetir una funcionalidad en diferentes partes del código, y las funciones permiten refactorizar dicho código.

```
function NombreFunction($parametros,...) {  
    //Código que realiza la función  
}
```

La finalización de ejecución de una función es retornar un valor o devolver el flujo de ejecución al punto desde el que fue llamado tras la ejecución de una o varias sentencias.

```
function NombreFunction($parametros,...) {  
    //Devolver valor  
    return $valorDevuelto;  
}
```

Casi todos los lenguajes de programación web permiten hacer uso de estas funciones. Las funciones pueden recibir parámetros o no según su utilización dentro de estas funciones. Cada lenguaje emplea su propia sintaxis para la creación o llamada a una función.

Para la definición de una función, **PHP** utiliza la palabra **function**, que indica al intérprete del código que comienza la declaración de una función. Esta irá seguida del nombre que se le asignará y será acompañada de los parámetros (en el caso de que sean necesarios).

- Pasar un argumento como parámetro:

```
function Duplica($num) {
    $num = $num * 2;
    return $num;
}

echo Duplica(5);
```

- Pasar array como parámetro de la función:

```
function RecogerArray($array) {
    var_dump($array);
}

$valores = array("Ilerna", "Online");
echo RecogerArray($valores)
```

- Pasar parámetros por referencia:

```
function completarCadena(&$txt)
{
    $txt .= 'en entorno servidor';
}
$ciclo = 'Desarrollo de aplicaciones web';
completarCadena($ciclo);
echo $ciclo;
```

- Utilizar parámetros predeterminados en funciones:

```
function examen($modulo = "Desarrollo web en entorno servidor")
{
    return "Hoy tengo examen de $modulo.</br>";
}
//Devuelve el valor del parámetro por defecto
echo examen();
//Devuelve el valor proporcionado al parámetro
echo examen("Despliegue de aplicaciones web");
```

Además, PHP incluye una serie de funciones adicionales ya creadas con un propósito específico (*mysqli_connect()*, *phpinfo()*, etcétera).

ASP.NET permite, de igual forma y mediante el uso del lenguaje C#, la creación de funciones específicas. Estas, a diferencia de PHP, deben especificar:

1. En primer lugar, el **tipo de dato** que devolverán al punto de llamada de dicha función. Permiten retornar cualquier tipo de dato.
2. En segundo lugar, el **nombre de la función**.
3. Por último, los **argumentos** (si son necesarios).


BUSCA EN LA WEB

Se pueden consultar dentro de la documentación en su página oficial:

<https://www.php.net/manual/es/indexes.functions.php>



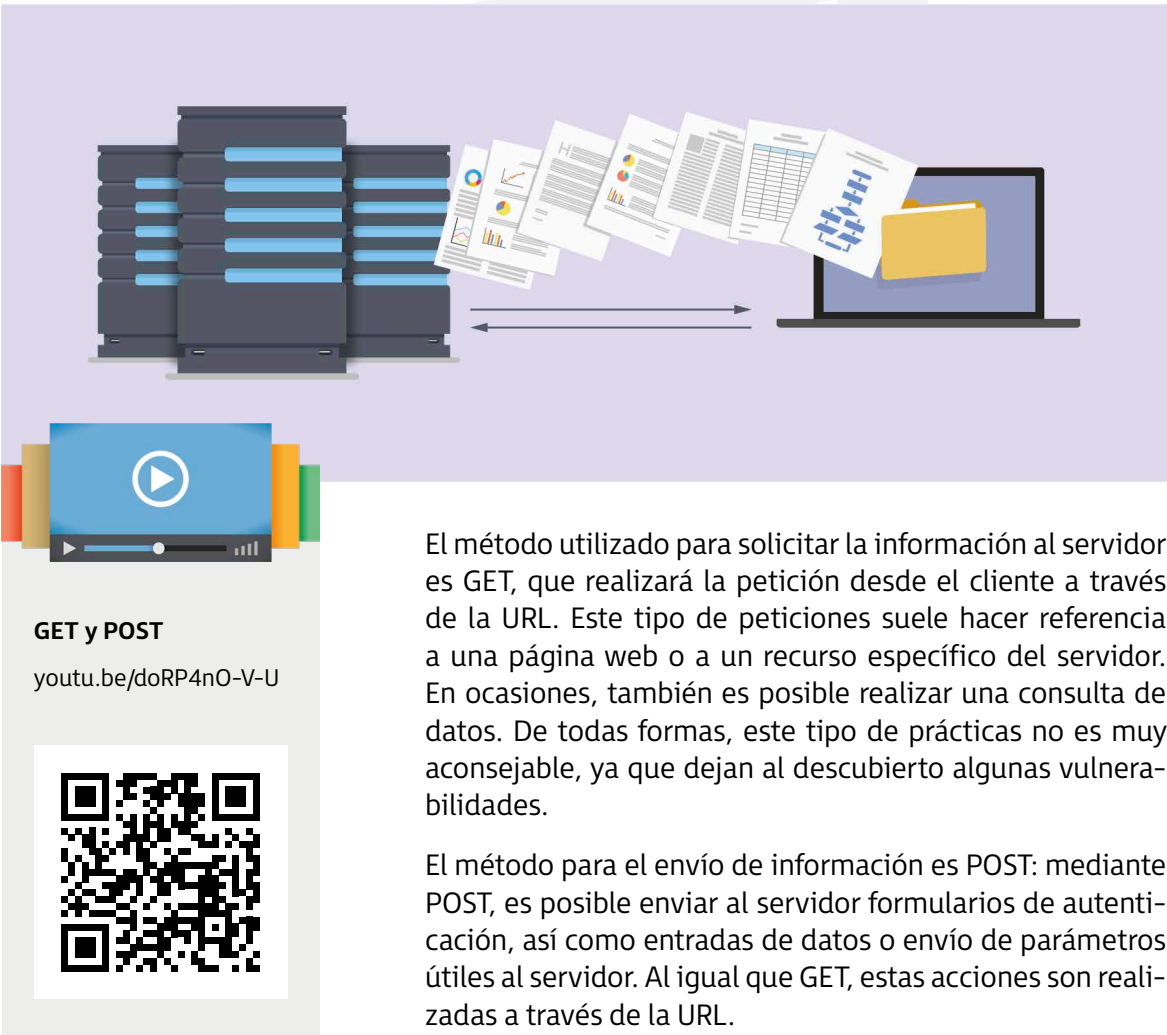

También es posible no devolver ningún valor. En este caso, será necesario establecer como tipo de dato la palabra reservada **void**, que indicará que la función volverá al punto en el que fue llamada sin esperar ningún valor de retorno.

En el caso de **JSP**, las funciones **siguen la misma estructura que para ASP.NET**. En primer lugar, se especifica el tipo de valor de retorno de la función y, en segundo lugar, el nombre especificado para esa función. Por último, se añaden entre paréntesis los parámetros necesarios.

3.5. RECUPERACIÓN Y UTILIZACIÓN DE INFORMACIÓN PROVENIENTE DEL CLIENTE WEB

Para poder recuperar los datos enviados por el cliente web, existen diferentes métodos. Dos de los más utilizados son **GET** y **POST**.

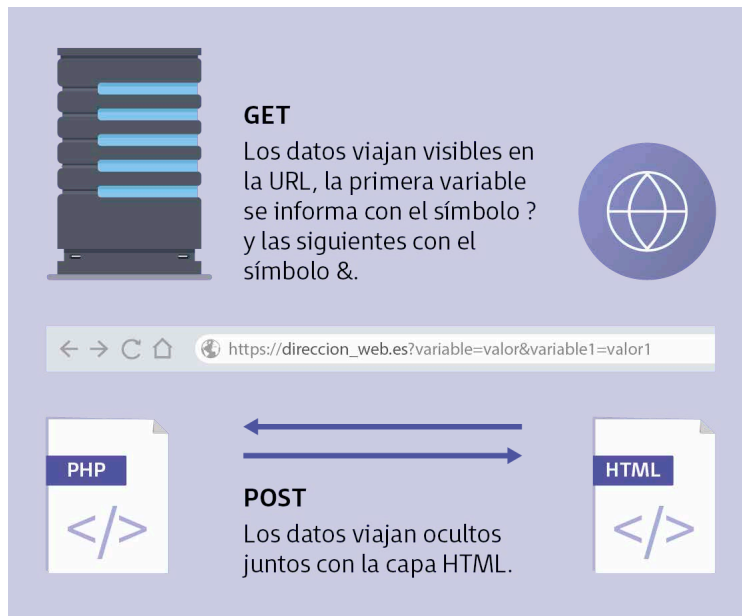
Estos métodos pertenecen al protocolo HTTP, permitiendo el intercambio de información entre el cliente y el servidor.



GET y POST
youtu.be/doRP4nO-V-U

El método utilizado para solicitar la información al servidor es GET, que realizará la petición desde el cliente a través de la URL. Este tipo de peticiones suele hacer referencia a una página web o a un recurso específico del servidor. En ocasiones, también es posible realizar una consulta de datos. De todas formas, este tipo de prácticas no es muy aconsejable, ya que dejan al descubierto algunas vulnerabilidades.

El método para el envío de información es POST: mediante POST, es posible enviar al servidor formularios de autenticación, así como entradas de datos o envío de parámetros útiles al servidor. Al igual que GET, estas acciones son realizadas a través de la URL.



Aunque ambos métodos parecen casi iguales, existen algunas **diferencias** que es importante señalar:

- **GET y POST codifican la información de forma diferente:**

- El método **GET** envía las variables dentro de la URL de la página, mientras que en el método **POST** la información va codificada en el cuerpo de la petición HTTP.
- El método **POST**, por ejemplo, permite el envío de formularios de forma más segura, pues la información del usuario viaja oculta. A pesar de esto, no es posible afirmar que un método es más seguro que el otro, puesto que el método POST también tiene vulnerabilidades, tales como la inyección de código SQL, que permite insertar código maligno en la propia base de datos de la aplicación.

¡RECUERDA!

El **uso** de cada uno de ellos viene determinado por su propia naturaleza, es decir, **GET para pedir información y POST para el envío**.

Como ya se ha comentado, ambos métodos hacen uso de las **URL**. Es importante destacar que una URL se compone de **cuatro partes**:

1. **Protocolo:** HTTP es el usado en la web.
2. **Dominio:** el nombre del sitio web.
3. **Directorio:** ruta o jerarquía de carpetas que indica el camino de acceso al fichero o recurso deseado.
4. **Fichero o recurso:** nombre del recurso o fichero almacenado en la base de datos.

ponte a prueba

Verdadero o Falso:
Aunque los métodos GET y POST codifican de idéntica forma la información, el primero sirve para solicitarla y el segundo para enviarla.

- a) Verdadero
- b) Falso

3.6. PROCESAMIENTO DE LA INFORMACIÓN INTRODUCIDA EN UN FORMULARIO

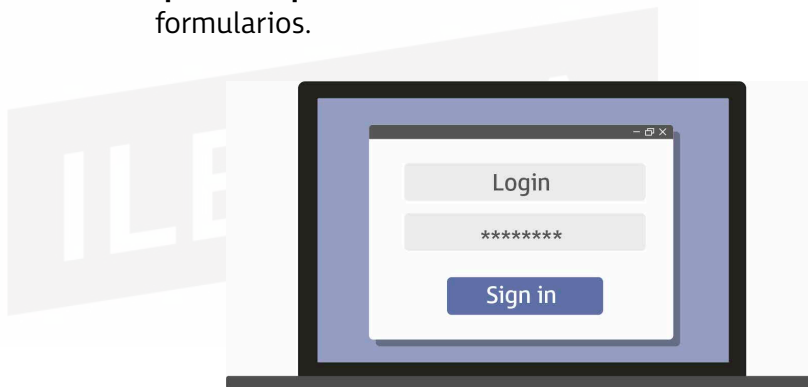
Los formularios son elementos web que nos facilitan la interacción con los datos que tenemos disponibles en un aplicativo web. Estos son utilizados para facilitar la comunicación entre el usuario y los datos, permitiendo modificar cualquier parte del aplicativo.

CONCEPTO

Los **formularios web** son uno de los elementos más comunes para la recuperación y validación de datos de los usuarios.

Permiten, mediante una sola petición, obtener una gran cantidad de información.

En prácticamente la totalidad de los **sitios web en los que se requiere de autenticación de acceso** se utilizan formularios.



Además, también sirven como elemento de filtrado de una consulta en una base de datos. En este caso, cada uno de los campos del formulario se convierte en los diferentes criterios de una sentencia SQL.

A registration form with fields for First Name, Last Name, Username, Email, Tel. Number, Password, and Confirm Password, along with an OK button.

Envío de
información del
formulario de registro.



Respuesta
del servidor.



Los formularios pueden ser enviados tanto por POST como por GET. El método elegido para realizar dicho envío se debe especificar dentro de la etiqueta **form** de HTML.

```
<form name="formulario" action="consultas.php" method="POST">
  <h1>FORMULARIO WEB</h1>
  <label>Usuario: </label><input type="text" name="nombre" value="" />
  <label>Contraseña: </label><input type="password" name="contrasena" value="" />
  <input type="submit" name="enviar" value="Enviar" />
  <input type="reset" name="cancelar" value="Cancelar" />
</form>
```

Una vez que se ha enviado la información, esta se procesa en el servidor. En el caso de los formularios de autenticación, servirán para buscar el usuario y contraseña dentro de la base de datos. Si la consulta es satisfactoria, se devuelve la nueva página, que será mostrada al cliente.

Para el caso de **formularios genéricos de información** es necesario, generalmente, validar estos campos antes de ser enviados al servidor. De esta forma, se puede evitar almacenar información incorrecta en la base de datos.



Una buena praxis de programación es realizar validaciones de los datos en diferentes puntos de nuestro aplicativo web:

- **Entorno cliente:** los datos son validados con lenguajes de entorno cliente, como, por ejemplo, las expresiones regulares de JavaScript.
- **Entorno servidor:** los datos son validados con lenguajes de este entorno. De este modo, podemos realizar varias comprobaciones de los datos antes de ejecutar las acciones en el servidor, como comprobar que el tipo de dato sea el mismo que espera la base de datos.

Por otro lado, junto con las validaciones podemos añadir restricciones en diferentes puntos del aplicativo web para complementar la seguridad y la integridad de los datos.

- **Entorno cliente:** HTML5 nos ofrece varias restricciones de datos de una forma muy sencilla y útil, como el uso de los atributos *restricted* de los *inputs* o los diferentes tipos de *input* de los que disponemos (*number*, *email*, *password*, etcétera).
- **Entorno servidor:** todos los clientes de bases de datos ofrecen diferentes restricciones para asegurar la integridad de los datos, este punto es el más importante de configurar en cualquier aplicativo. Podemos disponer de varios tipos de restricción, como puedan ser restricciones de tipo de datos en los campos de una tabla o restricciones de eliminación de datos por integridad entre tablas.

Una vez validados, los datos se enviarán y el servidor los procesará y ejecutará la sentencia SQL correspondiente para almacenarlos.



3.7. COMENTARIOS

En ocasiones, los **comentarios** son una de las partes que los programadores y desarrolladores dejan olvidadas.

Los comentarios pueden llegar a ser la **guía de utilización de una aplicación web completa**, permitiendo comprender y analizar todos los pasos que son ejecutados en la aplicación.

La **forma** en que se realizan los comentarios es el aspecto clave para entender su importancia:

- Es posible realizar un comentario junto a una línea de código o función para tratar de explicar o detallar algunas partes de ese código. Por otro lado, la funcionalidad que los comentarios ofrecen puede ser mucho mayor.
- Lenguajes como PHP permiten hacer uso de estándares propios de documentación basados en comentarios. Esto es lo que se conoce como **TODO comments**. Es posible, mediante la sintaxis **TODO** como comentario, dejar marcada una línea de código como algo pendiente de hacer.
- Al igual que **TODO**, existen otro tipo de comentarios configurables que permiten depurar y gestionar el desarrollo de una aplicación. Estos comentarios pueden ser usados incluso como filtros dentro de una aplicación.

El uso de comentarios dentro de una aplicación es la **base de la documentación** que permitirá a personas ajenas a su desarrollo entender el código y que diferentes programadores puedan trabajar sobre él.

La sintaxis empleada para la realización de comentarios difiere dependiendo del lenguaje de programación que se esté utilizando. Para conocer cuál se debe aplicar es necesario consultar la documentación propia de cada uno de ellos.

En la siguiente sintaxis veremos los diferentes modos que disponemos para realizar comentarios en PHP. Aunque todos son válidos, hay que tener en cuenta que los comentarios son uno de los puntos más importantes para agilizar la tarea de mantenimiento de los aplicativos, debiendo ser claros, concretos y no muy extensos si no es necesario.

```
//Comentario de una línea
#Comentario de una línea
/*
Comentario de
varias líneas
*/
```



4

DESARROLLO DE APLICACIONES WEB UTILIZANDO CÓDIGO EMPOTRADO

4.1. MANTENIMIENTO DEL ESTADO

Dentro de la navegación en una web es posible que, en un momento determinado, se produzca el paso de una pestaña a otra, o se dé la realización de una compra o una transferencia bancaria. Este tipo de acciones obligan a mantener determinados datos o estado de un cliente. En caso contrario, no sería posible garantizar que el usuario realice las peticiones en cada momento. El mecanismo que permite llevar a cabo este control son las sesiones.

CONCEPTO

Las **sesiones** permiten garantizar y controlar que los recursos son ejecutados de manera correcta.

Gracias al uso de sesiones, también es posible que un usuario pueda seguir realizando cualquier acción sin la necesidad de autenticarse en cada una de las pestañas.

Las sesiones web se pueden establecer entre un cliente y un servidor de las siguientes formas:

- **A través de la URL:** los datos son enviados como parámetros dentro de la URL bajo el uso del método GET. Se identifica al usuario en el servidor y este retorna un identificador denominado SID a dicho usuario. Una vez que el recurso solicitado se haya abandonado, la sesión caducará y será necesario renovar dicho SID. El **SID** es un número generado de manera aleatoria por parte del servidor.
- **Cookies:** el cliente almacena en su navegador un archivo que guarda los datos necesarios para mantener la sesión abierta. El servidor utiliza ese archivo para verificar la identificación del cliente.

Básicamente, estos son los mecanismos empleados para mantener el estado de los datos dentro de un sitio web. Lenguajes como PHP permiten el uso de ambos tipos, tanto las *cookies* como la URL.

Las sesiones no se generan automáticamente a menos que sea especificado, sino que estas se deben generar dentro del código. Para ello, es necesario hacer uso de las funciones correspondientes para cada uno de los lenguajes de programación.

Lo primero que se debe realizar tras la autenticación del usuario es iniciar la sesión. Esta función debe situarse al comienzo del código asociado a cada sitio web.

Una vez que finaliza la sesión, debe destruirse empleando el método que corresponda.



ponte a prueba

Las sesiones permiten garantizar y controlar que los recursos son ejecutados de manera correcta.

- a) Verdadero
- b) Falso

Dependiendo del lenguaje que estemos utilizando para trabajar con el servidor, la sintaxis de los ejemplos puede variar. En el siguiente ejemplo, donde veremos cómo iniciar sesiones y trabajar con variables de sesión, se utiliza PHP.

El primer paso para trabajar con sesiones es iniciar una sesión invocando la función predefinida **session_start()**:

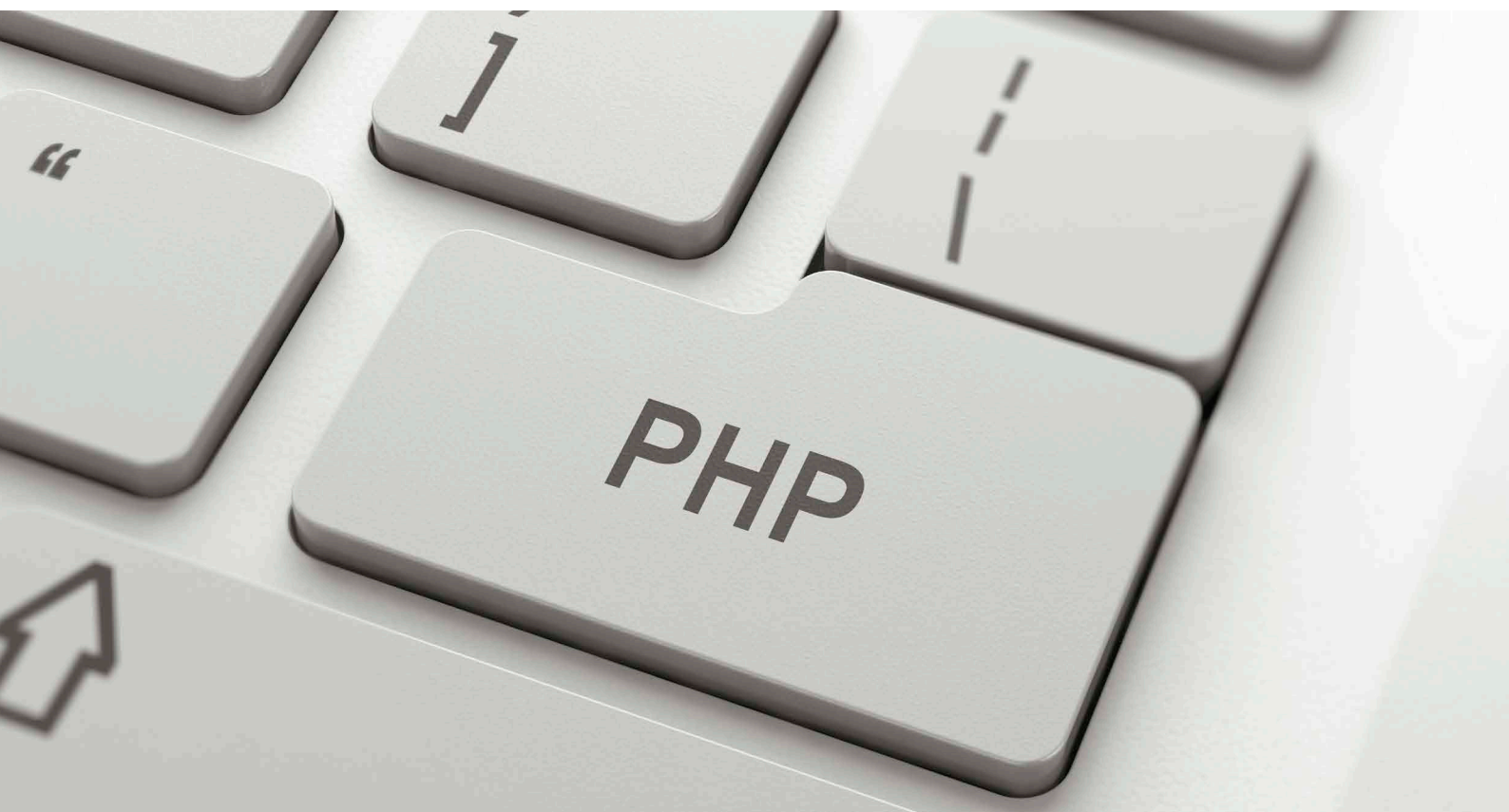
```
<?php
session_start();
if (!isset($_SESSION['temps'])) {
    $_Session['temps'] = 0;
} else {
    $_SESSION['temps'] = time();
}
?>
```

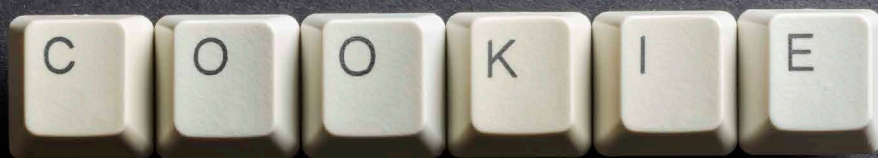
Una vez iniciada la sesión, podemos trabajar con las funciones de sesión, las cuales se identificarán por el nombre que indiquemos en su interior. Al igual que las variables del lenguaje, estas definirán el tipo de dato que tienen según el contenido que almacenen.

```
$_SESSION['nombre'] ;
```

Estas variables estarán disponibles siempre que la sesión este activa. En la siguiente sintaxis veremos cómo eliminar una variable de sesión:

```
unset($_SESSION['temps']);
```





4.2. COOKIES (GALLETAS)

CONCEPTO

Las **cookies** son pequeños archivos que permiten almacenar los datos de sesión asociados a un usuario una vez que se ha identificado y ha sido validado desde el servidor.

Las *cookies* son un mecanismo que, en ocasiones, puede resultar muy útil. Por el contrario, es posible también que a través de estas *cookies* se acceda a sitios sin permiso o se generen otros problemas de seguridad. Esto hace que algunas páginas web hayan bloqueado el uso de *cookies*. En este caso, se crearán sesiones por código.

Puesto que pueden contener información o **datos de carácter sensible**, es fundamental controlar su envío al servidor.

PHP posibilita esta gestión por medio de la función **setcookie()**. Esta función permite crear una *cookie* cuyos parámetros de configuración se establecerán bajo unos criterios deseados por el programador. Estos parámetros son nombre, valor, tiempo de vida, ruta, dominio y seguro.

Es posible consultar la documentación de PHP para conocer con más detalle la descripción de cada uno de estos parámetros.

También es posible mostrar el valor que tiene la *cookie*, este valor se recupera a través de la variable asignada para ello, que en PHP es **\$_COOKIE**.

Actualmente, el uso de estas *cookies* almacenadas en los navegadores ha sido utilizado por los sitios web para adquirir información sobre los usos e intereses habituales de los usuarios durante su navegación, tratando de conseguir, de esta forma, mostrar a cada usuario todos los productos que puedan ser potencialmente interesantes para él.

Hoy en día, las alertas sobre su uso se han formalizado de forma legal y los sitios web que las utilizan tienen que alertar a los usuarios.

Veamos la sintaxis de las sentencias básicas sobre el manejo de *cookies* con PHP:

- Para crear una *cookie* se necesita almacenar unos determinados valores: el nombre con el que la identificaremos, el valor que tiene que almacenar y el tiempo que va a perdurar:

```
<?php
/*
--nombre: El nombre con el que referenciamos la cookie
--valor: El valor que almacenara la cookie
--$t: el tiempo de expiración que tendrá la cookie
*/
setcookie("nombre","valor",time() + $t);
?>
```

- Con la sentencia `$_COOKIE[""]` podemos acceder a su valor:

```
<?php
//Visualizamos el valor de la cookie
echo $_COOKIE["nombre"];
?>
```

- Para actualizar su valor, tenemos que almacenar un nuevo valor en la *cookie* que queremos modificar:

```
<?php
//Actualización de una cookie
setcookie("programa","late motive");
echo $_COOKIE["programa"];
//Al recoger un dato en una cookie con el mismo nombre
//el valor de esta se actualiza
setcookie("programa","la resistencia");
echo $_COOKIE["programa"];
?>
```

- Para eliminarlas, tenemos diferentes formas de realizarlo, aunque la eliminación mediante la función `unset()` es más utilizada:

```
<?php
//Eliminamos una cookie
unset($_COOKIE["nombre"]);
//Otra forma de eliminar una cookie
setcookie("nombre","valor",time() - 1);
?>
```

4.3. SEGURIDAD: USUARIOS, PERFILES, ROLES. COOKIES

CONCEPTO

La **seguridad** es uno de los aspectos que más problemas e inquietudes generan en los programadores y desarrolladores de los sitios web.

En muchas ocasiones, es difícil controlar todos los posibles puntos de recepción y envío de información. Por tanto, es necesario intentar **establecer los mejores y más fiables mecanismos de seguridad dentro de cualquier tipo de aplicación.**

El primer punto que se debe comprobar es **el control de usuarios**: de manera genérica, cualquier tipo de aplicación suele dividir a los usuarios en función de unos determinados perfiles o roles. Estos indicarán al servidor si tienen permiso de acceso a los diferentes recursos de la página web.

Estos grupos de usuarios y roles no son fáciles de gestionar, por lo que existen *softwares* que pueden ayudar al administrador del sitio a configurar los accesos. La organización jerárquica de estos permisos o roles se recoge en una lista denominada **ACL** (*access control list*). Es posible crear una ACL que actúe de intermediaria entre el cliente y la base de datos, realizando un proceso de autenticación basado en los grupos y roles asignados.

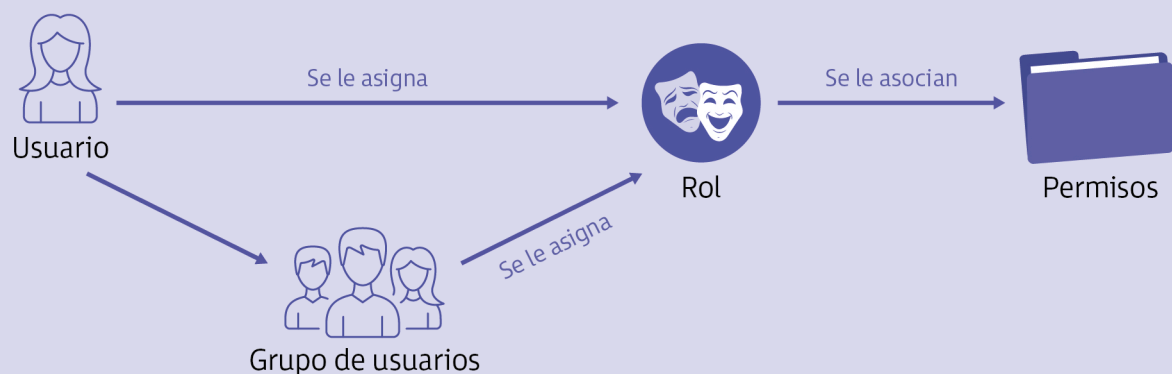
El uso de *cookies*, como se ha explicado con anterioridad, es un mecanismo que permite extraer información sobre los usuarios cuando realizan navegaciones por la web. A pesar de esto, **las cookies no pueden ni leer datos almacenados dentro del equipo de un usuario ni realizar instalaciones indeseadas.**



ponte a prueba

¿Qué es una ACL?

- a) Un tipo de *cookie*
- b) Una tecnología de autenticación
- c) Una lista de permisos
- d) Una etiqueta que define el rol de usuario





4.4. AUTENTICACIÓN DE USUARIOS

Habitualmente, el proceso de autenticación de usuarios se lleva a cabo mediante el envío de formularios. Este proceso es validado por el servidor, que gestiona el acceso que es devuelto al usuario.

El problema de estos mecanismos es que la evolución en las tecnologías ha hecho que la creación de nuevas aplicaciones web o de escritorio hayan crecido considerablemente. De esta forma, el usuario se ve obligado a rellenar un formulario previo de registro y debe recordar un usuario y contraseña para cada una de las páginas web. Este tipo de procesos puede llegar a incomodar en algunos momentos, puesto que es necesario tener una relación para cada una de las páginas.

Para tratar de hacer frente a este problema, han aparecido una serie de tecnologías de autenticación como OpenID y OAUTH.



¡RECUERDA!

El objetivo de estas tecnologías es **centralizar el acceso a cualquier tipo de servicio web mediante el uso de plataformas ya verificadas.**

Esto significa que es posible **identificarse dentro de una web a través de plataformas** como Facebook, Twitter, Google+ o Hotmail, entre otros.

Este proceso consiste en abrir una pasarela de acceso a un servicio externo que, mediante un formulario de acceso, permite generar una sesión para ese usuario sin necesidad de realizar el proceso de registro previo. Estos datos de acceso recabados a través de la plataforma externa son almacenados en la base de datos para que permita obtener los permisos necesarios y asignarlos al usuario en cuestión.

Esto permite a los servidores del almacenamiento de procesos de verificación de identidad, así como de generación y control de **tokens** de acceso su descarga.

Es importante destacar que el uso de estas tecnologías supone un problema de dependencia con el sistema externo, que debe ser fiable y que no depende del funcionamiento de la aplicación.



4.5. ADAPTACIÓN A WEBS EXISTENTES

Una vez tratados los puntos principales que forman parte del proceso de creación de una aplicación web, es fácil darse cuenta de que la exigencia y dinamismo que presentan las páginas web exigen continuos procesos de adaptación.

Este proceso no es fácil, pero existen determinadas normas que son de cumplimiento obligatorio y que, una vez que se ha realizado el desarrollo completo de la web, se deben revisar. Hay que comprobar que se cumple con los sistemas de autenticación correctos, que se establecen los requisitos mínimos para garantizar la privacidad de los datos, que se realizan el envío y la recepción de los datos a través de pasarelas seguras encriptadas, etcétera.

En muchas ocasiones, la creación de una nueva página web suele adaptarse con base en las webs existentes, cuya funcionalidad dentro del mercado es la misma.

4.6. PRUEBAS Y DEPURACIÓN

Independientemente del ciclo de vida escogido para crear la página web, es muy importante realizar pruebas.

CONCEPTO

Las **pruebas** son el mecanismo por el cual es posible comprobar si una aplicación cumple con las condiciones y requerimientos especificados. Este tipo de pruebas son las que permiten identificar los posibles errores.

Las pruebas se deben llevar a cabo teniendo en cuenta una serie de entradas y obteniendo como resultado una salida cuyos resultados han sido definidos con anterioridad. Si los resultados obtenidos cumplen o son los esperados, eso quiere decir que el desarrollo del programa se ha realizado correctamente. Si no es así, se puede afirmar que existe un **error** en alguna parte del programa y este se debe solucionar.

Existen diferentes **tipos de pruebas** en función de la parte del código sobre la que se realizan:

- Aquellas pruebas que se llevan a cabo validando la parte lógica se denominan pruebas de **caja blanca**.
- Aquellas pruebas que se realizan sobre las interfaces o servicios web se denominan pruebas de **caja negra**.

En el caso de los **proyectos con funcionalidades extensas**, es decir, proyectos en los que el tiempo de desarrollo es elevado y que se planifican a largo plazo, es recomendable realizar **pruebas unitarias**.

Las **pruebas unitarias** permiten comprobar el comportamiento de partes específicas del código. Por ejemplo, es posible ejecutar solamente un servicio web de una aplicación para comprobar o ejecutar un determinado *script* sin que afecte al resto de la estructura de la aplicación. La mayor parte de los lenguajes de programación web permiten este tipo de pruebas.

Durante el desarrollo de cualquier tipo de aplicación o página web, y de forma paralela a las pruebas, es conveniente realizar **procesos de depuración**. Estos procesos **permiten comprobar y realizar un seguimiento de la ejecución en tiempo real de dicha aplicación o página**. De esta forma, es posible comprobar los recursos utilizados por la aplicación. Analizar los valores que van tomando las variables permite verificar que son correctas y controlar el flujo de ejecución.

Para poder realizar estos procesos de depuración es necesario hacer uso de los IDE que soporten el lenguaje de programación utilizado. Algunos de los más utilizados son **Eclipse y Microsoft Visual Studio**.

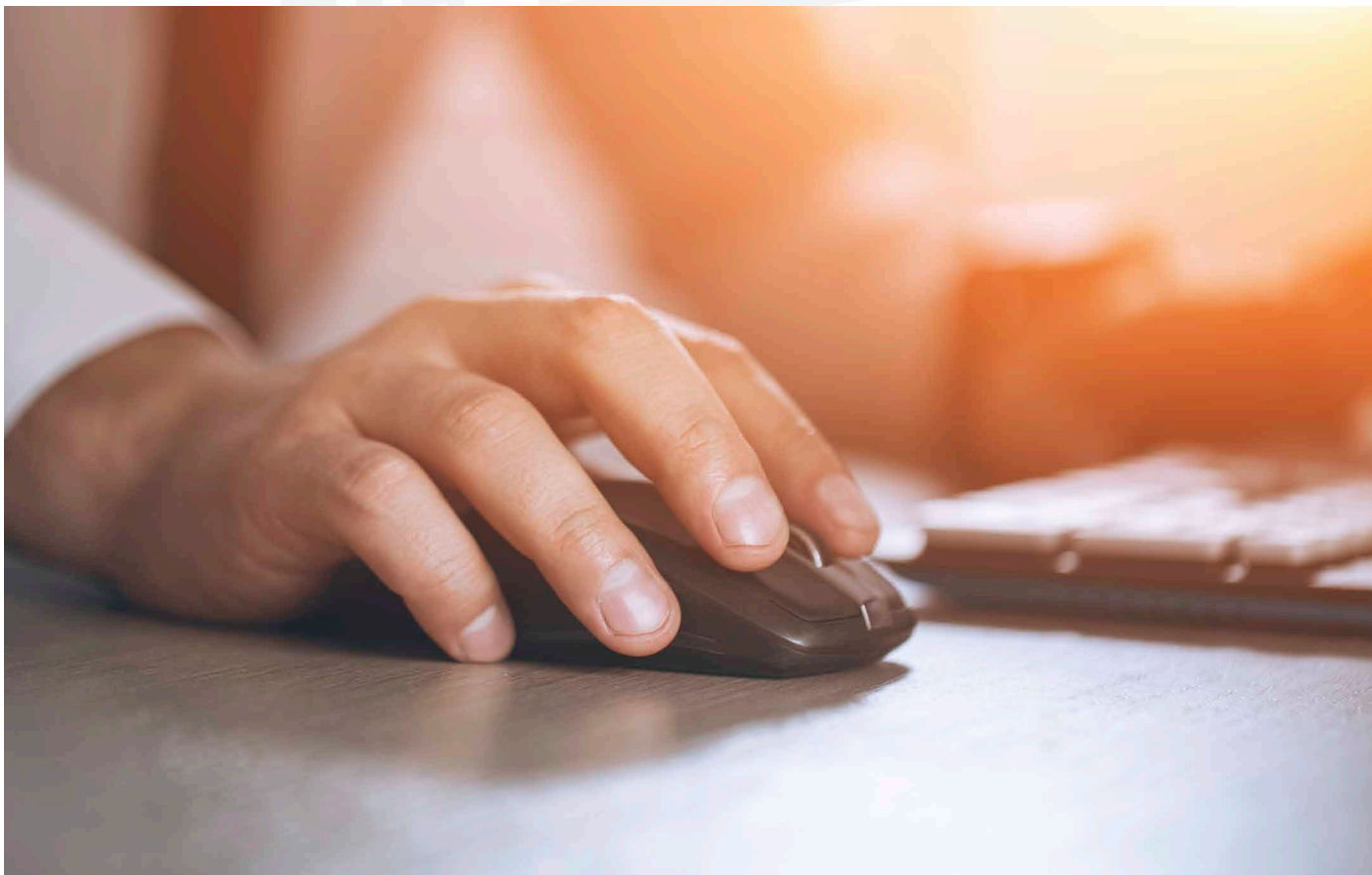


ponte a prueba

Verdadero o Falso: Las pruebas unitarias son aquellas que se llevan a cabo validando la parte lógica.

a) Verdadero

b) Falso





5

GENERACIÓN DINÁMICA DE PÁGINAS WEB

5.1. MECANISMOS DE SEPARACIÓN DE LA LÓGICA DE NEGOCIO

El desarrollo de una página web dinámica ofrece una gran cantidad de posibilidades y una mayor interactividad con el usuario. Para conseguir este resultado de la mejor forma posible, es necesario detallar y separar todos los aspectos relativos al diseño de la web, y también el modelo de negocio. Para ello, se utilizan técnicas de diseño por capas o niveles.

Estos niveles permiten definir cuál será el trabajo asociado a cada parte, con lo que se conseguirá que los cambios solamente afecten a una capa en concreto.

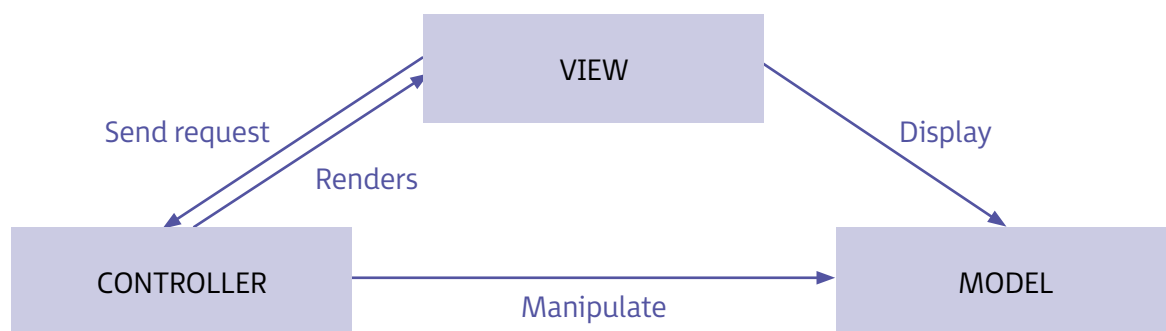
La **separación de capas** que generalmente se puede encontrar queda establecida en tres niveles:

- **Capa de presentación:** será aquella que el usuario visualizará y con la que podrá interactuar. Se encargará de comunicarse con la capa de negocio.
- **Capa de negocio:** esta capa será la encargada de procesar las peticiones del usuario. Se comunicará tanto con la capa de presentación para procesar dichas peticiones como con la capa de datos para solicitar la información deseada.
- **Capa de datos:** esta capa se encargará de acceder a los datos almacenados. Después, los gestores de bases de datos de los que se compone devolverán los datos a la capa de negocio.

De forma más esquemática, las capas de las que se compone una web pueden ser las siguientes:

- **Presentación:** capa web, define cómo se presentan los datos al usuario.
- **Lógica de negocio:** aplicación web. Se encargará de transformar los datos requeridos del mundo real en información que pueda ser mostrada y almacenada.
- **Datos:** almacenamiento en base de datos que permite la generación dinámica de páginas web.

Estas capas conformarían lo que se conoce como **patrón Modelo-Vista-Controlador (MVC)**. Este es uno de los más usados en el desarrollo de aplicaciones web, pues está basado en el concepto de separación por capas (responsabilidades).



Algunos de los lenguajes más utilizados en el desarrollo de páginas web dinámicas son HTML, Javascript, PHP, ASP, ASP.NET, JSP, Python, Ruby y Perl.

ponete a prueba

¿Cuál de las siguientes capas no forma parte de una web según el patrón Modelo-Vista-Controlador?

- a) Presentación
- b) Datos
- c) Lógica de negocio
- d) Programación

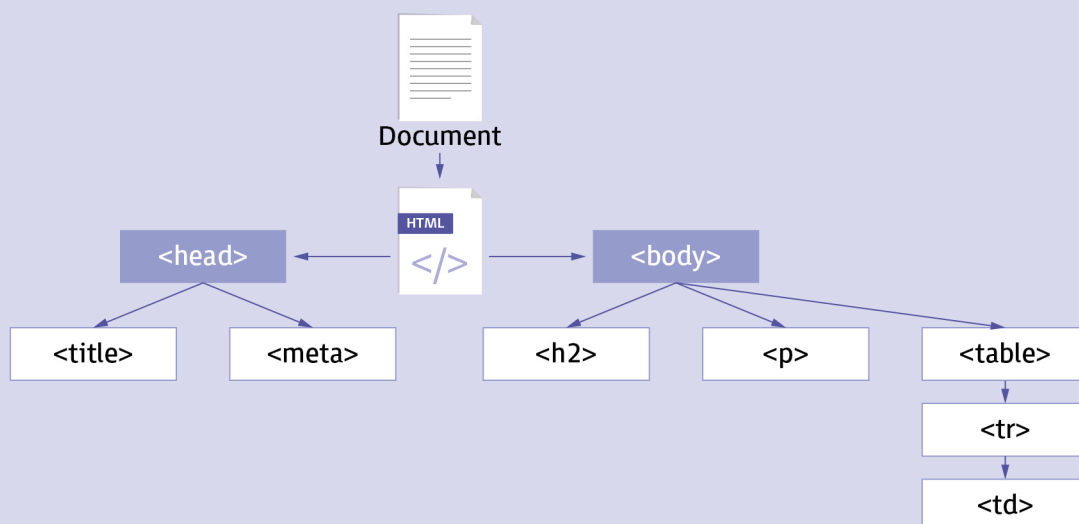
5.2. CONTROLES DE SERVIDOR

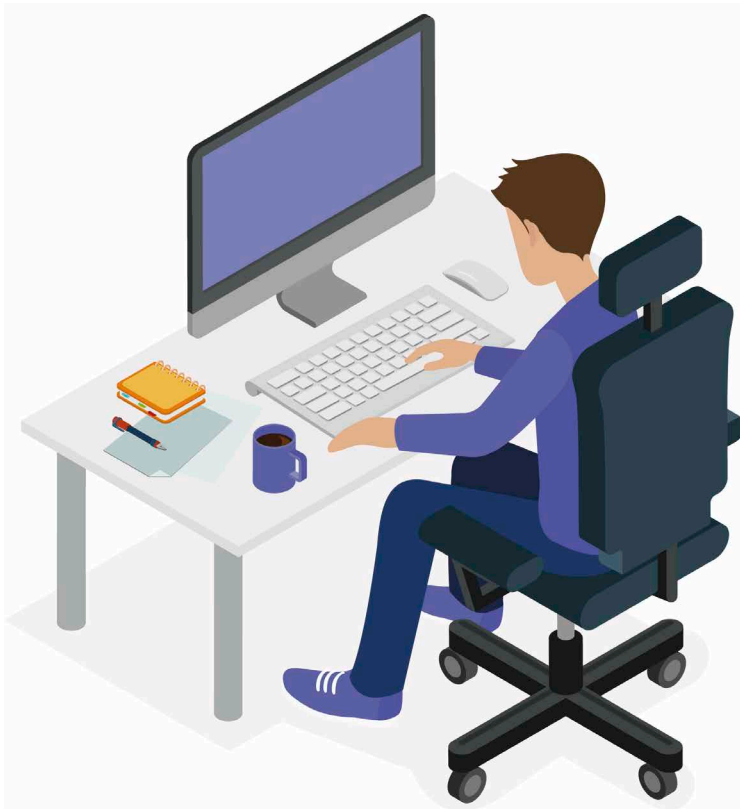
Los controles de servidor web son un **estándar formado por un determinado conjunto de controles con funciones muy específicas**.

Los controles de servidor web pueden estar formados por simples controles de formularios, tales como botones, *labels*, *textboxes* y cuadros de texto, o bien por algunos controles de mayor complejidad, como las tablas o las listas desplegables.

A su vez, estos controles proporcionan otras funciones que se utilizan frecuentemente por los usuarios, como la presentación de datos en cuadrícula, la elección de fechas, la visualización de menús, etcétera.

Estos controles web proporcionan como resultado un modelo de objetos que, procesados, generan lo que se conoce como **DOM** (*document object model*), por lo que los controles web disminuyen la complejidad de procesamiento de una web.





Algunas **características añadidas** son:

- **Detección automática del explorador:** los controles pueden identificar cuál es el navegador utilizado (ofreciendo el mejor resultado para el mismo).
- Permiten la utilización de **plantillas de diseño**.
- **Controlan los eventos de envío de datos al servidor**, determinando cuándo se hacen y qué evento producirá dicha acción. De la misma forma, permiten almacenar en la caché los datos de un formulario, por ejemplo, sin enviarlos al servidor hasta un momento determinado.

Además, lenguajes como **ASP.NET** ofrecen una guía de utilización y configuración muy detallada en su página oficial.

Por otra parte, **PHP** ofrece una gran cantidad de funciones que permiten controlar, además de las características anteriores, aspectos como las conexiones, las variables y los permisos, de forma que **primero se confirma la conexión con el servidor (antes de realizar el envío), después se comprueba que los datos siguen el formato correcto y, por último, se verifica que el cliente tiene los permisos necesarios para poder realizar dicha petición**. Esto permite que el servidor solamente se encargue de procesar la información, que almacenará si es necesario, y devolverá los datos al cliente. Esto conllevará que los procesos de validación y carga se vean aliviados.

ponte a prueba

Verdadero o Falso: Los controles de servidor web forman parte de la capa encargada de procesar las peticiones de usuario y solicitar la información requerida al servidor.

- a) Verdadero
- b) Falso

Es posible definir, por tanto, un **controlador web** como cualquier elemento web que permite ejecutar y mostrar una serie de secuencias de comandos, invocar todos los elementos de una página web, enviar variables a un servidor o cualquier otra que proporcione una funcionalidad en el cliente.

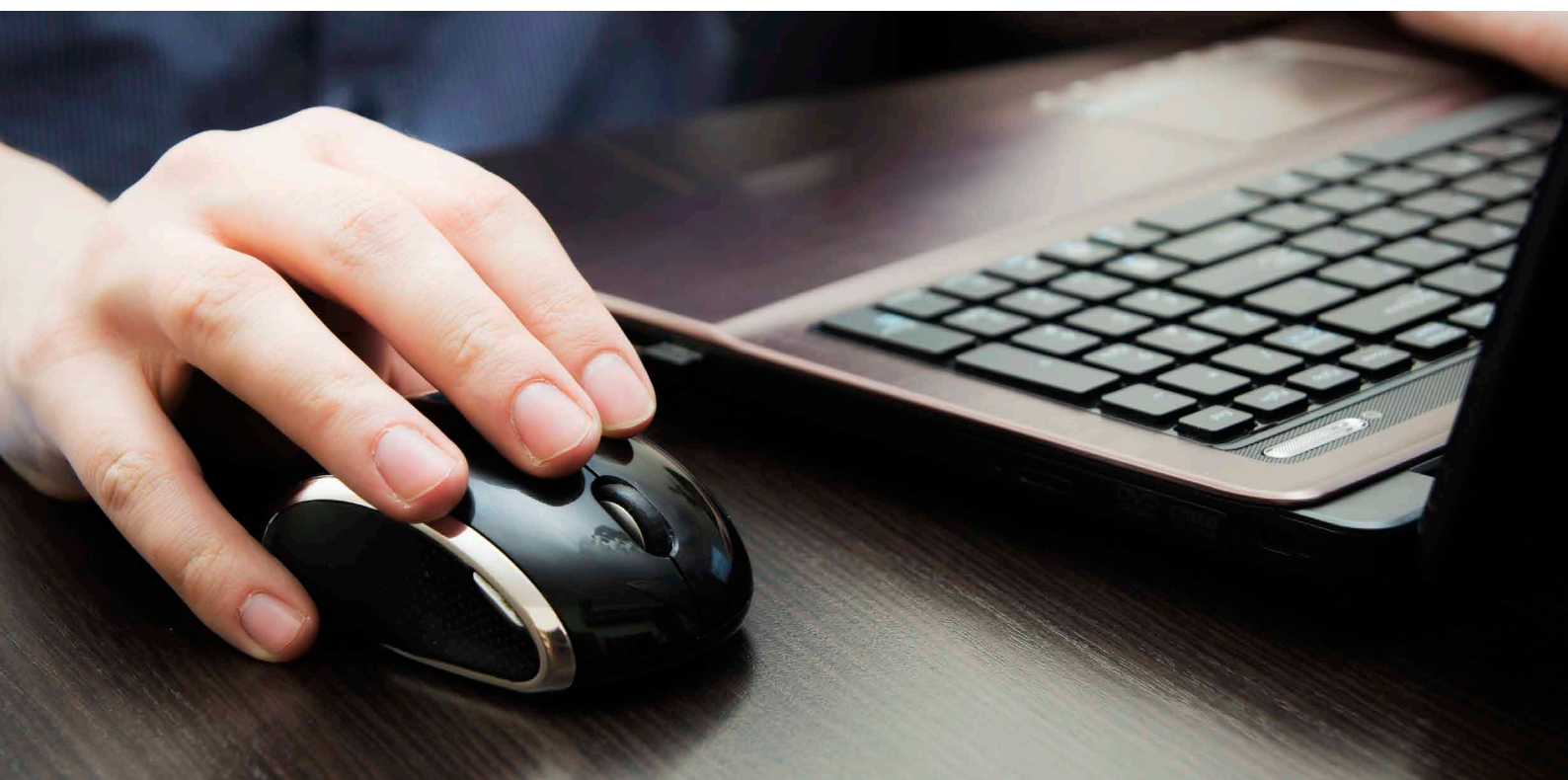
5.3. MECANISMOS DE GENERACIÓN DINÁMICA DE LA INTERFAZ WEB

CONCEPTO

El **objetivo principal de una web dinámica**, que la diferencia de una página estática, es ofrecer al usuario un contenido adaptado a cada una de las peticiones que se van realizando.

Este objetivo no es fácil, por lo que, para poder conseguirlo, es necesario emplear determinadas técnicas de creación de páginas web.

El primer paso es **definir la estructura de interacción entre las distintas partes en la web**, tanto del usuario con el cliente como del cliente con el servidor. Según el tipo de cliente utilizado, será necesario un mayor desarrollo y complejidad del servidor. Un ejemplo de cliente sería el navegador web, que actuará como hilo de conexión entre el usuario y los datos requeridos por el servidor. Este tipo de cliente es el más utilizado, pues requiere de pocos recursos *software* y su implementación es más simple.



Por otro lado, existen clientes que son capaces de procesar datos y son parte de la lógica de la aplicación. Esto reduce la complejidad del servidor, pero aumenta la complejidad del cliente.

Es posible conseguir un balance entre ambos mediante el uso de lenguajes como **JavaServer Faces** o los **controles de servidor de ASP.NET** ya mencionados. Estos lenguajes permiten generar interfaces de usuario.

También es interesante hacer uso de lenguajes que permitan implementar tecnologías íntegramente del lado del cliente, como JavaScript y CSS. JavaScript esté implementado en la mayoría de los navegadores, por lo que su uso está muy extendido. Permite, entre otras cosas, validar entradas de formularios, ocultar o mostrar información y realizar efectos visuales.

Los navegadores usan CSS para poder separar el contenido de la presentación en las páginas web, es decir, organizar los estilos de cada uno de los contenidos que se presentarán en dicha página.

Es frecuente **acompañar el uso de JavaScript con JQuery**, que permite crear animaciones y controlar eventos como, por ejemplo, el clic de un botón.

De la suma de este tipo de tecnologías nacen otras, como es el caso de **AJAX**. AJAX utiliza JavaScript y XML para realizar peticiones asíncronas al servidor. Esto permite un mayor dinamismo del contenido en una web, de la misma forma que JavaScript permite controlar la información antes de que se mande al servidor.

En resumen, la utilización de las distintas técnicas ofrece un gran número de posibilidades que permiten definir completamente el comportamiento y aspecto de una página web.

5.4. CONFIGURACIÓN

Toda creación o desarrollo de una página o aplicación web está ligado a establecer una serie de configuraciones que permitan definir cuáles van a ser sus mecanismos de seguridad, así como cuáles son los dominios en los que se podrá tener acceso a ella. Estas configuraciones, normalmente, son realizadas en el lado del servidor.

El dominio es un conjunto de caracteres fruto de la traducción, mediante un sistema DNS, de una IP que, a su vez, apunta al servidor en el que está alojada la página web.

Una vez que se ha escogido el nombre de la web, el siguiente punto es garantizar un acceso seguro a los usuarios (definiendo qué partes y qué contenidos serán visibles en la web).



Para ello, archivos como **htaccess** permiten configurar una página web. En este archivo se indica cuáles son las restricciones de seguridad en los directorios o carpetas. De esta forma, se garantiza que, antes de mostrar un determinado contenido, el usuario que ha solicitado el acceso tenga permiso para ello.

Otro de los aspectos que permite configurar este archivo es el control de la caché del servidor. Esta caché almacena documentos web, por lo que se agiliza la carga de una web o petición previamente solicitada y disminuye el número de procesos del servidor.



¡RECUERDA!

El archivo **robots.txt** se almacena en la raíz del servidor y permite a los buscadores de internet indexar el contenido de una web en los resultados mostrados por ellos. Además, permite el acceso a estos buscadores para que relacionen la búsqueda con el contenido de una web, ofreciendo el nombre de dicha web como resultado de la búsqueda.

Estas son algunas de las configuraciones básicas que se deben realizar de forma genérica en cualquier web, por lo que es importante definir y organizar correctamente todos estos aspectos.

Puede resultar útil utilizar test globales o unitarios que permitan comprobar el acceso a todos los servicios ofrecidos.

5.5. PRUEBAS Y DEPURACIÓN

CONCEPTO

Las **pruebas** son el mecanismo por el cual es posible comprobar si una aplicación web cumple con las condiciones y requerimientos especificados.

Estas pruebas forman parte de una fase dentro del ciclo de vida de la creación de estas aplicaciones, por lo que **se deben integrar y realizar de manera repetitiva** para poder detectar a tiempo el mayor número posible de errores.

En ocasiones, es bueno tratar de **dividir el código en funcionalidades**, intentando aislar unas partes de otras para que, en caso de producirse algún fallo, este no sea arrastrado de una parte del código a otra. Esto es lo que se conoce como **pruebas unitarias**, en las que solo se ejecuta una parte del código, de manera que el resto de las funcionalidades de la aplicación no se verán afectadas.

La mayor parte de los lenguajes de programación web permiten este tipo de pruebas. Durante el desarrollo de cualquier tipo de aplicación o servicio web, y de forma paralela a las pruebas, es conveniente realizar **procesos de depuración**.

Estos procesos permiten comprobar y realizar un seguimiento de la ejecución en tiempo real de dicha aplicación o página web. De esta forma, es posible comprobar los recursos utilizados por la aplicación. Analizar los valores que van tomando las variables permite verificar que son correctas y controlar el flujo de ejecución.

Para poder realizar estos procesos de depuración, es necesario hacer uso de los IDE que soporten el lenguaje de programación utilizado. Algunos de los más utilizados son **Eclipse y Microsoft Visual Studio**.



ponte a prueba

En ningún caso se aconseja tratar de dividir el código en funcionalidades.

- a) Verdadero
- b) Falso



6

UTILIZACIÓN DE TÉCNICAS DE ACCESO A DATOS

6.1. TECNOLOGÍAS QUE PERMITEN ACCEDER A DATOS DESDE UNA APLICACIÓN WEB

El acceso a base de datos es uno de los puntos clave en el desarrollo y construcción de cualquier tipo de aplicación web. La forma en la que se leen, modifican, almacenan y eliminan estos datos puede tener una gran influencia en el rendimiento de una web.

Existen diferentes **tecnologías que permiten el acceso a los datos**:

- **Data base manager system (DBMS)**: sistema manejador de bases de datos formado por un conjunto de programas que se encarga de gestionar el almacenamiento de una base de datos, como, por ejemplo, SQL Server, MYSQL o Oracle.
- **API**: conjunto de métodos o funciones que permiten, mediante un grupo de llamadas, el acceso a determinados servicios de la aplicación web.
- **Lenguajes de programación**: cualquier lenguaje que permita interpretar la sintaxis de acceso a datos. Algunos de los más utilizados son PHP, Prolog, ASP, ActionScript, ADA, Python, Pascal, C#, Basic, y Java.
- **Mapeo de datos**: el mapeo objeto-relacional (ORM) permite relacionar los objetos de una aplicación con su correspondiente tabla en una base de datos relacional. De esta forma, cada clase en el modelo de dominio tendrá su tabla en el modelo de datos. Es la técnica más extendida y más usada en el desarrollo de aplicaciones web. Existen una gran cantidad de *frameworks* que permiten hacer uso de esta técnica, como Hibernate, Java Persistence o Doctrine.

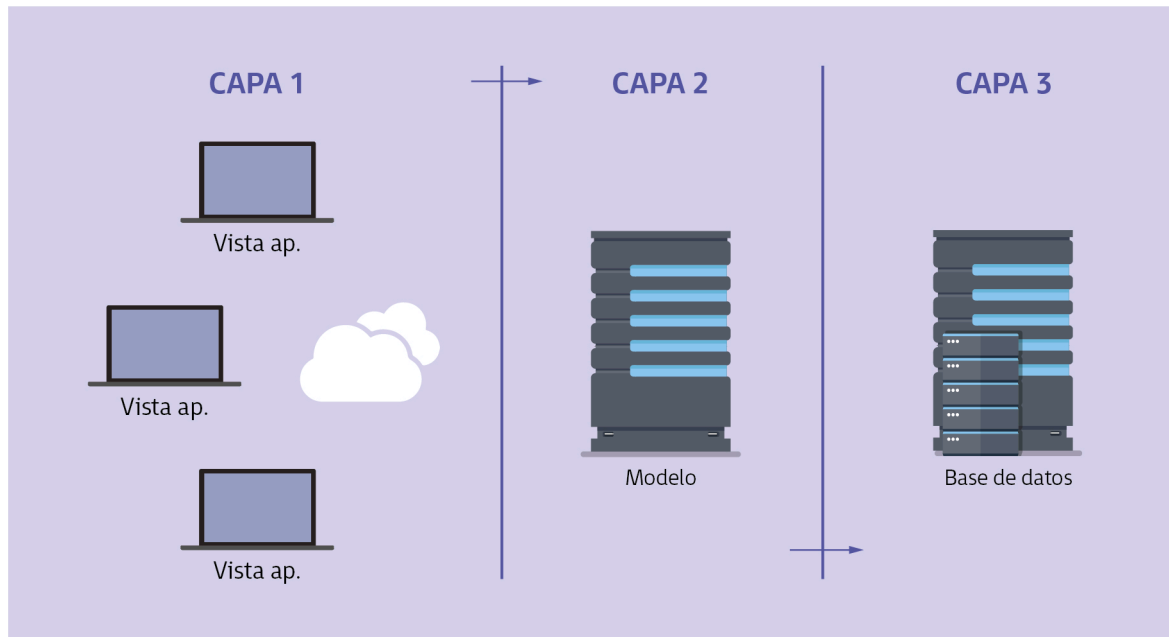
Estas tecnologías garantizan un mecanismo de acceso a los datos. Es importante elegir cuál emplear dependiendo de los requisitos de la aplicación, ya que será el vínculo de conexión entre esta y la base de datos.

La creación de una base de datos con sus tablas tiene como objetivo hacer uso de los registros en ellas almacenados para posteriormente mostrar esa información en las páginas web.

Para obtener los datos y mostrarlos en un HTML de la web, se deben seguir los siguientes pasos:

- En necesario que el *software* que se encarga de interpretar el lenguaje de programación escogido se comunique con el sistema gestor de la base de datos.

- Una vez que se ha identificado y autorizado el gestor de base de datos, se prepararán las sentencias SQL que se ejecutarán.
- Estas sentencias se ejecutarán en la base de datos para obtener el conjunto de datos solicitado.
- El conjunto de datos obtenido contiene todos los registros, que se irán recorriendo mediante algunas de las estructuras de control proporcionadas por el lenguaje de programación utilizado.



6.2. ESTABLECIMIENTO DE CONEXIONES

En informática, se podría definir una **conexión a base de datos** como la forma en la que un servidor de base de datos y su aplicación cliente se comunican entre sí.

Dicha aplicación hace uso de la conexión a base de datos para solicitar una información que será facilitada por el servidor. Una base de datos puede almacenar un conjunto de tablas en cualquier tipo de dispositivo de almacenamiento.

CONCEPTO

Las **conexiones** permiten ejecutar una serie de comandos que permiten obtener como resultado toda aquella información solicitada. Para ello, es necesario disponer de una conexión abierta y estable con la base de datos.

Una vez que la conexión se ha establecido correctamente, es posible solicitar todos los datos necesarios y, posteriormente, cerrar dicha conexión en cualquier momento.

Las conexiones son gestionadas por un controlador, y este proporciona una instancia al servidor donde está alojada la base de datos. Dicha instancia permitirá el acceso una vez que se haya identificado el cliente de esta petición.

El primer paso que seguir será **configurar los parámetros de conexión entre el entorno donde estará alojada la aplicación web y el entorno donde se almacenarán los datos**. Previamente, se debe conocer cuál será la arquitectura que reflejará esto, así como las distintas técnicas empleadas para ello.

Dependiendo del lenguaje de programación empleado en la aplicación web, se utilizarán las funciones correspondientes para crear dicha conexión, teniendo en cuenta que el servidor donde se almacena la base de datos debe ser capaz de interpretar el mismo lenguaje.

Los **parámetros** que se deben incluir **dentro de las funciones de conexión** son:

- **Host:** es el servidor o equipo en el que se encuentra instalado el sistema gestor de base de datos (como, por ejemplo, MYSQL). Probablemente, en ese mismo ordenador estará instalado el intérprete del lenguaje. En el caso de que la aplicación esté alojada en el propio equipo de forma local, el nombre del *host* que se utilizará es "localhost" de forma genérica.

- **Usuario:** es necesario tener permisos de acceso antes de ejecutar las sentencias SQL, por lo que en este campo se seleccionará el usuario con autorización para ello. Por lo general, este será por defecto *root*.
- **Contraseña:** contraseña asociada al usuario con autorización de acceso en MySQL.
- **Base de datos:** nombre con el que se ha identificado al esquema de la base de datos.

```
mysqli_connect('localhost', 'usuario', 'contraseña', 'bd');
```

Con estos parámetros, se establecerá la conexión con la base de datos correspondiente y, al estar autorizado el usuario, será posible preparar y ejecutar las sentencias SQL.

6.3. EJECUCIÓN DE SENTENCIAS SQL. RECUPERACIÓN Y EDICIÓN DE INFORMACIÓN

Hasta el momento, se ha explicado que se utilizarán sentencias **SQL**. El uso de este lenguaje es un **estándar para el acceso a base de datos**. Por este motivo, es la forma en la que el sistema gestor de la base de datos comprenderá cuáles son las peticiones y el conjunto de datos requeridos por el cliente.

Las **sentencias de recuperación y edición de información** son las recogidas dentro el **data manipulation language**. El lenguaje de manipulación de datos más utilizado actualmente es SQL.

Existen diferentes sentencias dependiendo del tipo de acción que se quiera ejecutar en la base de datos. Se pueden clasificar en tres **tipos de comandos SQL**:

- **DDL (data definition language):** permite crear y definir nuevas bases de datos.
- **DML (data manipulation language):** permite generar consultas para recuperar, ordenar y filtrar los datos.
- **DCL (data control language):** define los permisos sobre la base de datos.

Los **comandos** empleados para dicho propósito son:

- **SELECT:** selecciona los datos que cumplan el criterio establecido para, posteriormente, mostrar este resultado. Este comando obtiene filas de la base de datos.

```
SELECT * FROM nombreTabla;
```

ponte a prueba

¿Qué instrucción SQL se utiliza para realizar modificaciones en un conjunto de datos almacenados en una tabla?

- Select
- Insert
- Alter
- Update

- **UPDATE:** permite realizar modificaciones en un conjunto de datos. Esto actualizará los registros o campos de una tabla.

```
UPDATE nombreTabla SET columna1 = valor1, columna2 = valor2 ..., columnN  
= valorN WHERE [condicion];
```

- **INSERT:** se utiliza para crear nuevos registros en una tabla específica, que se añadirán al final de las columnas ya existentes en dicha tabla.

```
INSERT INTO nombreTabla (columna1, columna2, columna3, ...)  
VALUES (valor1, valor2, valor3, ...);
```

- **DELETE:** elimina un conjunto de datos de una tabla específica.

```
DELETE FROM nombreTabla WHERE condicion;
```

La utilización de estos comandos se asigna a una variable. Este proceso se conoce como preparar una orden de consulta para, posteriormente, ser ejecutada y almacenada en la variable especificada.

A partir de estos comandos, es posible obtener prácticamente la totalidad de los datos que conformarán la parte dinámica de una aplicación web.





6.4. EJECUCIÓN DE SENTENCIAS SQL. UTILIZACIÓN DE CONJUNTOS DE RESULTADOS

Tan importante como obtener los datos es saber agruparlos y gestionarlos para mostrarlos de la mejor manera posible en la interfaz.

Existen **diferentes sentencias SQL** que permiten esto:

- **GROUP BY:** permite agrupar los resultados siguiendo un criterio. De esta forma, todos los resultados son filtrados, mostrándose aquellos que cumplen con el criterio establecido.

```
SELECT columna1, columna2, ... FROM nombreTabla  
GROUP BY columnaDeAgrupacion;
```

- **DISTINCT:** puesto que los resultados devueltos pueden contener registros duplicados, esta sentencia nos devuelve valores únicos, es decir, aquellos que son diferentes.

```
SELECT DISTINCT columna1, columna2, ... FROM  
nombreTabla
```

- **ORDER BY:** ordena los registros obtenidos siguiendo un criterio. Esta sentencia permite, a su vez, un parámetro adicional al final de la consulta, pudiendo indicar si se desea que el orden sea descendente (*desc*) o ascendente (*asc*).

```
SELECT columna1, columna2, ... FROM nombreTabla  
ORDER BY columna1, columna2, ... ASC|DESC;
```

- **COUNT**: indica el número de registros o filas de una tabla devueltos que cumplan con el criterio establecido.

```
SELECT COUNT(nombreColumna) FROM nombreTabla
WHERE condicion;
```

- **MAX y MIN**: calculan el valor máximo y mínimo respectivamente de entre todos los registros obtenidos.

```
SELECT MIN(nombreColumna) FROM nombreTabla WHERE
condicion;
```

```
SELECT COUNT(nombreColumna) FROM nombreTabla
WHERE condicion;
```

- **LIMIT**: cuando se prevé que la cantidad de registros devueltos puede ser muy grande, es posible limitarlos. Esta sentencia es útil cuando el contenido a mostrar en la web es parcial o es necesario que sea paginado.

```
SELECT columna1, columna2, ... FROM nombreTabla
LIMIT 3;
```

Estas son las principales sentencias SQL que se usan en la gestión de los resultados obtenidos de una consulta. Estos datos obtenidos como resultado se devolverán a la interfaz desde la que se ejecutó el servicio.

6.5. CREACIÓN DE APLICACIONES WEB CON ACCESO DE ESCRITURA A BASES DE DATOS

Como ya se ha mencionado en un punto anterior, es importante definir los permisos de acceso de un cliente de una aplicación web en la base de datos. La aplicación solo podrá hacer uso de aquellos servicios a los que tiene permiso.

Este aspecto es uno de los puntos críticos a la hora de garantizar la consistencia en la base de datos, puesto que permitir que una aplicación pueda realizar modificaciones debe estar supervisado, bloqueando todas aquellas peticiones que no se tengan controladas.

Para definir los tipos de permiso, en la mayor parte de los sistemas gestores de bases de datos es necesario establecer una serie de roles que determinarán los permisos asociados a ellos.

Así, **se indicará de forma explícita cuáles de estos roles podrán leer, escribir o ambas cosas en la base de datos.**

En el caso de querer asignar permisos de escritura, bastará con indicar el usuario y los comandos que puede ejecutar en la base de datos. Es posible establecer estos permisos desde una interfaz proporcionada por un sistema gestor de bases de datos, o bien a través de código dentro de la aplicación.

En función de los permisos establecidos, es posible diferenciar **dos tipos de usuarios** que podrán añadir registros en las tablas de la base de datos:

- **Administradores:** son los administradores del contenido de un sitio web. Serán los encargados de gestionar la información para que el usuario la pueda visualizar.
- **Usuarios:** los usuarios clientes del sitio web podrán, de forma controlada, añadir registros mediante el envío de formularios u otros campos habilitados para ello.

Dependiendo del gestor de base de datos utilizado, las tablas de usuarios y sentencias pueden variar.

Vamos a ver la diferencia en las sentencias más básicas para gestionar usuarios en una base de datos entre dos de los posibles gestores que utilizan diferentes lenguajes de programación:

- Las bases de datos cuentan con una serie de usuarios creados de forma automática con la instalación del servicio. Dependiendo del sistema gestor utilizado, puede variar el número de usuarios y los permisos de estos, aunque un punto que siempre es común en todos los sistemas gestores es una tabla con la información de los usuarios.

MySQL	SELECT * FROM mysql.user;
ORACLE	SELECT * FROM SYS.DBA_USERS;



- Es recomendable que los usuarios creados por el gestor de forma automática se utilicen para gestionar la base de datos como administrador. Para un correcto funcionamiento del sistema, se recomienda crear diferentes usuarios con los permisos necesarios para la labor que van a desempeñar (tantos como el aplicativo necesite).

MySQL	CREATE USER 'usuario'@'localhost' IDENTIFIED BY 'password';
ORACLE	CREATE USER usuario IDENTIFIED BY 'password';

- Para realizar una correcta configuración de los usuarios, hay que restringir los permisos de estos dependiendo de la labor que vayan a desempeñar, de este modo, evitamos que el usuario pueda realizar acciones no deseadas (*SQL injection*).

MySQL	GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP ON base_de_datos.tabla TO 'usuario'@'localhost';
ORACLE	GRANT SELECT (campos_tabla, ...) ON tabla TO usuario@localhost;

- Finalmente, comentamos algunas sentencias a tener en cuenta en la gestión de usuarios.

Elimina un usuario.	DROP USER 'usuario'@'localhost';
Refresca los privilegios de los usuarios de la base de datos.	FLUSH PRIVILEGES;
Elimina permisos concretos de un usuario de la base de datos.	REVOKE UPDATE, DELETE ON bbdd.tabla FROM 'usuario'@'localhost';
Elimina todos los permisos de un usuario de la base de datos.	REVOKE ALL PRIVILEGES ON bbdd.tabla FROM 'usuario'@'localhost';

El **proceso de escritura en la base de datos**, por tanto, queda dividido en dos etapas:

- El cliente realiza el envío de la información a través de un formulario HTML en la página web, es decir, se trata de una funcionalidad realizada en un archivo HTML.
- Esta información es procesada y validada en un fichero cuya extensión es la establecida por el lenguaje de programación utilizado. Este interpretará todos los datos y ejecutará la consulta SQL, que añadirá los registros correspondientes en la base de datos.

6.6. TRANSACCIONES

CONCEPTO

Las **transacciones** dentro de una base de datos son un conjunto de órdenes cuyas tareas se ejecutan de forma independiente.

ponte a prueba

¿Cómo se denomina el mecanismo de SQL que permite eliminar todas las operaciones realizadas??

- a) BEGIN TRAN
- b) ROLLBACK TRAN
- c) DELETE TRAN
- d) COMMIT TRAN

Estas transacciones en SQL se han desarrollado bajo el cumplimiento del principio **ACAD (atomicidad, consistencia, aislamiento y durabilidad)**. Este principio garantiza el uso correcto de dichas transacciones, de forma que se puede asegurar que, o se cumplen todas las operaciones requeridas, o no se realizará ninguna.

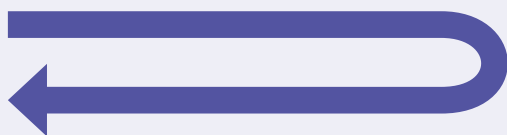
SQL provee una serie de **mecanismos** relacionados con estas:

- **BEGIN TRAN | TRANSACTION**: indica que va a comenzar una transacción.
- **COMMIT TRAN | TRANSACTION**: indica el final de una transacción que ha resultado correcta.
- **ROLLBACK TRAN | TRANSACTION**: permite eliminar todas las operaciones realizadas, volviendo a un punto estable previo a la realización de una determinada transacción.

Estas transacciones **permiten mover datos de un lugar a otro** manteniendo en un estado consistente la base de datos.

Un buen **ejemplo** de transacción podría ser realizar una transferencia de dinero de una cuenta a otra en un banco. Esta tarea conllevará dos operaciones que se realizarán de forma independiente. Primero, una de ellas restará la cantidad requerida en la cuenta de origen. En segundo lugar, se añadirá la cantidad en la cuenta de destino. En caso de que alguna de estas operaciones no se cumpla, ninguna de ellas quedará realizada.

En caso de fallo de la conexión o del sistema la transacción vuelve las operaciones a su estado inicial.



Si la tarea ejecutada sucede de forma correcta sin ninguna incidencia esta se finaliza y almacena los datos de forma permanente.



ROLLBACK

COMMIT

Al igual que en los puntos anteriores referentes a la gestión de bases de datos, dependiendo del gestor utilizado pueden variar algunas de las sintaxis utilizadas en los ejemplos.

6.7. PRUEBAS Y DOCUMENTACIÓN

CONCEPTO

Las **pruebas** son el mecanismo por el cual es posible comprobar si una aplicación web cumple con las condiciones y requerimientos especificados.

Estas pruebas forman parte de una fase dentro del ciclo de vida de la creación de estas aplicaciones, por lo que se deben integrar y realizar de manera repetitiva para poder detectar a tiempo el mayor número de errores posible.

```
USE nombreBaseDatos BEGIN TRAN UPDATE  
nombreTabla SET columna=valorColumna1 WHERE  
condicion COMMIT TRAN;
```

En ocasiones, es una buena idea dividir el código en funcionalidades, tratando así de aislar unas partes de otras para que, en caso de producirse algún fallo, este no sea arrastrado de una parte del código a otra. Esto es lo que se conoce como pruebas unitarias, en las que solamente es ejecutada una parte del código, de manera que el resto de las funcionalidades de la aplicación no se verán afectadas. La mayor parte de los lenguajes de programación web permiten este tipo de pruebas.

Durante el desarrollo de cualquier tipo de aplicación o servicio web, y de forma paralela a las pruebas, es conveniente realizar **procesos de depuración**. Estos procesos permiten comprobar y realizar un seguimiento de la ejecución en tiempo real de dicha aplicación o página web. De esta forma, es posible comprobar los recursos utilizados por la aplicación. Analizar los valores que van tomando las variables permite verificar que son correctas y controlar el flujo de ejecución.

Para poder realizar estos procesos de depuración, es necesario utilizar los IDE que soporten el lenguaje de programación utilizado, algunos de los que lo permiten son Eclipse y Microsoft Visual Studio.





7

PROGRAMACIÓN DE SERVICIOS WEB

7.1. CARACTERÍSTICAS Y ÁMBITOS DE APLICACIÓN

CONCEPTO

El **servicio web** puede ser definido como un conjunto de protocolos que permiten comunicar diferentes aplicaciones a través de internet, permitiendo un intercambio de información entre ellas. La invocación de estos servicios web requiere del uso de los estándares web.

Las principales **características** con las que debe contar un servicio web son:

- Un servicio web debe estar **accesible desde la red** a través de protocolos como HTTP y codificando la información en un lenguaje que cualquier tipo de cliente pueda interpretar.
- Es recomendable que un servicio web contenga una **descripción de sí mismo**. De esta forma, un cliente puede conocer cuál es la función de un determinado servicio web.
- Además de ser accesible, un servicio debe **estar bien localizado**. Esto permitirá a una aplicación web localizar el servicio de forma automática.

Los servicios web nacen de la necesidad de comunicar diferentes aplicaciones que tienen distintos lenguajes de programación, así como distintas plataformas utilizadas.

En la actualidad, todo el mundo se comunica a través de internet, por lo que un servicio web será el medio de comunicación con el que es posible obtener y ofrecer una determinada información a un número elevado de personas de forma dinámica basándose en los criterios que las aplicaciones requieren.

A lo largo de los años, la evolución y desarrollo de *software* para diferentes plataformas ha traído consigo la necesidad de centralizar el acceso a un determinado recurso y que este sea capaz de comunicarse con todas estas plataformas.

Las **organizaciones OASIS** (Organization for the Advancement of Structured Information Standards) son los comités responsables de la arquitectura y reglamentación de los servicios web.



ponte a prueba

Verdadero o Falso: Un servicio web se basa en el uso de protocolos que permiten el intercambio de información entre aplicaciones mediante Internet, pero que, al haber sido desarrollados por diversas empresas, no se acogen a ningún estándar.

a) Verdadero

b) Falso



7.2. VENTAJAS

- Son independientes de la plataforma y el lenguaje de programación utilizados.
- Permiten establecer un canal de comunicación entre aplicaciones a través de la web.
- El continuo desarrollo tecnológico hace necesario adaptarse rápidamente a la gran oferta de dispositivos desde los que se puede acceder a un servicio web. Un servicio web no debe verse afectado cuando existen cambios en los diferentes dispositivos, pues funcionan de manera independiente.
- El proceso de creación e integración de un servicio web no requiere de un coste computacional elevado.
- La mayor parte de los sitios web trabajan ofreciendo estos servicios, su estandarización y garantías en el mercado están ya comprobadas.
- Gracias a esta versatilidad, se ha abierto un gran abanico de oportunidades hacia diferentes tipos de negocio, ayudándolos a llevar a cabo sus actividades comerciales.
- Han reducido notablemente el tiempo de desarrollo de aplicaciones, cualquier tipo de aplicación puede utilizar este servicio sin necesitar de una configuración de código adaptada a cada tipo de aplicación.
- Puesto que la comunicación no requiere de un elevado coste de computación, liberan de carga a los servidores.
- Dado que el protocolo utilizado es HTTP, casi siempre es permitido por los *firewalls* de red. Esto evita muchos problemas de denegación de servicios por parte de los clientes.



- El funcionamiento de un servicio web es sencillo y, al funcionar bajo el protocolo HTTP, permite conseguir avances mucho más rápidos en esta tecnología frente a otras.
- Los procesos de actualización son más sencillos, permitiendo mantener cualquier sitio web con el último desarrollo, ya que este no afectará a los clientes.
- Permiten múltiples conexiones de usuarios al mismo tiempo de manera eficaz.

7.3. MECANISMOS Y PROTOCOLOS IMPLICADOS

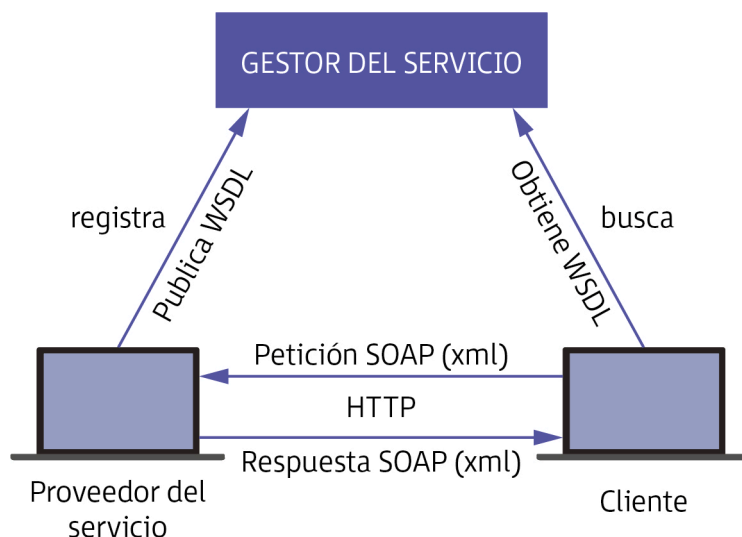
Todos los protocolos que utilizan los servicios web están ya estandarizados y controlados.

CONCEPTO

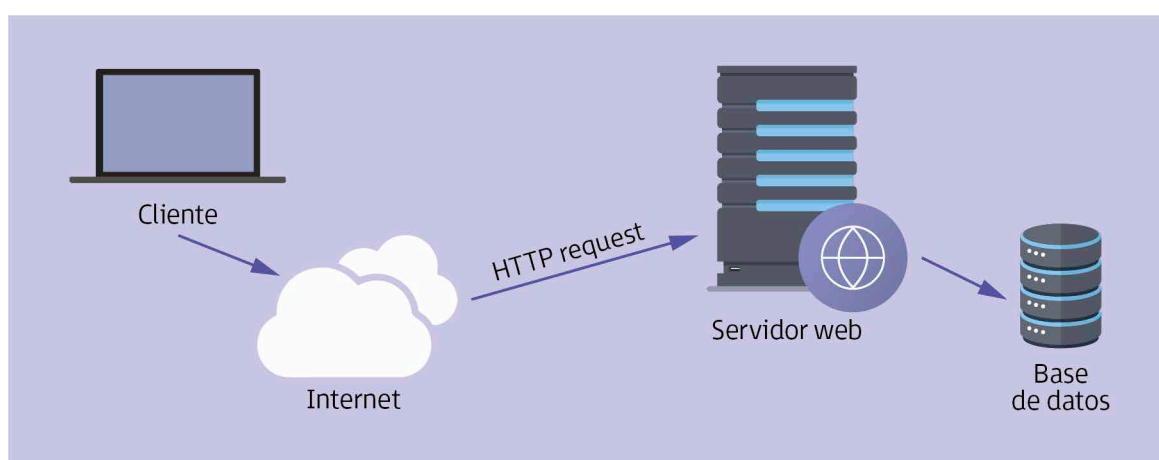
El lenguaje utilizado en estos servicios es **XML (Extensible Markup Language)**, un tipo de formato establecido para realizar este intercambio de información. El protocolo utilizado para este intercambio es HTTP. Además, existen otros protocolos, como XML-RPC.

Dependiendo del tipo de servicio web creado, se pueden distinguir **estos tipos de protocolos**:

- **Simple Object Access Protocol (SOAP)**: utiliza mensajes XML para el intercambio de información y, a su vez, retornará un XML como documento de respuesta. Es necesario establecer por parámetros los datos requeridos como información. Es una evolución del protocolo XML-RPC.



- **File Transfer Protocol (FTP)**: protocolo utilizado para la transferencia de archivos.
- **Simple Mail Transfer Protocol (SMTP)**: protocolo utilizado para el envío de correos electrónicos.
- **UDDI (Universal Description, Discovery and Integration)**: protocolo para publicar la información de los servicios web. Permite comprobar qué servicios web están disponibles.
- **REST (Representational State Transfer)**: mediante el uso de esta arquitectura y el protocolo HTTP, proporciona una API de utilización de los métodos asociados a esta arquitectura. Los métodos son GET, POST, PUT, DELETE.



Estos son algunos de los protocolos más utilizados, siendo **REST** y **SOAP** los más destacados dentro del apartado de comunicación a través de servicios web. Existen diferencias entre ambos protocolos, la elección de uno u otro vendrá determinada por el mecanismo de acceso a los recursos que se desea establecer.

Puesto que su uso está muy extendido, a continuación se detallará más cuáles son las **diferencias** entre ellos.

- En **SOAP** el mensaje que se desea enviar viene encapsulado en un **XML**, este mensaje incluye el nombre del método al que se hace la petición y los argumentos que sean necesarios. Implica un esquema de desarrollo de acceso a las API más complejo. Es importante destacar que la **seguridad** ofrecida es elevada.
- **REST** es fácil de entender por el protocolo **HTTP** y hace un mejor uso del ancho de banda. A diferencia de SOAP, no almacena estado, por lo que las peticiones no mantienen unos registros de los resultados devueltos anteriormente para otras peticiones. Esto conlleva que cada petición debe enviar todos los datos necesarios para realizar las peticiones, incluidos, por ejemplo, los datos de autenticación

de un usuario. REST, para poder mantener estos datos y no realizar las mismas peticiones una y otra vez, hace **uso de los sistemas de cachés**. Esto permite que los servidores no hagan uso de una gran cantidad de recursos destinados a insertar y eliminar información de manera continuada. El formato de respuesta de REST no es solamente XML, sino que permite responder a otros, como JSON.

- La **simplicidad de implementación de REST**, así como por la escalabilidad que ofrece al trabajar con sistemas multiplataforma, ha sido clave para que haya llegado a ser el protocolo más usado hoy en día en el acceso a los recursos de servicios web. Por este motivo, la mayor parte de los servicios web eligen REST como protocolo de comunicación.
- Si lo que se desea es una aplicación web cuya funcionalidad está orientada a **sistemas de seguridad, bancos, telecomunicaciones, etcétera**, el protocolo escogido será en la mayoría de los casos **SOAP**.

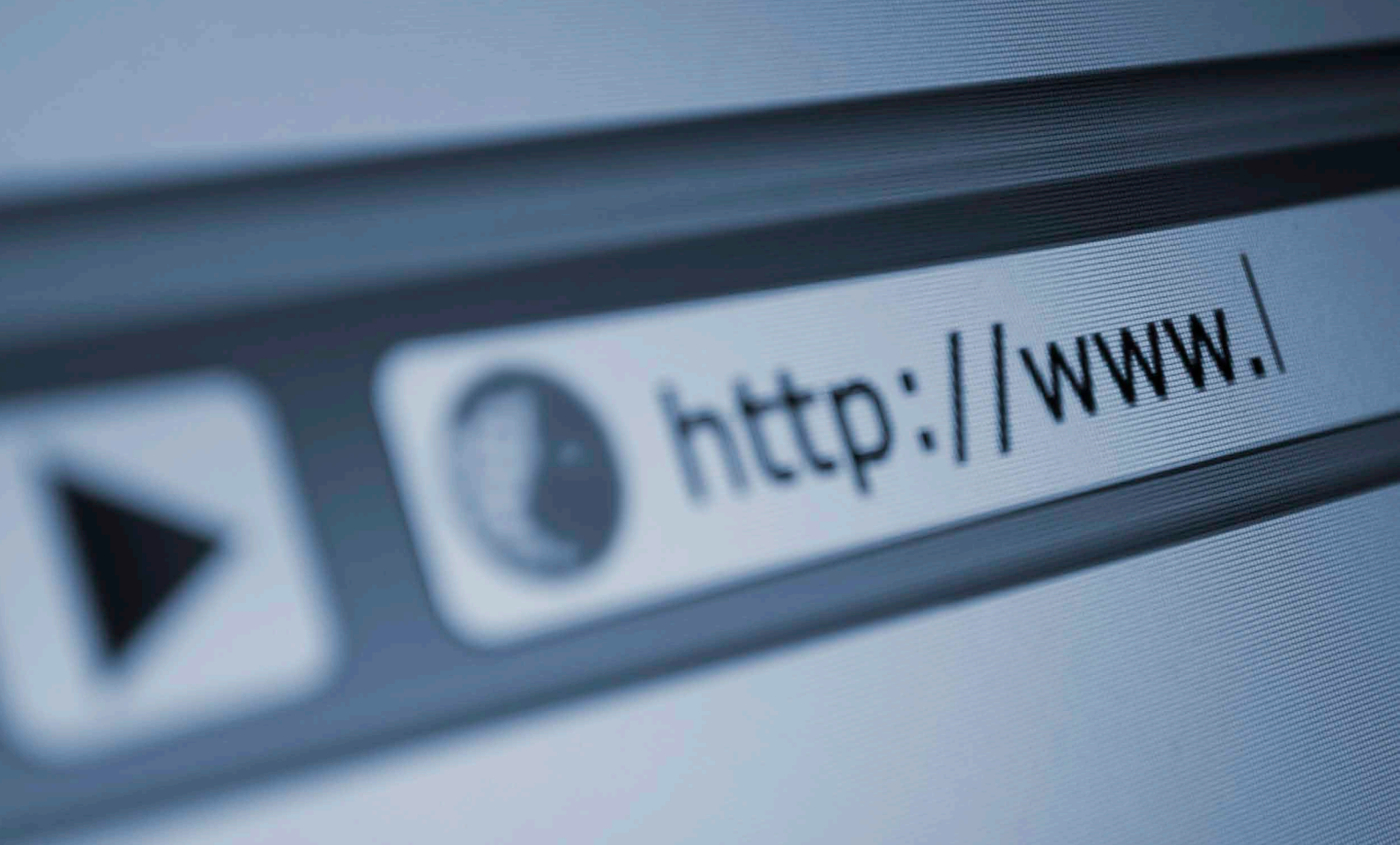


ponte a prueba

¿Cuál de estos protocolos no se utiliza en ningún servicio web?

- a) FTP
- b) UDP
- c) SMTP
- d) UDDI





7.4. GENERACIÓN DE UN SERVICIO WEB

La creación y configuración de un servicio web debe seguir una serie de pasos. Dependiendo del lenguaje utilizado, los pasos a seguir podrán ser consultados en la API correspondiente.

Desde esta API se facilitan todos los métodos de creación de un nuevo servicio. Es necesario generar el código que ejecutará dicho servicio en primer lugar. El segundo paso es identificar la interfaz asociada al servicio. Después se tendrá que elegir una ruta de acceso a él en la web. Lenguajes como ASP.NET, PHP o Java disponen de clases que automatizan estas tareas y permiten crear servicios de una forma sencilla.

Los pasos para la **creación y puesta en marcha de un servicio web** son:

- **Configurar la base de datos:** se deben configurar todas las columnas y tablas que proveerán el conjunto de resultados al cliente.
- **Crear el servicio:** esta parte se dividirá en la creación de tres archivos diferentes:
 - **Archivo de configuración:** archivo de configuración inicial que definirá los parámetros de configuración de la base de datos.

- **Archivo de conexión:** ejecutará la conexión con la base de datos con los parámetros devueltos en el archivo anterior.
- **Archivo de petición:** realizará la petición de los datos mediante consultas SQL. Este archivo definirá el formato en el que se devuelven los datos para mostrarlos. Estas consultas se recogerán en una clase diferente.
- **Archivo de resultados:** instanciará la clase creada y llamará al método correspondiente para almacenar el conjunto de datos devueltos.
- **Comprobación de resultados:** se ha de configurar el mensaje que retornará el servidor para la comprobación correcta de los resultados devueltos.

Un aspecto importante y que se debe tener en cuenta tras la creación de un servicio web es que el espacio de nombres asociado por un servicio web debe ser único en la aplicación.

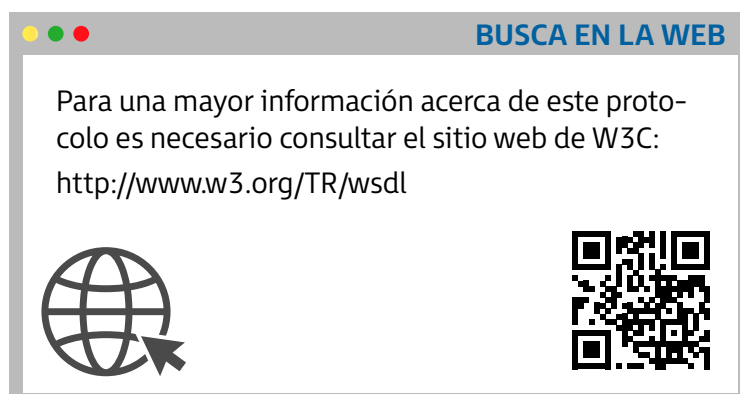
En algunas ocasiones, es posible comprobar la ejecución de un servicio del lado del cliente observando cómo la URL del navegador ha invocado un nuevo archivo con su extensión asociada:

`www.ejemplo.com/nombreServicio.extension`

Actualmente, la creación de un servicio web se ha automatizado de una forma notable. Lenguajes como ASP.NET, a través de sus herramientas de desarrollo, proveen una interfaz que permite la creación de un servicio de manera fácil y rápida.

7.5. DESCRIPCIÓN DE UN SERVICIO

La descripción de servicio web es un documento con extensión XML escrito en lenguaje XML denominado WSDL. Es decir, WSDL describe el formato de los mensajes que serán enviados.



La descripción de un servicio debe documentar las acciones que tienen que realizar los posibles clientes para comunicarse con el servicio. Esto ayuda al cliente a prever el resultado que devolverá la invocación a un determinado servicio.

Es necesario especificar cómo será la comunicación, tanto para las de tipo bidireccional, en las que se espera una respuesta de contestación, como para las unidireccionales. En esta situación, el cliente no recibirá ningún mensaje del servicio web, ni siquiera los mensajes de error.

Además de las definiciones de formato de mensaje y modelos de mensajería, la descripción de servicio puede contener opcionalmente la dirección asociada a cada punto de entrada del servicio web XML. Esta dirección puede ser una URL o una dirección de correo electrónico.

Como parámetros principales, la descripción del servicio debe incluir el nombre del método, los parámetros que se deben enviar y su tipo y el valor de retorno de la respuesta del servidor.

7.6. INTERFAZ DE UN SERVICIO WEB. PRUEBAS

La **interfaz de un servicio web** es uno de los aspectos más importantes. Es la parte que mejor definirá los diferentes tipos de servicio ofrecidos en la aplicación. Si un cliente es capaz de comprender y conocer de antemano cómo y dónde debe acceder para obtener toda la información que desea, entonces se podrá decir que dicha interfaz ha quedado bien definida.

Esta interfaz normalmente utilizará XML para su programación, pero esta labor es muy costosa y es posible que pierda dinamismo y resulte menos comprensiva. Por ello, algunos **IDE como Eclipse ofrecen una serie de herramientas WSDL para crear de forma más eficiente estas interfaces usando XML como lenguaje de mensajería**. La interfaz delega las peticiones recibidas a la lógica de negocio y procesa todas las peticiones de los clientes.

Otro apartado para tener en cuenta es **la realización de pruebas del servicio creado**. El tiempo de respuesta es un punto clave de las pruebas, se debe garantizar que el servidor tiene un grado de disponibilidad alto. También se debe comprobar que la mayoría de las peticiones o su totalidad son recibidas y procesadas, permitiendo así múltiples peticiones simultáneas al servicio y respondiendo correctamente a ellas.

Esta interfaz ejecutará por debajo todos los aspectos detallados en la descripción del servicio vista en el apartado anterior. De forma general, es habitual encontrar un servicio como un botón con un nombre descriptivo del resultado que devolverá tras su ejecución.

Algunos de los elementos que se deben tener en consideración en una fase de pruebas de un servicio web son:

- **Sistema de navegación:** definir los pasos que se deben aplicar durante la ejecución de un servicio.
- **Áreas de contenidos:** en qué posición de la web se colocarán los elementos de interacción de ejecución de estos servicios.
- **Áreas de interacción:** elementos que encapsularán el nombre o logo que iniciará el evento asociado al servicio, como, por ejemplo, botón, hipervínculo, etcétera.
- **Experiencia de usuario:** es una buena práctica analizar cómo ha sucedido cada una de las experiencias de navegación de los clientes hasta la petición de un servicio. Esto puede ayudar en la realización de un diseño web más eficiente.



ponte a prueba

El tiempo de respuesta no es uno de los puntos clave de las pruebas del servicio creado.

a) Verdadero

b) Falso





8

GENERACIÓN DINÁMICA DE PÁGINAS WEB INTERACTIVAS

8.1. EJECUCIÓN DEL CÓDIGO CLIENTE Y SERVIDOR

Para poder ejecutar una aplicación, es necesario publicar el dominio en internet para que este sea accesible desde una web. También es posible ejecutar dicha aplicación para uso personal dentro de un equipo. Para ello, será necesario que el ordenador actúe, a su vez, de servidor donde se alojará dicha aplicación.



¡RECUERDA!

Los diferentes archivos son **alojados en un servidor con una dirección IP de acceso a la web.**

El cliente abre su navegador y selecciona la dirección o nombre vinculado a la aplicación web. De esta forma comienza la ejecución del código cliente.

Los diferentes archivos de código de la aplicación se ejecutarán de forma automática en respuesta a la petición de un determinado servicio por parte del cliente, o bien se ejecutarán periódicamente los *scripts* programados para ello.

Por tanto, el código asociado al cliente se corresponde con todos los aspectos web, así como la información que el cliente incluye en los elementos habilitados, permitiendo al servidor componer la totalidad de la web a mostrar según se realizan peticiones. De alguna manera, es posible entender que el cliente es capaz de generar nuevas porciones de código que el servidor interpretará para mostrar los resultados de forma dinámica en cada una de las solicitudes realizadas.

Por otro lado, el servidor es el encargado de ejecutar los ficheros o *scripts* que el evento del servicio web ha solicitado. Estas ejecuciones dependen de manera general del cliente, pues son aquellas que están asociadas a elementos de una web.

También existen algunos *scripts* que se ejecutan de forma automática, estos servicios suelen tener una carga elevada de procesamiento de datos o se aplican en la comprobación de algunas reglas de configuración general de aplicación, es decir, servicios relacionados con los administradores del sitio web. Por ese motivo, su ejecución será controlada y monitorizada periódicamente.



8.2. LIBRERÍAS Y TECNOLOGÍAS RELACIONADAS

Para poder desarrollar una web dinámica, se debe especificar y programar usando una tecnología. Esta determinará cuáles son los requisitos que permitirán ejecutar una aplicación.

Algunas de las **tecnologías** que son empleadas para este fin y los **requisitos** necesarios son:

- **ASP**: un sistema operativo Windows, con el servidor web IIS.
- **ASP.NET**: Windows como sistema operativo, con IIS y el *framework* de .NET.
- **PHP**: el sistema operativo es indiferente y el servidor necesita tener instalado el módulo de PHP que permita interpretar el código. Una de las prácticas más habituales es tener PHP sobre Linux y Apache.
- **JSP**: permite su uso sobre cualquier sistema operativo. Necesita un servidor web como Apache Tomcat o Sun Java System Web Server, y alguna versión del J2SDK.

Para poder realizar las pruebas, necesitaremos tener un servidor local instalado o un *hosting* que permita la gestión del sitio web. En caso de realizar las pruebas de manera local, existen tecnologías que permiten emular un servidor dentro de un equipo, como son easyPHP, XAMPP, WAMP, etcétera. Esto proveerá de todo el *software* necesario para programar, por ejemplo, en PHP y MYSQL.

En cuanto a las librerías, su uso dependerá del lugar de aplicación. Por ejemplo, algunas **librerías enfocadas a la ejecución** dentro de la interfaz de la web son:

- **Jquery:** es una librería Javascript cuyo objetivo es facilitar al máximo el trabajo con Javascript.
- **ReactJS:** una librería Javascript del lado del cliente que permite desarrollar interfaces de usuario.
- **PolymerJS:** un *framework* que combina HTML, CSS y Javascript para crear componentes web.
- **Bootstrap:** *framework* para la creación de web con diseño *responsive* basado en HTML5 y CSS3.
- **ATLAS:** *framework* que integra una serie de librerías para el desarrollo del lado del servidor.

8.3. GENERACIÓN DINÁMICA DE PÁGINAS INTERACTIVAS

Es posible definir una página web interactiva como aquella que es capaz de cambiar su aspecto y comportamiento en función de las peticiones que el cliente va realizando.

Para poder realizar estos cambios y que pasen desapercibidos para el usuario, es necesario ejecutar partes de código en el propio navegador.



¡RECUERDA!

El navegador manda una petición al servidor: se ejecuta el código en el servidor y esto genera una nueva página web que será enviada de nuevo al navegador. La página web, entonces, modificará su aspecto, mostrando al cliente la nueva información obtenida.

Este proceso es el que da nombre a este tipo de páginas web, puesto que continuamente son generadas nuevas páginas web con base en las peticiones realizadas por el cliente en la web.

La creación de este tipo de web se basa en el uso del lenguaje HTML, pero es necesario combinar este lenguaje con otros, como PERL CGI, PHP, JavaScript o ASP.

Estos ayudarán a dar dinamismo a los elementos e interacciones del usuario.



ponte a prueba

¿Cuál de las siguientes opciones no es una tecnología que determina cómo se programa y cuáles son los requisitos de ejecución de páginas web dinámicas?

- a) PHP
- b) JSP
- c) ASP
- d) ATLAS



¡RECUERDA!

El **objetivo** de estas webs es proporcionar al usuario información completamente actualizada en cada momento.

Esto ha hecho que los usuarios que entran en estos sitios web sean parte involucrada en la creación de contenido. Es posible ver este concepto como **web colaborativa**, donde el cliente puede añadir comentarios, respuestas, añadir elementos de compra a un carrito o incluso filtrar determinado tipo de contenido, lo que permite a los desarrolladores añadir una gran cantidad de funcionalidades y características a una web.

Tal es la expansión e importancia de este tipo de páginas web que la mayor parte de empresas grandes de internet hacen uso de esta tecnología. Una de las tecnologías que ayudan al dinamismo empleadas en estos sitios web son las **cookies**.

CONCEPTO

Las **cookies** son datos que identifican a un navegador. El navegador almacenará en estos archivos los datos que el servidor le envía, así como el reenvío de los datos por parte del navegador hacia el servidor.

Esto permite que un servidor pueda identificar a un usuario durante la navegación dentro de todos los dominios de este sitio web. Algunos de los datos dinámicos que se pueden mostrar en estos sitios web son, por ejemplo, el nombre de usuario, el número de productos que contiene el carrito de compra o el idioma establecido para ver el contenido del sitio web.



8.4. OBTENCIÓN REMOTA DE INFORMACIÓN. FORMULARIOS

Cuando se quiere mostrar los datos que han sido enviados desde un determinado enlace es posible hacer uso de métodos que utilizan etiquetas HTML, como puede ser PHP. Esto conlleva una espera en la visualización de los datos, pues es posible que primero hayan de ser validados por el servidor. Si se desea ver estos datos al momento (una vez que han sido enviados), se utilizarán formularios. Estos datos son procesados en el navegador y son el paso previo a la obtención de los permisos para hacer uso de los servicios web.



¡RECUERDA!

Los **formularios** pueden ser programados en lenguajes como HTML, Perl, PHP, Java o .NET.

El proceso de obtención de esta información es relativamente sencillo. Todos los formularios cuentan con un método de envío, normalmente este evento es ejecutado a través de un botón en la interfaz. Esta información es validada primero, es decir, se comprueban los datos antes de guardarlos en la base de datos. Es posible añadir estructuras de control que permitan una mejor validación de la información que se va a enviar. De esta forma, se consigue una correcta persistencia en las tablas de la base de datos. Una vez que se han validado, los datos son enviados al servidor.

Algunos de los ejemplos de formularios que se pueden encontrar en una web son los formularios de acceso a una plataforma, en los que se hace el envío de unas credenciales de usuario y contraseña. En este caso, se realiza una consulta con la base de datos para buscar una coincidencia y, si todo es correcto, el servidor responderá con un estado y realizará la redirección a la dirección oportuna.

Estos formularios permiten obtener a los servidores toda la información necesaria para crear una pasarela de acceso a los usuarios, así como para darles permisos para poder hacer uso de los distintos servicios web.

```
<form id="formEdit" class="form" action="/ruta1/ruta2" method="post" enctype="multipart/form-data">
  <div class="form-group">
    <label for="mailAddress">{{'Slect recipient' | tr}}</label>
    <select name="mailAddress" class="select2">
      <option value="nombre@mail.com" selected="selected">{{'Tech support' | tr}}</option>
      <option value="nombre@mail1.com">{{'Academic secretary' | tr}}</option>
      <option value="nombre@mail2.com">{{'Student services' | tr}}</option>
    </select>
  </div>
  <div class="form-group">
    <label for="subject">{{'Case' | tr}}</label>
    <input type="text" name="subject" />
  </div>
  <div class="form-group">
    <label for="message">{{'Message' | tr}}</label>
    <textarea name="message"></textarea>
  </div>
  <div class="button-container">
    <input type="submit" for="submit-upload" class="button" style = "margin-bottom:10px;border-radius:20px;" value="{{'Send' | tr}}>
  </div>
</form>
```



8.5. MODIFICACIÓN DINÁMICA DE LA ESTRUCTURA DE LA PÁGINA WEB

Una de las características que ofrecen las páginas web dinámicas es la forma en la que estas webs son generadas. A diferencia de una web estática, estas páginas **no tienen el contenido ya fijado**. Es decir, las páginas que se mostrarán aún no han determinado qué información será la que el usuario verá. Una vez que se envían los datos al servidor, este devolverá una web con una parte del contenido ya creado, y el resto de contenido lo obtendrá de diferentes sitios. **El intérprete del lenguaje instalado en el servidor será el encargado de transformar la información del usuario en la obtención de la información asociada a dicha petición**, por lo que la web irá cambiando su aspecto según el usuario vaya ejecutando una serie de eventos.

Esto ha permitido la aparición de una gran cantidad de webs que realizan cualquier tipo de venta. La creación de carritos de compra y la presentación de información en tiempo real según la interacción con el usuario son dos buenos ejemplos de webs dinámicas.

Por otro lado, las páginas web de noticias, blogs o foros necesitan actualizar la información mostrada constantemente, por lo que este tipo de webs se ajusta a sus necesidades.



ponte a prueba

En una web dinámica, ¿cómo se denomina el componente encargado de transformar la información del usuario en la obtención de la información asociada a dicha petición?

- a) Compilador
- b) Biblioteca
- c) Protocolo
- d) Intérprete

8.6. PROGRAMACIÓN DE APLICACIONES WEB

La creación completa de una aplicación web está estructurada en una serie de **fases**:

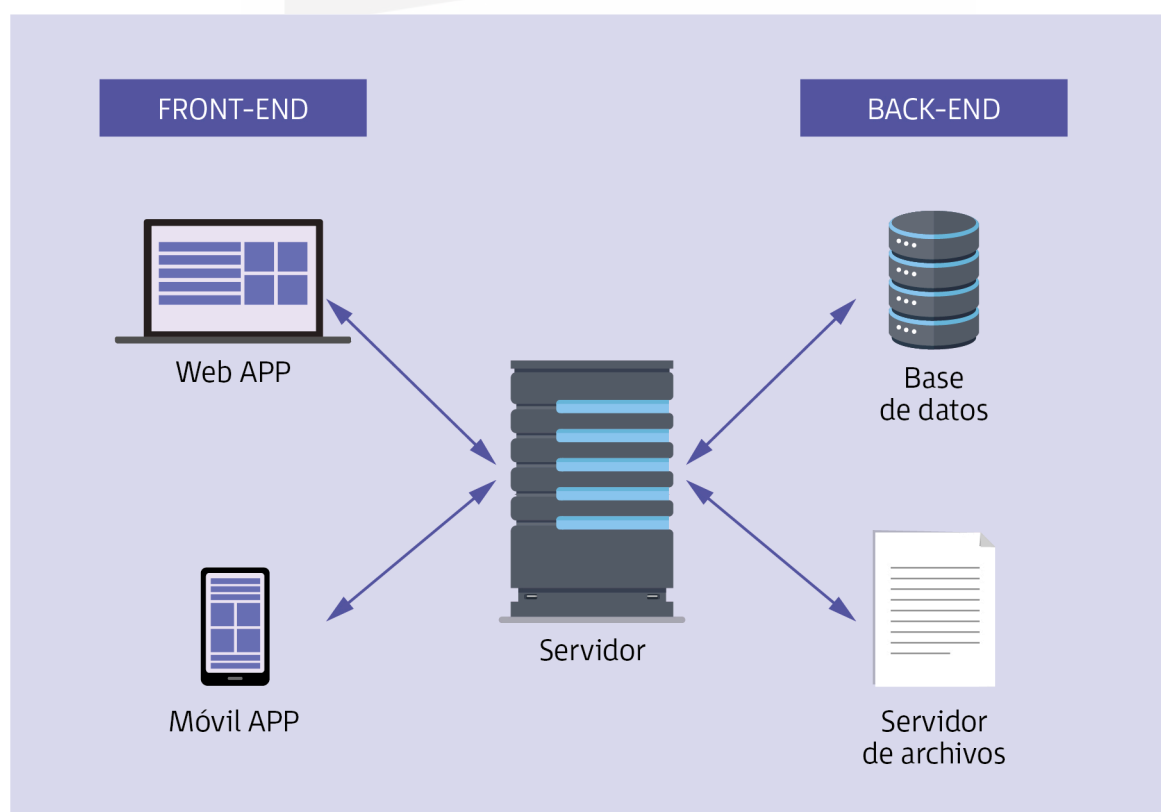
1. **Identificación del sitio donde se alojará el código de la aplicación** y definición del lenguaje de programación en el que se desarrollará la creación de dicha web, tanto para el cliente como para el servidor.
2. **Diseño y construcción de la interfaz de usuario:** a diferencia de la creación de *software*, suele crearse la interfaz antes que la lógica de negocio.
3. **Creación de la arquitectura de la base de datos:** debe escogerse el intérprete del lenguaje y gestor de cliente encargado de la ejecución de sentencias SQL en la base de datos, como, por ejemplo, MySQL.
4. **Desarrollo del código de aplicación:** donde se aplicará la lógica de negocio. Esta determina la funcionalidad de la aplicación web.

5. **Publicación de la web:** para que una página sea accesible, es necesario identificar el nombre que permitirá que todos los clientes accedan a ella.
6. **Pruebas y mantenimiento:** una vez que se ha publicado, es posible encontrar situaciones que mejorar o algún problema, por lo que será necesario repetir esta fase cada vez que se realice una corrección o una mejora de la web.

Estas fases describen el recorrido general para la creación y programación de una aplicación web. Las tecnologías y estándares más utilizados en la programación de aplicaciones web son HTML y JavaScript para la interfaz. Del lado del servidor, PHP, C#, Java (JavaEE con Web Beans o *servlets*) y ASP.NET.

La programación de una aplicación web es un proceso que conlleva el empleo de una gran variedad de tecnologías que deben integrarse, Por lo que su desarrollo suele estar dividido en la parte profesional en **dos tipos de especialidades:**

- **Front-end:** conlleva el desarrollo de todos los aspectos relacionados con el diseño de la aplicación web, la creación de todas las interfaces de los sitios web del dominio.
- **Back-end:** es lo que se entiende como programación del lado del servidor. Los programadores se encargarán de desarrollar toda la lógica de la aplicación que procesará todos los datos y los almacenará en la base de datos.







9

DESARROLLO DE APLICACIONES WEB HÍBRIDAS

9.1. REUTILIZACIÓN DEL CÓDIGO Y DE LA INFORMACIÓN

Hoy en día, los dispositivos de acceso a una aplicación web son muy diferentes y cada uno requiere unas determinadas características. El tipo de información que se visualizará se adaptará a las distintas plataformas y a las características *hardware* de cada uno.

Este tipo de aplicaciones se basan en la **combinación entre la versatilidad de un desarrollo web y la integración de la aplicación dentro de un dispositivo como si fuera nativa**. Es posible hacer uso de los lenguajes propios del desarrollo web, como HTML5, y aprovechar las funcionalidades del dispositivo tales como la cámara, el GPS o los contactos para interactuar con la aplicación web. Esto reduce el coste de desarrollo de una aplicación, puesto que una aplicación nativa para cada dispositivo elevaría dicho coste. **Una única aplicación web es interpretada por los diferentes dispositivos a través de su navegador**.

Por el contrario, hay algunos aspectos, como **el rendimiento del dispositivo, que se pueden ver afectados**, pues las aplicaciones web no pueden gestionar los recursos internos de los dispositivos, ventaja que las aplicaciones nativas sí pueden ofrecer.

Cabe destacar que **la reutilización de código es un proceso que se debe contemplar durante todo el desarrollo de la aplicación**. Todas aquellas funciones o partes de código que sean susceptibles de ser reutilizadas en un futuro deben ser correctamente **documentadas** y detalladas.

Para lograr esto, existen diferentes **mecanismos**:

- Incluir en un mismo fichero todas las constantes.
- Encapsular en funciones las partes de código que se deben usar en varios sitios de la misma aplicación.
- Hacer uso de comentarios o mecanismos de documentación de código.
- Utilizar librerías externas que provean de funciones que cumplan con la funcionalidad requerida.

CONCEPTO

La **reutilización de código** permite un desarrollo de código más limpio y mejor estructurado. Esto posibilita una mayor escalabilidad de la aplicación y la integración con cualquier tipo de seguridad web.

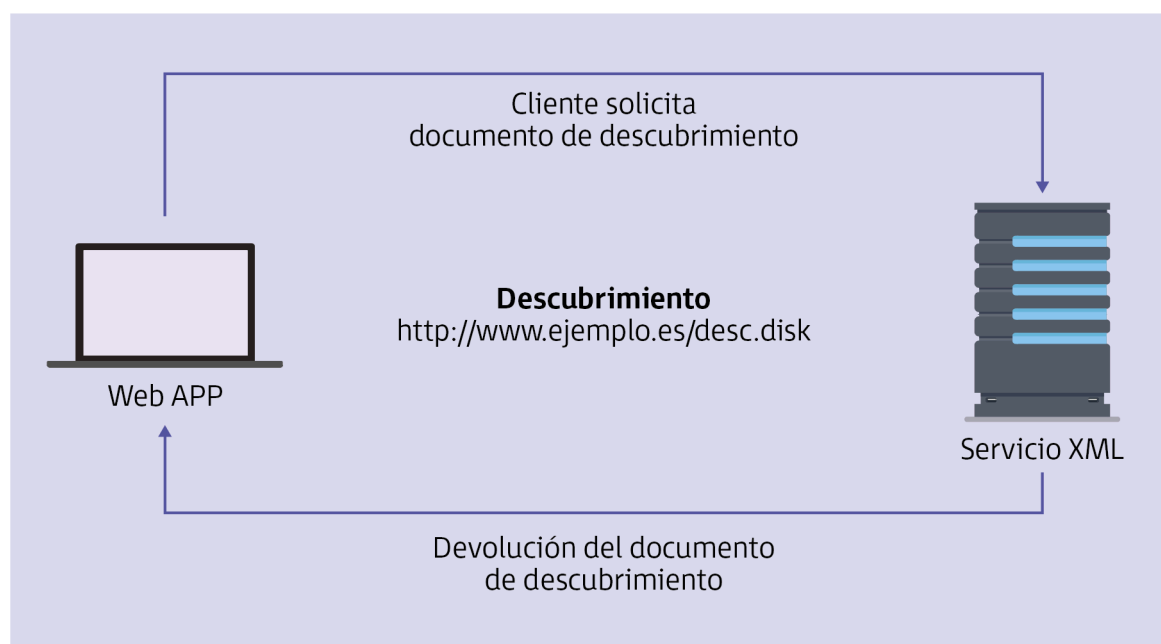
A modo de conclusión, el uso de aplicaciones híbridas facilita llegar con un menor coste a un gran número de clientes, reduciendo también la complejidad y especialización de los desarrolladores y diseñadores y permitiendo, a su vez, el acceso a todos los dispositivos independientemente del sistema operativo y características.

9.2. UTILIZACIÓN DE INFORMACIÓN PROVENIENTE DE DEPÓSITOS

El **catálogo de negocios de internet UDDI** proporciona un método para poder obtener y publicar información sobre diferentes sitios web. El objetivo que persigue esta organización es la creación de una plataforma donde se publiquen servicios web.

Este **proceso** se divide en diferentes etapas:

1. **Buscar un servicio de directorio:** antes es necesario ubicar el servicio web en XML dentro de UDDI.
2. **Descubrimiento de un servicio:** una vez que se ha publicado el servicio, UDDI vincula una dirección a ese servicio de forma que este sea localizable y accesible dentro de UDDI. En ese archivo dentro de UDDI se guardarán los enlaces a los recursos especificados de este servicio.
3. **Conocer la URL del servicio:** dentro de este repositorio, cada cliente busca un determinado servicio, después UDDI le indica dónde se puede encontrarlo. Una vez encontrado el archivo, es posible conocer la URL asociada al servicio.





CONCEPTO

Los **servicios web** se comunican mediante el uso de mensajes bajo el protocolo HTTP y mediante el uso del lenguaje XML.

Dentro de ese archivo XML se puede encontrar la **descripción asociada al servicio**. Si esto no cumple con la descripción del servicio publicada, no es posible realizar la comunicación.

Para realizar las conexiones, es necesario establecer un formato de conexiones, puesto que es posible modificar el protocolo HTTP por otro. No es obligatorio, aunque sí es recomendable, ya que es un protocolo ya utilizado de forma global y fiable.

Existen **dos tipos de formatos de conexión**:

- **HTTP**: es el más común y recomendado, mediante el uso de los métodos GET y POST es posible el envío de los parámetros necesarios junto con la petición realizada por la URL.
- **SOAP**: protocolo simple y ligero basado en el lenguaje XML. El mensaje es enviado separado del resto de parámetros, lo que añade seguridad y añade complejidad al servidor.

UDDI permite el uso centralizado de una gran cantidad de servicios web, que además son comprobados previamente. Una vez que se ha entendido y detallado el funcionamiento de este tipo de repositorios de información, el último paso sería crear un nuevo repositorio.



ponte a prueba

El objetivo que persigue el catálogo de negocios de internet UDDI es la creación de una plataforma donde se publiquen servicios web.

- a) Verdadero
- b) Falso

9.3. CREACIÓN DE DEPÓSITOS

La **creación de repositorios** para el uso de distintos servicios web dentro de un solo dominio aún es algo que no está ampliamente aceptado. La mayor parte de las empresas, cuando crean un determinado servicio, lo acaban publicando dentro de su propio dominio.

Los servicios web son relativamente nuevos, por lo que UDDI aún mantiene la esperanza de que sea posible unificar estos servicios en un mismo repositorio en un futuro, cuando el uso de los servicios web termine de estar asentado dentro de las páginas web.

La creación de repositorios encargados de buscar estos determinados servicios, por tanto, es algo que aún no se ha planteado.

Hay que pensar que el objetivo principal de estos servicios web es comunicar las diferentes plataformas, pero esto es algo que nace de la necesidad propia de cada empresa, por lo que su desarrollo es implantado directamente en su propia aplicación.

9.4. INCORPORACIÓN DE FUNCIONALIDADES ESPECÍFICAS

El resultado obtenido tras la creación de una web, junto con sus servicios, debe ir acompañado de una documentación. **Esta documentación contiene una serie de métodos que indican de qué forma es posible comunicarse con una aplicación externa. Esto es lo que se conoce como una API.** Una API es una interfaz de programación que facilita y provee el acceso a los métodos creados dentro de un determinado sitio web.

Además, es posible que la API incluya una serie de especificaciones que indiquen cómo manejar todos los tipos de datos que se tendrán dentro de la aplicación. El uso de estas API, fundamentalmente, está destinado a los desarrolladores de páginas web. Cada API está escrita en un lenguaje de programación concreto, por lo que, para hacer uso de ella, es necesario usar la sintaxis especificada por dicho lenguaje.

El objetivo es conseguir que estas API permitan reconocer el máximo número de lenguajes posibles, independientemente del lenguaje que ha sido empleado para su creación.

Todo lenguaje de programación provee una serie de funciones. Estas funciones han sido creadas con un propósito específico, pero es posible que ninguna de ellas permita satisfacer las necesidades del servicio. Por eso, durante

alguna de las etapas de desarrollo, será necesario crear mecanismos para desarrollar esta funcionalidad. Estas nuevas funciones creadas por los programadores permiten extender las posibilidades de un servicio.

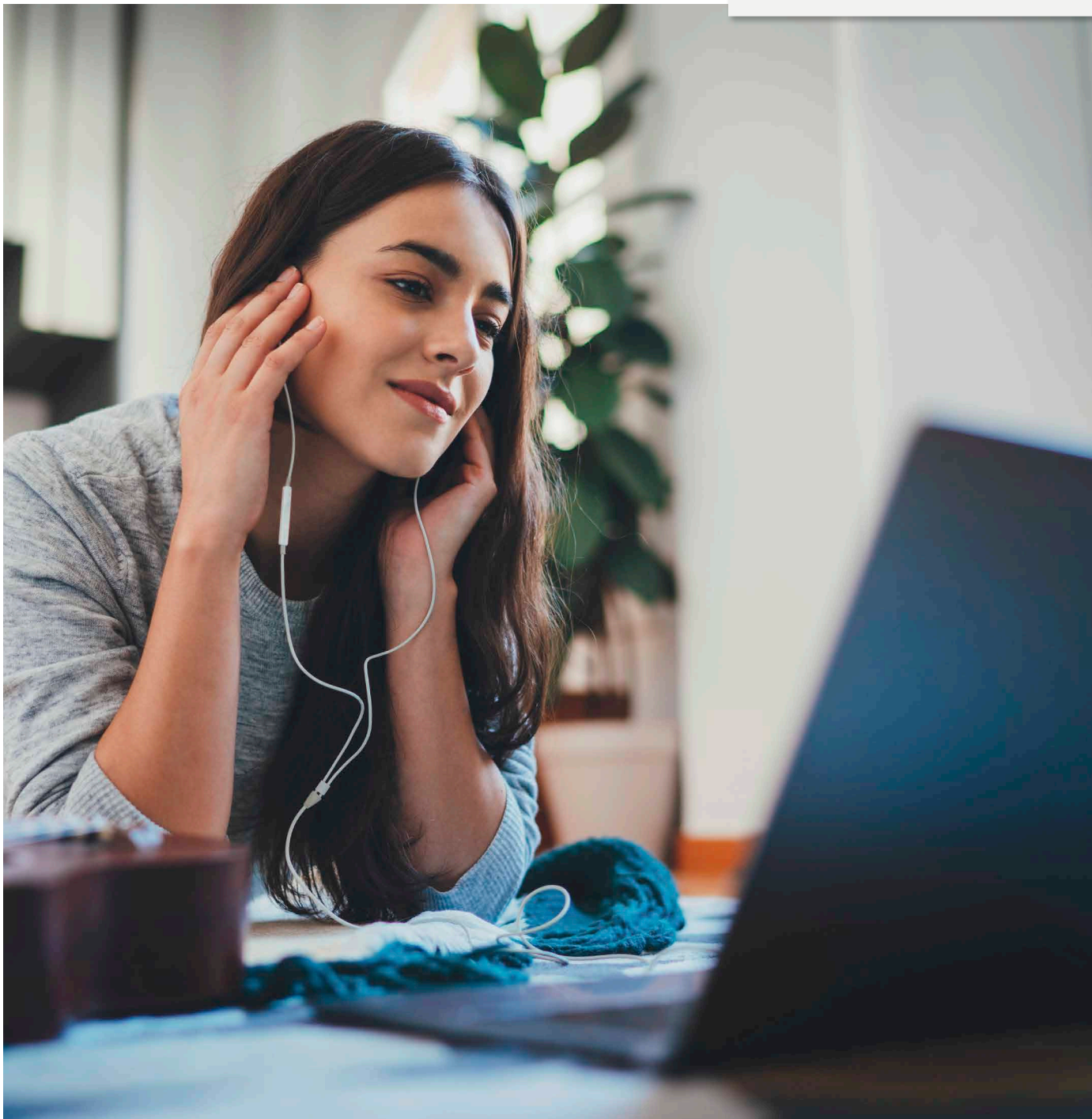
El **auge de las redes sociales** ha permitido a grandes empresas hacer uso de las API como un mecanismo para facilitar que otro tipo de páginas web integren contenido de páginas web externas, incluso permiten que un programador utilice una plataforma de acceso que es ofrecida por un servicio de otra web, como, por ejemplo, verificar la identidad de un usuario.



ponte a prueba

Verdadero o Falso: Una API es una interfaz de programación que facilita y provee el acceso a los métodos creados dentro de un determinado sitio web.

- a) Verdadero
- b) Falso





9.5. PROGRAMACIÓN DE SERVICIOS Y DE APLICACIONES WEB

En primer lugar, para poder comprender las **diferencias entre un servicio web y una aplicación web**, se va a definir cada uno de ellos:

- **Servicio web:** es una tecnología que, a partir del uso de una serie de protocolos, actúa como canal de comunicación entre dos plataformas diferentes entre sí. Los servicios web permiten el intercambio de mensajes entre un cliente y un servidor independientemente del lenguaje de programación de cada uno de ellos.
- **Aplicación web:** tipo de *software* que se codifica en un lenguaje capaz de ser interpretado por los navegadores que, posteriormente, se encargan de su ejecución.

Los servicios web están muy relacionados con las aplicaciones web, pero no son lo mismo. Una aplicación web puede contener un número determinado de servicios, por lo que la existencia de un servicio está directamente relacionada con la existencia de una aplicación web donde se ejecute. Es importante entender que la ejecución de un servicio viene determinada por la petición de un recurso por parte del cliente.

La creación de un servicio web es una tarea compleja si se realiza completamente por líneas de código. Estos servicios web se encargan de convertir los datos recibidos por parte del cliente a un lenguaje que pueda ser interpretado por el servidor. Esto implica que se deben tener en cuenta todas las normas y reglas de uso de los protocolos que intervienen durante el proceso.

El continuo crecimiento de este tipo de servicios requiere de mecanismos que permitan crear un servicio web de una forma más sencilla. Para ello, algunos de los IDE de desarrollo de aplicaciones web permiten crear y publicar un servicio a través de una interfaz que abstrae a los programadores de esa complejidad.

9.6. PRUEBAS, DEPURACIÓN Y DOCUMENTACIÓN

CONCEPTO

Las **pruebas** son el mecanismo por el cual es posible comprobar si una aplicación web cumple con las condiciones y requerimientos especificados.

Las pruebas forman parte de una fase dentro del ciclo de vida de la creación de estas aplicaciones, por lo que se deben integrar y realizar de manera repetitiva para poder detectar a tiempo el mayor número posible de errores.

Muchas veces, es bueno tratar de dividir el código en funcionalidades, intentando aislar unas partes de otras para que, en caso de producirse algún fallo, este no sea arrastrado de una parte del código a otra. Esto es lo que se conoce como pruebas unitarias, en las que solamente se ejecutada una parte del código, de manera que el resto de las funcionalidades de la aplicación no se verán afectadas. La mayor parte de los lenguajes de programación web permiten este tipo de pruebas.

Durante el desarrollo de cualquier tipo de aplicación o servicio web, y de forma paralela a las pruebas, es conveniente realizar procesos de depuración. Estos procesos permiten comprobar y realizar un seguimiento de la ejecución en tiempo real de dicha aplicación o página web. De esta forma, es posible comprobar los recursos utilizados por la aplicación. Analizar los valores que van tomando las variables permite verificar que son correctas y controlar el flujo de ejecución.

Para poder realizar estos procesos de depuración, es necesario hacer uso de los IDE que soporten el lenguaje de programación utilizado, algunos de los que permiten esto son Eclipse y Microsoft Visual Studio.

BIBLIOGRAFÍA / WEBGRAFÍA

- ☐ Beáti, H. (2012). *El gran libro de PHP. Creación de páginas web dinámicas*. Barcelona, Marcombo.
- ☐ López, M. et al. (2012). *Programación web en entorno servidor*. Madrid, Ra-Ma.
- ☐ Vicente, J. L. (2014). *Programación web en entorno servidor*. Madrid, Garceta.

solucionario

1.3. Lenguajes de programación en entorno web

¿Qué es el HTML5?

d) Un lenguaje de marcas para definir el contenido de una página web.

1.6. Integración con los lenguajes de marcas

El objetivo de CGI es hacer más compleja la interacción de los usuarios con las webs.

b) Falso

2.2. Tecnologías asociadas: PHP, ASP, JSP, miniaplicaciones de servidores (servlets), entre otras

Verdadero o falso: El JSP es un lenguaje interpretado que para realizar peticiones al servidor y devolver la respuesta al navegador a través de una URL.

b) Falso

2.3. Etiquetas para inserción de código

¿Cuáles son las etiquetas que abren y cierran un fragmento de código PHP dentro de un documento HTML?

c) `<?php y ?>`

3.1. Tomas de decisión

Las estructuras condicionales permiten controlar y gestionar todas y cada una de las ejecuciones que realizará la aplicación web.

a) Verdadero

3.2. Bucles

¿Cuál es el mejor uso de la instrucción *while*?

d) Ejecutar un bucle condicional

3.5. Recuperación y utilización de información proveniente del cliente web

Verdadero o Falso: Aunque los métodos GET y POST codifican de idéntica forma la información, el primero sirve para solicitarla y el segundo para enviarla.

b) Falso

4.1. Mantenimiento del estado

Las sesiones permiten garantizar y controlar que los recursos son ejecutados de manera correcta.

a) Verdadero

4.3. Seguridad: usuarios, perfiles, roles. Cookies

¿Qué es una ACL?

c) Una lista de permisos

4.6. Pruebas y depuración

Verdadero o Falso: Las pruebas unitarias son aquellas que se llevan a cabo validando la parte lógica.

b) Falso

5.1. Mecanismos de separación de la lógica de negocio

¿Cuál de las siguientes capas no forma parte de una web según el patrón Modelo-Vista-Controlador?

d) Programación

5.2. Controles de servidor

Verdadero o Falso: Los controles de servidor web forman parte de la capa encargada de procesar las peticiones de usuario y solicitar la información requerida al servidor.

b) Falso.

5.5. Pruebas y depuración

En ningún caso se aconseja tratar de dividir el código en funcionalidades.

b) Falso.

6.3. Ejecución de sentencias SQL. Recuperación y edición de información

¿Qué instrucción SQL se utiliza para realizar modificaciones en un conjunto de datos almacenados en una tabla?

d) Update



solucionario

6.6. Transacciones

¿Cómo se denomina el mecanismo de SQL que permite eliminar todas las operaciones realizadas?

b) **ROLLBACK TRAN**

7.1. Transacciones

Verdadero o Falso: Un servicio web se basa en el uso de protocolos que permiten el intercambio de información entre aplicaciones mediante Internet, pero que, al haber sido desarrollados por diversas empresas, no se acogen a ningún estándar.?

b) **Falso**

7.3. Mecanismos y protocolos implicados

¿Cuál de estos protocolos no se utiliza en ningún servicio web?

b) **UDP**

7.6. Interfaz de un servicio web. Pruebas

El tiempo de respuesta no es uno de los puntos clave de las pruebas del servicio creado.

b) **Falso**

8.2. Librerías y tecnologías relacionadas

¿Cuál de las siguientes opciones no es una tecnología que determina cómo se programa y cuáles son los requisitos de ejecución de páginas web dinámicas?

d) **ATLAS**

8.5. Modificación dinámica de la estructura de la página web

En una web dinámica, ¿cómo se denomina el componente encargado de transformar la información del usuario en la obtención de la información asociada a dicha petición?

d) **Intérprete**

9.2. Utilización de información proveniente de depósitos

El objetivo que persigue el catálogo de negocios de internet UDDI es la creación de una plataforma donde se publiquen servicios web.

a) **Verdadero**

9.4. Incorporación de funcionalidades específicas

Verdadero o Falso: Una API es una interfaz de programación que facilita y provee el acceso a los métodos creados dentro de un determinado sitio web.

a) **Verdadero**